CS201 QUIZ 3 WITH MOAZ,JUNAID FILES Final term preparation 2022

Introduction to Programming CS201

© Copyright Virtual University of Pakistan

Lecture No. 30

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 3

3.17

Summary

1) Reference data type
2) Example 1
3) Difference Between References and Pointers
4) Dangling References
5) Example 2

Reference data type

Out today's topic is about references. This is a very important topic from the C++ prospective. Today we will see what is a reference, how can we use them. C++ defines a thing by which we can create an alias or synonym of any data type. That synonym is called reference. How do we declare a reference? We declare it by using & operator. Now it is little bit confusing. We have used & as address-of operator and here we are using it for referencing. We will write as

```
int &i;
```

It means that *i* is a reference to an integer. Keep that statement very clear in your mind. It is easier to read from right to left. A reference is a synonym. If we want to give two names to same thing then we use reference. Reference has to be initialized when it is declared. Suppose if we have an integer as *i* and we want to give it second name. We will reference it with *j* as:

```
int &j = i;
```

We declared an integer reference and initialized it. Now *j* is another name for *i*. Does it mean that it creates a new variable? No, its not creating a new variable. Its just a new name for the variable which already exists. So if we try to manipulate *i* and *j* individually, we will came to know that we have been manipulating the same number.

© Copyright Virtual University of Pakistan

Lets take a look at a very simple example. In the main function we take an int variable *i* and then we write *int &j = i;* Now we assign some value (say 123) to *i*. Now display the value of *i* using *cout*. It will show its value as 123. Display the value of *j* using *cout*. We will not use & operator to display the value of *j*. We will only use it at the time of declaration and later we don't need it. The & is not reference operator rather it acts as reference declarator. The value of *j* will be same as of *i* i.e. 123.

```
int i;
int &j = i;
i = 123;
cout << "\n The value of i = " << i;
cout << "\n The value of j = " << j;
```

Now what will happen if we increment *i* as *i++;* and print the values of *i* and *j*. You will note that the value of *i* and *j* both have been incremented. We have only incremented *i* but *j* is automatically incremented. The reason is that both are referring to the same location in the memory. *j* is just another name for *i*.

What is the benefit of reference and where can we use it? References are synonyms and they are not restricted to int's, we can have reference of any data type. We can also take reference of a class. We wrote a function to show the use of pointers. That function is used to interchange two numbers. If we have two integers *x* and *y*. We want that *x* should contain the value of *y* and *y* should get the value of *x*. One way of doing this is in the main program i.e.

```
int x = 10;
int y = 20;
int tmp;
tmp =  y;
y = x;
x = tmp;
```

The values of both *x* and *y* have been interchanged. We can also swap two numbers using a function. Suppose we have a swap function as *swap(int x, int y)* and we write the above code in it, what will happen? Nothing will be changed in the calling program. The reason is call by value. So when the main function calls the function *swap(x, y)*. The values of *x* and *y* will be passed to the swap function. The swap function will get the copies of these variables. The changes made by the swap function have no effect on the original variables. Swap function does interchange the values but that change was local to the swap function. It did not effect anything in the main program. The values of *x* and *y* in the main program remains same.

We said that to execute actual swap function we have to call the function by reference. How we did that. We did not send *x* and *y* rather we sent the addresses of *x* and *y*. We used address operator to get the addresses. In the main function we call swap function as *swap(&x, &y);* In this case we passed the addresses of two integers. The prototype of the swap function is *swap(int*, int*)* which means that swap function is expecting pointers of two integers. Then we swap the values of *i* and *j* using the * notations. It works and in the main program, the values are interchanged. This was a clumsy way. We can use reference in this case with lot of ease. Let us see

© Copyright Virtual University of Pakistan

how we can do that. Lets rewrite the swap function using references. The prototype will be as:

```
swap (int &i, int &j);
```

Swap is a function that is expecting *i* as a reference to an integer and the second argument is *j* which is also a reference to an integer. The calling function has to pass references. What will we write in the body of the function? Here comes the elegance of the references. In the body we will treat *i* and *j* as they are ordinary integers. We will take a temporary integer and interchange the values of *i* and *j*.

```
swap (int &i, int &j)
{
        int  temp;
        temp = x;
        x = y;
        y = temp;
}
```

In the main program, you will see that the values of two integers have been interchange. What is the way to call this function? In the main program, we will call this function as *swap(x, y)*. Its an ordinary call but the function is expecting addresses which is automatically done. The memory locations of the integers are passed and the function is interchanging the original numbers. This is one beautiful example in which we avoided all the cumbersome of pointer notation. What is the downfall of this? As nothing comes for free. In this case when you are reading the main function you will see *swap (x, y)* which seems a call by value. This is a rule of C/C++ that when we pass two variables to some function they are passed by values. You will have to look for the definition of swap function to realize that it is not call by value but is call by reference. Second thing is if we have another swap function, which is receiving two integers. Can we define two functions as *swap(int x, int y)* and *swap(int &x, int &y)*? One function is receiving two integers and other is receiving two references of integers. Can we do that? Types are different so we can overload. Unfortunately not, in the main function the way to call both functions is same i.e. *swap(x, y)*. How does the compiler know that which functions is being called? There is no way for the compiler to find out. Therefore there is an ambiguity and that is not allowed. The only thing to realize is the side effect. Side effects are critical to take care of whenever you are doing call by reference. Here in this example we do want that two numbers should be interchanged. There may be some situation where we want to send the references and don't want that original data should be affected. These situations arise when we want to pass a large data structure to a function. To understand this we have to understand how the function call executes. We have discussed it before, now lets recap it. In real world, suppose I am reading a book. While reading I notice a word which I think I have to look up. I stop reading, markup the page and then look that word in dictionary or encyclopedia. While reading the definition of that word I look another special word which I need to lookup. I put a marker here and go for looking the definition of that new word. Eventually I understand all the words I need to look up. Now I want to go to the point at which I left the study. I close the dictionary or encyclopedia and goes back to the original book which I was studying. Now I understand all the words in my study and continue the study from the point where I

© Copyright Virtual University of Pakistan

left. If we think about it, it was a function call. We were doing some work, suddenly we call a function and stop that work and execution went to the function. When the execution of the function came to end, we came back to our calling function and continued with it. Computers do the same work with stack. So when the program comes back from the function it should know the point at which it lefts. We supposed here a word to look up, now consider that it was a paragraph or essay I am going to look up. Now to lookup that essay in other books I have to take the entire paragraph or essay with me to that book. Think about the stack. On the stack, the original condition of the program (state) has saved. Now we put our essay or paragraph on it and then opened the other book and searched the book for this essay. In this way, we want to explain that the thing we passed to the function from the main was itself a huge/large thing (as we resemble it with paragraph or essay). So there was a big overhead in writing that thing out into a temporary space in memory and then picking it up and looking it up.

We can make this process more efficient. The issue is that in this example we do not want to change the paragraph or essay which we are going to look up. We only want to look it up. We want only to use it but don't want to change its words. Unfortunately the baggage that comes with doing this is that first make a copy of this (essay) then go with this copy and when the work with it ends, leave (through away) the copy and start the original work. This is inefficient.

But if we took the reference of that essay and passed the address of it and went to the function to look it up. There is a danger that comes with the address, that is while looking up that essay I underlined different words and when I came back to original book I saw that these line marks were also there. Thus we passed something by value rather we passed something by reference. By passing the reference, we actually pass the original. Think about it in another way. We go to a library and said the librarian to issue us a book, which we want to take home for study. Suppose, that book is the only one copy available in the library (or in the world). The librarian will not issue the book. Because it is the only copy available in the world. He does not want to issue this original book to someone as someone can marks different lines with a pen and thus can damage the original book. The librarian will do that he will take a photocopy of that book and issue it. Making a photocopy of the book and then take the book is a bothersome work.

Here we don't want to damage the book. We just want to read it. But can I somehow take the original book? Put it in a special polythene bag and give it to you in such a way that you can read it without damaging it. By doing this we get efficiency but danger is still there. This is actually a call by reference. We have the reference (the original book). If we do something to the original book, the library book will be damaged. Can we somehow prevent this from happening? And also have the efficiency of not having to make a copy.

Now come back to the computer world. Suppose we have a data structure. There is a string of 1000 characters in it. We want to pass that data structure to a function. If we pass it by value which is sake, the original structure will not be affected. We first will copy that string of 1000 characters at some place, which is normally made on the stack. Then the function will be called. The function will take the copy of these 1000 characters and will manipulate it. Then it will give the control back to the caller

© Copyright Virtual University of Pakistan

program and will destroy that copy of string. For efficiency, we want that instead of making a copy of this string, its reference should be written. We have been doing this with pointers and addresses. So we write there the address and pass it to the function. How we can prevent the side effects? There may be these side effects with references. So be very careful while using references with function calls.

Can we do something to prevent any changes? The way we do it is by using the *const* key word. When we write the *const* key word with the reference, it means that it is a reference to some thing but we cannot change it. Now we have an elegant mechanism. We can get the efficiency of call by reference instead of placing a string of 1000 characters on the stack, we just put the address of the string i.e. reference on the stack. In the prototype of the function, it is mentioned that it takes a *const*. This is a reference that may be to a char, int, double or whatever but it is a *const*. The function cannot change it. The function gets the address, does its work with it but cannot change the original value. Thus, we can have an efficiency of a call by reference and a safety of a call by value. To implement all this we could have used the key word *const* with an address operator or a pointer but we can use a reference that is an elegant way. There is no need in the function to dereference a reference by using * etc, they are used as ordinary variable names.

Example 1

Now let us have an example. Here we defined a structure *bigone* that has a string of 1000 characters. Now we want to call a function by three different ways to manipulate this string. The first way is the call by value, which is a default mechanism, second is the call by reference using pointers and the third way is call by reference using reference variables. We declared the prototypes of these functions. Here we declared three functions. The first function is *valfunc* which uses a call by value. We simply wrote the value of the structure. The function prototype is as under.

```
void valfunc( bigone v1 );
```

The second function is *ptrfunc* in which we used call by reference using pointers. We passed a pointer to the structure to this function. The prototype of it is as follows.

```
void ptrfunc( const bigone *p1 );
```

The third function is *reffunc* which uses the way of calling by reference using references. We wrote its prototype as

```
void reffunc( const bigone &r1 );
```

Note that we wrote & sign with the name of the variable in the prototype of the function, we will not write it in the call of the function.

In the main program, we called these function. The call to the *valfunc* is a simple one we just passed the name of the object of the structure i.e. *v1*. As the function is called by using the call by value the manipulation in the function will not affect the original value. We wrote it as:

```
valfunc ( bo );
```

In this call a copy of *bo* is placed on the stack and the function uses that copy.

Next we called the function *ptrfunc*. We passed the address of the structure to *ptrfunc* by using the & operator. Here we are talking about the function call (not function prototype) and in function call we write *ptrfunc (&bo) ;* which means we passed the

© Copyright Virtual University of Pakistan

address of *bo* (the object of structure) to the function. The efficiency here is that it writes only the address of the object to the stack instead of writing the whole object.

The call to the third function *reffunc* is simple and looks like the call by value. There is no operator used in this call it is simply written as:

reffunc (bo) ;

Here we cannot overload the valfunc and reffunc, their names must be different. Otherwise the calls look same and become ambiguous.

The pointer call and reference call are sending the references to the original structures so these are dangerous. If we want to prevent the function from changing that then we should define the function by *const* keyword with its argument pointer or reference. Then the function can not modify the original value, it can only read it. So by this we get the efficiency of the call by reference and the safety of the call by value.

The complete code of the example is given here.

```
// Reference parameters for reducing overhead
// and eliminating pointer notation

#include <iostream.h>

// A big structure
struct bigone
{
   int serno;
   char text[1000];  // A lot of chars
} bo = {123, "This is a BIG structure"};

// Three functions that have the structure as a parameter
void valfunc( bigone v1 );                    // Call by value
void ptrfunc( const bigone *p1 );      // Call by pointer
void reffunc( const bigone &r1 );      // Call by reference

// main program
void main()
{
   valfunc( bo );                 // Passing the variable itself
   ptrfunc( &bo );                // Passing the address of the variable
   reffunc( bo );                 // Passing a reference to the variable
  }

//Function definitions
// Pass by value
void valfunc( bigone v1 )
{
   cout << '\n' << v1.serno;
   cout << '\n' << v1.text;
}
// Pass by pointer
void ptrfunc( const bigone *p1 )
```


© Copyright Virtual University of Pakistan

```
{
  cout << '\n' << p1->serno;     // Pointer notation
  cout << '\n' << p1->text;
}
// Pass by reference
void reffunc( const bigone &r1 )
{
  cout << '\n' << r1.serno;       // Reference notation
  cout << '\n' << r1.text;
}
```

Following is the output of the above program.

```
123
This is a BIG structure
123
This is a BIG structure
123
This is a BIG structure
```

Difference Between References and Pointers

The reference in a way keeps the address of the data entity. But it is not really an address it is a synonym, it is a different name for the entity. We have to initialize the reference when we declare it. It has to point to some existing data type or data value. In other words, a reference cannot be NULL. So immediately, when we define a reference, we have to declare it. This rule does not apply to functions. When we are writing the argument list of a function and say that it will get a reference argument, here it is not needed to initialize the reference. This reference will be passed by the calling function. But in the main program if we declare a reference then we have to initialize it. When a reference is initialized, we cannot reassign any other value to it. For example, we have *ref* that is a reference to an integer. In the program we write the line *int &ref = j ;*

Here *j* is an integer which has already been declared. So we have declared a reference and initialized it immediately. Suppose we have an other integer *k*. We cannot write in the program ahead as *ref = k;* Once a reference has defined, it always will refer to the same integer location as *j*. So it will always be pointing to the same memory location. We can prove this by printing out the address of the integer variable and the address of the reference that points to it.

In programming, normally we do not have a need to create a reference variable to point to another data member or data variable that exists, because creating synonym that means two names for the same thing, in a way is confusing. We don't want that somewhere in the program we are using *i* (actual name of variable) and somewhere *ref* (reference variable) for manipulating the same data variable. The main usage of it is to implement the call by reference through an elegant and clean interface. So reference variables are mostly used in function calls.

© Copyright Virtual University of Pakistan

The difference between pointers and references is that we can do arithmetic with pointers. We can increment, decrement and reassign a pointer. This cannot be done with references. We cannot increment, decrement or reassign references.

References as Return Values

A function itself can return a reference. The syntax of declaration of such a function will be as under.

datatype& function_name (parameter list)

Suppose we have a function *myfunc* that returns the reference to an integer. The declaration of it will be as:

int & myfunc() ;

Dangling Reference

The functions that return reference have danger with it. The danger is that when we return a value from such a function, that value will be reference to some memory location. Suppose that memory location was a local variable in the function which means we declare a variable like *int x;* in the function and then returns its reference. Now when the function returns, *x* dies (i.e. goes out of scope). It does not exist outside the function. But we have sent the reference of that dead variable to the calling function. In other words, the calling program now has a reference variable that points to nowhere, as the thing (data variable) to which it points does not exist. This is called a dangling reference. So be careful while using a function that returns a reference. To prevent dangling reference the functions returning reference should be used with global variables. The function will return a reference to the global variable that exists throughout the program and thus there will be no danger of dangling reference. It can be used with static variables too. Once the static variables are created, they exist for the life of the program. They do not die. So returning their reference is all right.

So, never return a reference to a local variable otherwise, there will be a dangling reference. Some compilers will catch it but the most will not. The reason is that the function that is returning a reference has defined separately. It does not know whether the reference is to a global or local variable, because we can do many manipulations in it and then return it. But normally compilers will catch this type of error.

Example 2

Let us look at an example of functions returning references. First, we declare a global variable that is an integer called *myNum* and say it is zero. Then we declare a function *num* that returns a reference to an integer. This function returns *myNum*, the global variable, in the form of reference. So now when there will be a function call, the return of the function will be a reference to the global variable called *myNum*. Now we can write the main function. Here we write *myNum = 100* ; This assigns a value 100 to the global variable. Next we write

int i ;
i = num () ;

Now a reference to *myNum* is returned. We would want to assign a reference to a reference but we can use it as an ordinary variable. Thus that value is assigned to *i*.

© Copyright Virtual University of Pakistan

Now look at the next line which says *num () = 200* ; We know that the left hand side of the assignment operator can only be a simple variable name, what we called l-value (left hand side value). It cannot be an expression, or a function call. But here in our program the function call is on left hand side of the assignment. Is it valid? In this case it is valid, because this function called *num* is returning a reference to a global variable. If it returns a reference, it means it is a synonym. It is like writing *myNum = 200* ; The example shows that it can be done but it is confusing and is a bad idea. We can put a reference returning function on the left hand side of an assignment statement but it is confusing and bad idea.

Following is the code of the example.

```
/*Besides passing parameters to a function, references can also be used to return values from a function */

#include <iostream.h>

int myNum = 0;    // Global variable

int& num()
{
   return myNum;
}
void main()
{
   int i;
   i = num();
   cout << " The value of i = " << i << endl;
   cout << " The value of myNum = " << myNum << endl;
   num() = 200;  // mynum set to 200
   cout << " After assignment the value of myNum = " << myNum << endl;
}
```

Following is the output of the program.

```
The value of myNum = 0
 After assignment the value of myNum = 200
```

The references are useful in implementing a call by reference in an efficient fashion and writing the function very elegantly without using dereference operators.

We use & sign for declaring a reference. In the program code, how do we find out that it is a reference or an address is being taken? The simple rule is that if in the declaration line there is reference symbol (& sign) with the variable name then that is a reference declaration. These will be like *int &i, float &f* and *char &c* etc. In the code whenever we have simply &i, it means we are taking address. So it's a simple rule that when, in the code, we see a data type followed by & sign, it's a reference. And when the & sign is being used in the code with a variable name then it is the address of the variable.

In C and C++ every statement itself returns a value. It means a statement itself is a value. Normally the value is the value of left hand side. So when we write *a = b*; the

© Copyright Virtual University of Pakistan

value of *b* is assigned to *a* and the value of *a* becomes the value of the entire statement. Therefore when we write *a = b = c* ; first *b = c* executes and the value of *c* is assigned to *b*. Since *b = c* is a statement and this statement has the value of *b*. Now *a* takes the value of this statement (which happened to be *b*). So *a = b* also works. Similarly *a + b + c* also works in the same way that the value of *c* is added to *b* and then this result is added to *a*.

What happens when we write *cout << "The value of integer is " << i << endl* ;
Here first extreme right part will be executed and then the next one and so on or the other way. On the screen the "The value of integer is" displayed first and then the value of the *i* and in the end new line. So it is moving from left to right. When *cout* gets the first part i.e. "The value of integer is", this is a C statement. When this will be executed, the sentence "The value of integer is" is displayed on the screen. But what will be its value? That has to do something with the next << part and is needed with this << sign. We know that we need *cout* on the left side of << sign. So actually what happened is when the first part of the statement is executed. When the statement *cout << " The value of integer is"* executed *cout* is returned. The next part is *<< i* and it becomes *cout << i;* the value of *i* is printed and as a result of the statement *cout* is returned again which encounters with *<< endl;* and a new line is inserted on the screen and *cout* is returned as a result of the statement execution. The return of the complete statement remains *cout*. The *cout* is stream, it does not have value per se. The reference to the stream is returned. The same reference which we have discussed today. The same thing applies to operators like +, -, *, /. This will also apply to = (assignment operator) and so on. We will be using lot of reference variables there.

Summary

We have learned a new data type i.e. reference data type. We said that reference is synonym or alias for another type of data. Take int's synonym or double's synonym. In other words, it's the second name of a variable. Then we talk about some do's and dont's. Normally we do not use two names for the same variable. It's a bad idea and leads to confusing the programmer. Then we found the most useful part of using a reference. If we have to implement call by reference with function then using the prototype of the function which is expecting references and it leads to clean programming. You use the names of the arguments without using any dereferencing operator like *. The most useful part is implementing the call by reference. Then we looked at the difference of pointers and references. We cannot increment the reference variable. Arithmetic is not allowed with references but most importantly, reference variables must be initialized when they are declared. This is import. We can declare pointers and later can assign it some value. The use of reference with classes will be covered later. We have also seen a preview of the usage of references. In that preview we have learned new things that every statement itself has some value and that value is returned. Use it or not it's a different issue. We call a function on a single line like *f(x);* may be *f(x)* returns some value and we did not use it. Not a problem. Similarly if we say *a = b;* this statement itself have some value whether we use it or not. Then we see how the cout statement is executed. Every part of the statement returns some value which is the reference to *cout* itself. It becomes the reference to the stream.
How these references will be declared and used? We will cover this with operator overloading. Try to write some programs using references and implement call by reference using references instead of pointers.

© Copyright Virtual University of Pakistan

Tips

- The use of reference data type is the implementation of call by reference in an elegant way.
- We cannot do arithmetic with references like pointers.
- Reference variables must be initialized immediately when they are declared.
- To avoid dangling reference, don't return the reference of a local variable from a function.
- In functions that return reference, use global or static variables.
- The reference data types are used as ordinary variables without any dereference operator.

Lecture No. 31

Reading Material

Deitel & Deitel - C++ How to Program ***Chapter 8***

8.2, 8.3, 8.4, 8.6, 8.7

Summary

- Lecture Overview
- What is Operator Overloading and Why is it Required
- Where is it Relevant to Apply
- Operators to Overload
- Restrictions on Operator Overloading
- Examples of Operator Overloading
- Non-member Operator Functions
- Example Program 1
- Example Program 2
- Tips

Lecture Overview

The topic of this lecture is **Operator Overloading**. In previous lectures, we discussed about it a bit while discussing about references. So we will see in detail what is operator overloading, how to overload operators, where it is relevant to apply and what are the restrictions on it.

© Copyright Virtual University of Pakistan

What is Operator Overloading and Why is it Required?

Operator overloading is to allow the same operator to be bound to more than one implementation, depending on the types of the operands.

As you know that there are standard arithmetic operators in C/C++ for addition (+), subtraction (-), multiplication (*) and division (/). We should only use these operators for their specific purposes. If we want to add two **int**s, say **i** and **j**, the addition will take place in the following manner i.e. **i + j**. To add two **double** numbers, we use the same operator and write **d1 + d2**. We may add two floats with the help of the same operator as **f1 + f2**. Similarly other operations of **-**, ***** and **/** on the primitive types (sometimes called as native or built-in types) can be employed. In other words, these operators are already overloaded for primitive types in C++. But these C++ operators cannot be used for classes and their objects. We have to write our own operator functions that can work with objects.

Let's take an example of complex numbers. There are two parts of a complex number i.e. **real** and **imaginary**. As complex numbers are part of mathematical vocabulary, so the mathematical manipulations are done on them like addition, subtraction and multiplication. Suppose, we write our own class for complex numbers named **Complex**, but we can't add two complex numbers **c1** and **c2** as **c1 + c2** because until now we don't know how to write it. Although, we are able to write a function say **cadd()** to serve this purpose.

```
Complex cadd ( Complex c1, Complex c2 ) ;
```

It accepts two complex numbers as parameters and returns back the resultant complex number. But the usage of this function to add two complex numbers is generally clumsy. It gets more cumbersome and complex if we want to carry out cascading operations like

i + j + k . It is better to use the standard operators of **+**, **-**, ***** and **/** as they are more readable and elegant.

Where is it Relevant to Apply?

Firstly, the operator overloading gets relevant whenever there is the application of the mathematical functions of addition, subtraction, multiplication and division. Complex number is one example of it. As discussed earlier, in case of **Date** class, the operators can be effectively used to get the future or past dates.

Secondly, the operators are also used sometimes in case of non-mathematical manipulation. The example of **String** class to manipulate strings help us understand it in a better way. The operator **+** can be used to concatenate two strings. Previously, we used **strcat()** function declared inside **string.h** header file to concatenate two strings. As compared to **strcat()**, the use of **+** to concatenate two strings is definitely easier and more readable. But there is a little bit cost associated with this process of operators overloading.

The cost is involved whenever we overload an operator. We have to write a function and make use of the operator semantics correctly while implementing the function.

© Copyright Virtual University of Pakistan

This means that the function written to overload + operator should do addition or concatenation of strings in case of String objects.

Operators to Overload

There are two types of operators to overload:

1. Unary
2. Binary

Unary operators are the ones that require only one operator to work. Unary operators are applied to the left of the operand. For example, ^, **&**, ~ and **!**.

Binary operators require two operands on both sides of the operator. **+**, **-**, *****, /, **%**, **=**, **<** and **>** are examples of binary operators.

The complete list of C++ operators that can be overloaded is as follows:

+	-	*	/	%	^	&
|	~	!	=	<	>	+=
-=	*=	/=	%=	^=	&=	|=
<<	>>	>>=	<<=	==	!=	<=
>=	&&	||	++	- -	-> *	,
[ ]	( )	new	new[ ]	delete	delete[ ]	

The following operators can't be overloaded.

.	:	::	.*	?	sizeof

Let's start with operator overloading mechanism. Consider an object **date** of the **Date** class. The data member **day** can be accessed as follows:

date.day = 2;

In this statement, the **day** data member of the **date** object is accessed and assigned value **2**. This expression (**date.day**) is driven by the object name at left.
Similarly, while using operators, the statement like **a + b** is driven by the object at the left. In this case, + operator function for the object **a** will be called and **b** object is passed explicitly to the + operator function as an argument. The rules of function overloading are applied to the operator overloading. We cannot write two + operator functions with exactly identical parameters. Following the overloading rules, the two operator functions have to be different by the type or number of arguments.

The syntax of the prototype of the overloaded operator function is:

return-type operator *operator-symbol* *(parameter-list);*

operator is the keyword here. An example of this will be as follows:

Complex operator + (Complex &);

© Copyright Virtual University of Pakistan

We sometimes write only operator to refer to the operator function in our discussion.

Restrictions on Operator Overloading

There are some restrictions on operator overloading.

- The operator overloading functions for overloading **()**, **[]**, **->** or the assignment (=) Operators must be declared as class members.
- The arity (number of operands) cannot be changed. If you are overloading an operator that requires two operands e.g. *****. It cannot be used as a unary operator that requires one operand.
- No new operators can be created. Like in Fortran language, we have ****** as 'raise to the power (exponent) operator' but this operator does not exist in C++. Therefore, it can't be overloaded. Hence, only existing operators of C++ are used.
- Overloading can't be performed for the built-in (sometimes called primitive or native) data types. For example, we cannot change how two **int**s are added. That means that operators are overloaded to use with defined data types like classes.
- Precedence of an operator cannot be changed. For example, the ***** has higher precedence than **+.** This precedence cannot be changed.
- Associativity of an operator cannot be changed. If some operator is right associative, it cannot be changed to be left associative.

Examples of Operator Overloading

Let's take the complex number's class **Complex** and define a **+** operator function.

We know that when we write the following line:

```
x = y + z ;
```

y and **z** operands are take part in the addition operation but there is no change in them due to this operation. This is the **+** operator's functionality. The resultant is being assigned to the variable **x**. This is assignment operator's functionality.

Now we will discuss a little bit about the assignment operator as well. Let's say we write the following statement for two complex numbers **c1** and **c2**.

```
c1 = c2 ;
```

Here **c2** is being assigned to **c1**. Will this assignment work when we have not written any assignment operator function for complex number? Apparently, it looks that the statement will produce a compilation error (as there is assignment operator defined by us) but this is not true. Whenever, we write our own class and compile it, the compiler automatically generates a default assignment operator. The default assignment operator makes a member to member assignment. This works fine unless there is a pointer data member inside our class and that pointer is pointing to some data inside memory. For that case (when there is a pointer data member) we have to write our own assignment operator otherwise the default assignment operator works fine for us. That will be discussed in the subsequent lectures.

© Copyright Virtual University of Pakistan

By definition of addition of complex numbers, we know that whenever two complex numbers are added, the real part of one number is added into the real part of other number. Similarly, the imaginary part of one number is added to the imaginary part of the other number. We also know that when a complex number is added to another complex number, the resultant is also a complex number consisting of real and imaginary parts. This addition of real, imaginary parts and return of resultant complex number is the functionality of the **+** operator function we are going to write.
Another thing to decide for this + operator is whether this operator will be a member operator or a **friend** operator. Normally, operators are member operators but there are situations when they cannot be member operators. In case of member operator, following is the syntax of its prototype:

```
Complex operator + (parameter-list);
```

For member operator, the object on the left side of the **+** operator is driving this **+** operation. Therefore, the driving object on the left is available by **this** pointer to **+** operator function. But the object on the right is passed explicitly to the **+** operator as an argument.
We can define a member operator as under:

```
1.  Complex Complex :: operator + (Complex c)
2.  {
3.      Complex temp ;
4.      temp.real = real + c.real ;
5.      temp.imag = imag + c.imag ;
6.  return temp ;7. }
```

Let's see this code line by line.

Line 1 indicates that the return type is **Complex**, it is an **operator +** function and it is accepting a **Complex** object by value as an argument.
In line 3, a local **Complex** object is declared, called **temp**.
In line 4, **real** part of the calling object (that is the one, driving) on the left of the **+** operator is being added to the **real** part of the object **c**, where **c** is passed as an argument.
In line 5, **imag** part of the calling object (that is the one, driving) on the left of the **+** operator is being added to the **imag** part of the object **c**, where **c** is passed as an argument.
In line 6, the **Complex** object **temp** containing the resultant of **+** operation is being returned by value.

In our code, we can write something as:

```
Complex c1, c2, c3 ;
. . .
. . .
c3 = c1 + c2 ;
```

In the above statement (c3 = c1 + c2;), c1 is the object that is calling or driving the **+** operator. **c2** object is being passed as an argument to the **+** operator. So c1 and c2 objects are added by the **+** operator and resultant **Complex** object containing the

© Copyright Virtual University of Pakistan

addition of these two numbers is returned back. That returned **Complex** object is assigned to the **c3 Complex** object using the default assignment operator (that is created by the C++ compiler automatically).

What happens if we want to add a **double** number to a complex number (a instance of **Complex**)? Like the following:

```
c3 = c1 + d ;
```

This **+** operation is driven by the **c1** object of **Complex** while double number **d** of type **double** is passed as argument. Therefore, our above written **+** operator is not useable for this operation of addition. We need to overload **+** operator for accepting a parameter of type **double**, i.e. we need to write another operator function. The definition of this newly overloaded **+** operator is:

```
Complex Complex :: operator + (double d)
{
        Complex temp ;
        temp.real = real + d ;      // d is added into the real part
        temp.imag = imag ;
        return temp ;
}
```

By now, you should have noticed that operator overloading and function overloading are quite similar.

When we write the following statement:

```
c3 = d + c1;
```

The operand on the left of **+** operator is a **double** number **d.** Therefore, this + operation should be driven by (called by) the **double** number. Until now, we have not written such an operator. Our previously written two **+** operators were driven by the **Complex** object. Operator functions, not driven by the class type objects, are kept as **friend**s to the class. **friend** is the keyword used to declare such functions. A **friend** function to a class also has access to the private members of that class.

```
friend Complex operator + (double d, Complex c)
{
        Complex temp;
        temp.real = d + c.real;    // d is added into the real part of c
        temp.imag = c.imag;
        return temp;
}
```

You might have noticed that all the three overloaded **+** operator functions are accepting and returning variables by value. To make these functions better, we can also use references. So our first member **+** operator's prototype can be rewritten as:

```
Complex& operator + (Complex& c);
```


© Copyright Virtual University of Pakistan

Now this operator function is accepting a complex number **Complex** by reference and returning a reference to the resultant complex number.

As discussed above, in case of assignment, the default assignment operator is used because we have not implemented (overloaded) our own assignment operator ('='). But in case, we want to perform the following operation where the two operands are added and the resultant is assigned to one of them as:

```
c1 = c1 + c2;
```

There is one operator (**+=**) that can be used to do both the operations of addition and assignment instead of doing these operations separately within **operator +** and **operator =.** So we can overload this one operator (**+=**) here to make the code more efficient and reduce our work. Therefore, instead of writing:

```
c1 = c1 + c2;
```

We will write:

```
c1 += c2;
```

We will write our **operator +=** as:

```
void Complex :: operator += ( Complex& c )
{
        real += c.real;
        imag += c.imag;

}
```

Non-member Operator Functions

Now we are much clear that when an operator function is implemented as a member function, the leftmost operator must be a class object or reference to a class object of the operator's class.

When an operator function is implemented as a non-member function, the left-most operand may be an object of the operator's class, an object of a different class, or a built-in type. Now we discuss it in a detailed manner.

We can always write our operators as non-member functions. As a non-member functions, the binary operators like **+** gets both the operands as arguments. One thing to take care of while writing non-member functions that they cannot access the private members of classes. Actually, this is just to this reason that we make those non-member functions as **friend**s to the classes whose **private** data members are required to be accessed. But the question arises, can we write a non-member operator function without making it a **friend** of a class. The answer to this question is yes; If there are **public** member functions to access the private data members of the class then they serve the purpose. In this case of **Complex** class, let's say we have two public member functions:

```
double real( );
double imaginary( );
```

to access the **private** data members **real** and **imag** respectively. Then we can write non-member **operator +** function as:

© Copyright Virtual University of Pakistan

```
Complex operator + (Complex& c1, Complex& c2)
{
        Complex temp;
        temp.real ( c1.real() + c2.real() );
        temp.imaginary ( c1.imaginary() + c2.imaginary() );
        return temp;
}
```

But this non-member operation functions without declaring a **friend** of the class is definitely slower than the member function or a **friend** one. The reason for this is obvious from the code that it is making three additional function calls of **real()** and **imaginary()** for each private data member. Also it is not easy to write as compared to member functions. Therefore, it is recommended to write the member functions for operators instead of non-members.

Let's take an example where the operators are performing a non-arithmetical operation. We are writing a class **String** for strings manipulation as:

```
class String
{
  private :

    char string [ 30 ] ;

  public :

    String ( )
    {
        strcpy ( string , "" ) ;
    }

    void getString ( )
    {
       cout << "Enter the String : " ;
       cin >> string ;
    }

    void displayString ( )
    {
      cout << "The String Is : " << string << endl ;
    }

    // Declaration (prototype) of overloaded sum operator

    String operator + ( String & s ) ;
};
```

We want to write + operator to concatenate two strings. Firstly, we will see the operator's behavior in ordinary context (behavior with primitive variables for example) and try to implement the same behavior for this class. We want to

© Copyright Virtual University of Pakistan

concatenate two strings (two **String** objects) and then assign the resultant string to a new **String** object. Here is how we will write + operator function.

```
String String :: operator + ( String &s )
{
   String temp;                    // Declared object temp of String type
   strcpy ( temp.string , "" );    // Initialized the temp with empty string
   strcat ( temp.string , string ); // Concatenated the driving object's string to //
temp object
strcat ( temp.string , s.string );  // Concatenated the argument's string to the
// temp object
   return  temp;                   // Returned the temp object
}
```

As you might have guessed already, the **String** object on the left will be the one to drive this + operation and the second String object on the left of + will be passed as an argument to this function. Note that we are not doing the error checking here, the size of the resultant string **temp** may increase the array size **30** (the array size defined in the class).

Example Program 1

Rudimentary implementation of a class named **Complex** class to cater complex numbers. A + operator function has been implemented to add two complex numbers.

```
/* This program implements the basic class for complex numbers and demonstrates +
operator function */

#include <iostream.h>

class Complex
{
   private :
      double  real ;    // Real Part
      double  imag ; // Imaginary Part

   public :
      /* Parameterless Constructor */
      Complex ( )
      {
         cout << "\n Parameterless Constructor called ..." ;
      }

      /* Parameterized Constructor */
      Complex ( double  r, double  i )
      {
         cout << "\n Parameterized Constructor called ...";
         real  =  r ;
         imag  =  i ;
      }
```


© Copyright Virtual University of Pakistan

```
    /* Setter of real data member */
    void  real ( double  r)
    {
       real  =   r ;
    }

    /* Getter of the real data member */
    double  real ( )
    {
          return real ;
    }

    /* Setter of the imag data member */
    void  imaginary ( double  i )
    {
       imag = i ;
    }

    /* Getter of the imag data member */
    double  imaginary ( )
    {
       return   imag ;
    }

    /* A Function to display parts of a Complex object */
    void  display ( )
    {
       cout  <<  "\n\n Displaying parts of complex number ...";
       cout  <<  "\n Real Part : " << real << endl ;
       cout  <<   " Imaginary Part : " << imag << endl ;
    }

    /* Declaration (prototype) of overloaded sum operator */
    Complex  operator  + ( Complex & c2 ) ;

};

Complex  Complex  ::  operator + ( Complex  & c1 )
{
   cout  <<  "\n Operator + called ...";
   Complex  temp ;
   temp.real = real + c1.real ;
   temp.imag  =  imag  +  c1.imag ;
   return  temp ;
}

void main ( )
{
```


© Copyright Virtual University of Pakistan

```
    Complex c1 ( 1 , 2 ) ;    // Consturct an object using the parameterized constructor
    Complex c2 ( 2 , 3 ) ;    // Consturct another object using the parameterized
                              // constructor

    Complex result ; // Construct an object using a parameterless constructor

    result = c1 + c2 ;  // Call the Operator + to add two complex numbers (c1 & c2)
                        // and then assign the result to 'result' object

     result.display ( ) ; // Display the result object contents

}
```

The output of the program is as follows:

```
Parameterized Constructor called ...
Parameterized Constructor called ...
Parameterless Constructor called ...
Operator + called ...
Parameterless Constructor called ...

Displaying parts of complex number ...
Real Part : 3
Imaginary Part : 5
```

The + operator function can be enhanced to return reference of Complex object. We can also implement += operator. += operator and the enhanced operator + are implemented as:

```
Complex &  Complex  ::  operator + ( Complex &  c1 )
{
   real =  real  +  c1.real ;
   imag = imag + c1.imag ;
   return  *this;
}

// Declaration (prototype) of overloaded sum assignment operator definition
Complex & Complex :: operator += ( Complex & c2 )
{
   real += c2.real ;
   imag += c2.imag ;
   return  *this;
}
```

Example Program 2

Rudimentary Implementation of String class to manipulate strings. It uses + operator to concatenate strings.

```
/* This program implements the basic class for strings and demonstrates + operator
function to concatenate two strings*/
```


© Copyright Virtual University of Pakistan

```
#include <iostream.h>
#include  <string.h>

class  String
{
   private :

      char    string [ 30 ] ;  // Array to store string

   public :

      /* Parameterless  Constructor */
      String ( )
      {
           strcpy ( string , "" ) ;
      }

      /* Getter function of string */
      void  getString ( )
      {
          cout  <<  "Enter the String: " ;
          cin  >>  string ;
      }

      /* Function to display string */
      void  displayString ( )
      {
         cout  <<   "The String is : "   <<   string   <<   endl ;
      }

      // Declaration (prototype) of overloaded sum operator

      String  operator  + ( String & s ) ;
};


String  String  ::  operator + ( String &s )
{
   String  temp ;
   strcpy ( temp.string , "" ) ;
   strcat ( temp.string , string );
   strcat ( temp.string , s.string );
   return  temp;
}

void main ( )
{
     String   string1 , string2 ;          // Declared two String objects
     string1.getString ( ) ;               // Get string for string1 object
     string2.getString ( ) ;               // Get string for string2 object
```


© Copyright Virtual University of Pakistan

```
    String hold = string1 + string2 ; // Concatenate string1 and string2 and store the
                                      // result in hold object
    hold.displayString ( ) ;          // Display the string
}
```

The output of the above program is as follows:

```
Enter the String: Operator
Enter the String: Overloading
The String is : OperatorOverloading
```

Tips

Operator Overloading is quite similar to Function Overloading.

There are two types of operators to overload: unary and binary.

C++ built-in operators work for built-in (primitve) types but for user defined data types, user has to write his/her own operators.

There are some restriction while performing Operator Overloading. For example, only existing C++ operators are overloaded without creating a new one in the language. Also, it should not impact the type, semantics (behavior), arity (number of operands required), precedence and associativity of the operator.

For binary member operators, operands on the left drives (calls) the operation.

Operator functions written as non-members but friends of the class, get both the operands as their arguments.

Operators can be written as non-members and even without making them friends. But this is tedious and less efficient way, therefore, it is not recommended.

© Copyright Virtual University of Pakistan

Lecture No. 32

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 8

8.6, 8.7, 8.12

Summary

6) Recap
7) Overloading Minus Operator
8) Operators with Date Class
9) Unary Operators

Recap

Before further discussing the concept of the 'Overloading', we will recapture the things dilated upon in the previous lecture. It is necessary to know that new operators i.e. new symbols cannot be introduced. Only existing symbols can be overloaded. Overloading of operators is exactly like writing functions. However, one should remain close to the original meaning of the operator. Similarly, it is good not to define something in opposite terms e.g. 'plus operator is doing subtraction or multiplication operator carrying out division'. We can do that but it will ultimately a bad thing for a programmer. It makes our program practically unreadable and mis-interpretable. Under operator overloading technique, the binary and unary operators will remain unchanged that is we cannot make unary operator work as binary operator or vice versa. In the previous lectures, we also came across some concepts in terms of driving force behind the operator, e.g. in case of binary operator, the driving force is left hand operand. We have also studied when to use member operators and non-member operators. Today we continue discussion on 'use of operators'.

Overloading Minus Operator

Let's define minus operator (-) with special reference to the complex class. The process of defining the minus operator is quite similar to that of the plus operator. Let's first understand the action of minus operator. It is a binary operator, having two arguments. In this case, both the arguments will be complex numbers. When we subtract two complex numbers, it always return a complex number. Here the subtraction of complex numbers is defined as, 'subtract the real part from real part and subtract the imaginary part from the imaginary one". So a member operator will look like as under:

© Copyright Virtual University of Pakistan

Complex operator – (Complex c)

As we are defining it as a member operator, only one argument will be passed to it. It is going to be on the right hand side of the minus operator. The left-hand-side will call this as it is already available to this function. In the body, we will declare a temporary Complex number. This means that the real part of this temporary complex number is the difference of the calling Complex number and the Complex number passed as argument i.e.:

tmp.real = real – c.real;

In the next line, we calculate the difference of imaginary part as:

tmp.imag = imag – c.image;

and return the tmp Complex number. By defining, the minus operator does not mean that minus equal operator has also been defined. If we want to overload the minus equal operator (-=), it is necessary to define it. Let's see how the defining process is carried out. Minus equal to operator like the plus equal to operator behaves in the way that the value of calling party (i.e. the complex number which is on the left hand side) will also be changed. So now we will see that the number itself changing the value when it takes part in the minus equal to operator. Again, we will make this a member function. So only one argument will be passed to it. The complex number will be on the right hand side of the minus equal to operator. In the body of the function, there is no need of any temporary complex number as we are going to change the number on the left hand side of the minus equal to operator. We can write it as:

```
real -= c.real;
imag -= c.image;
```

Here *c* is the complex number which is passed as an argument. Now the minus equal to (-=) operator, used in the above statements, is an ordinary minus equal to operator for the integers defined by the C++. So this is a classic example of overloading i.e. the operator being overloaded is using the original or basic operator of same type. That is the end of this function. The original number has been changed. We can return its reference. It depends on its usage.

Here is the code:

```
// The minus operator definition
Complex Complex::operator - ( Complex c )
{
   Complex tmp; // defining a temporary var
   tmp.real = real - c.real;
   tmp.imag = imag - c.imag;
   return tmp;
}

// The -= operator definition
Complex Complex::operator -= ( Complex c )
{
```


© Copyright Virtual University of Pakistan

```
    real -= c.real ;
    imag -= c.imag ;
}
```

Last time, we discussed the *string* class besides defining the plus operator as joining the two strings. Can we define minus for the *string* class? Is the minus operator relevant to the class *string*? For me it does not. Unless we come with some very artificial definition. Suppose we have a string as "This is a test" and a second string as "test". The subtraction of these two strings means the deletion of a word or words of second string from the first string. It may make some sense in this example. What will happen if the second string contains "My name is xyz". The subtraction of these strings does not make any sense. The thing we need to understand at this point is that every operator does not make sense for every class. Operators should be used only when these make some common sense so that reader can understand it easily. When you add two strings, it makes lot of sense. We can use either cat function or write this plus operator. As subtraction of strings does not make much sense, so it is not advisable to define it. Only define things that are self-explanatory, readable and understandable.

Operators with Date Class

We have so far been using the *Date* class. Let's think what operators make sense for *Date* class. What will be the meaning of plus operator or minus operator? Here we want to remind you a key thing i.e. "Paying attention to detail". Suppose you have some date and want to add some number to it like today's date plus 5. Does that make sense to you? We will get a new date by adding five to today's date i.e. date after five days. Similarly, if we want to subtract, say 10 from today's date, we should get the date of ten days before. Here is the usage of plus and minus which makes some sense. Can we subtract two dates together like subtraction of 1st Jan. 2002 from 15th Oct. 2002. What meaning it will convey? Perhaps nothing.

Let's consider the addition of a number to a date. Adding an integer to some date, according to the definition we will get some date in the future. The *Date* object will be returned from this function. We need a new date after the addition of integer number. We are defining this as a member-function so that the *Date* object that is calling it, will be passed to the function. The integer that is on the right hand side should be passed as an argument. Therefore in the parenthesis, we will have the integer. Now let's discuss it in detail. How can we add an integer to some date? Let's take today's date. Write it in your copy and see how can five be added to it. If you try to add a number to date, there are so many possibilities that can happen. Suppose, today is second day of the current month. After adding five to it, we will get 7th of this month. That was case I. Let's take the case II. Today is 27th of any month. Now what will be the new date after adding five. First thing, which is very obvious, that the month will get changed. But what will be the date of this new month. It depends whether there are 30 days or 31 days in this month. It may be the month of February. Is it the leap year or not? If it is non-leap year, there will be 28 days in February. Otherwise there will be 29 days. What is a leap year? There are rules to determine whether the year is leap year or not. If the year is divisible by four, it will be leap year. Similarly, being a century year, it may be divided by 400. Then again it is a leap year. Now we have seen that there are many cases when we are adding five to 27th of

© Copyright Virtual University of Pakistan

any month. Two things happen. The month is changed and the date changes according to the days in the month. What if it is the 27th of the December? Now you want to add five days. There are 31 days in December, after adding five it will be 1st of next month. We may have declared an array of twelve months. As December is the twelfth month, the last month of the year, so we have to go to first month of the next year. Here the year has also changed. We will also need to increment 1 to year too. It seems very simple that we have to add an integer number of days to some date. It becomes a complex function. Now suppose we have written this complex function and embedded all the rules in it. Then our life will become much easier. Suppose our semester starts from any date. After adding the period of semester, we will get the end date of the semester. We can do date arithmetic. This is a classic example of "paying attention to detail". To use the class for general purposes, we cannot miss even a single case. If you want to publish this class for others to use, you need to pay attention to detail and make sure that your class handles all of the stuff.

Here is the complete code of the program.
File "Date.h"

```
// The Date class is defined here

class Date{
   private:
      int day;
      int month;
      int  year;
      int daysOfMonth(Date d);                  // returns the no of days in a month
      static const int daysInMonth[]; // array containing the 12 month's days
      bool leapYear(int);                       // tells the year is leap year or not

   public:
      Date(int d = 1, int m = 1, int y = 1900); // constructor with default arguments
      void setDate(int, int, int);                          // set the date with given
arguments
      void display();              // Display the date on the screen
         // operators prototypes
      Date operator ++ ();       // pre increment operator used as ++date1
      Date operator + (int);  // Plus operator used as date1 + 5
};

// The implementation of the date class.
// initializing the no of days, take 0 for month zero.
const int Date::daysInMonth[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

// Displaying the function on the screen
void Date::display()
{
   cout <<"\nDate:" << day << "-" << month << "-" << year;
}
//constructor of the date
Date::Date(int d, int m, int y)
{
```


© Copyright Virtual University of Pakistan

```
    setDate(d, m, y);
}

// setting the date as given arguments
void Date::setDate(int d, int m, int y)
{
    year = y;

    // if month is wrong then set it to 1
    if (month < 1 && month > 12)
        month = 1;
    else
        month = m;
    // if day is wrong then set it to 1
    if (month == 2 && leapYear(y))
        if (d >=1 && d <=29)
            day = d;
        else
            day = 1;
    else
        if( d >= 1 && d <= daysInMonth[month])
                day = d;
        else
                day = 1;
}

// This function return the number of days in a month
int Date::daysOfMonth(Date d)
{
    if (d.month == 2 && leapYear(d.year))  // if leap year then Feb is 29
        return 29;
    else
        return daysInMonth[d.month];
}

// Testing that the year is leap or not.
bool Date::leapYear(int y)
{
    if ( (y%400 == 0) || (y%100 != 0 && y%4 == 0))
        return true;
    else
        return false;
}

// + operator overloaded for the date. Used as date1 + 5
Date Date::operator + (int numberOfDays)
{
    for (int i = 1; i <= numberOfDays; i++)
    {
        ++(*this);    // calling the pre increment operator
```


© Copyright Virtual University of Pakistan

```
    }
    return *this;
}

// Pre increment operator
Date Date::operator ++ ()
{
    if (day == daysOfMonth(*this) && month == 12) // end year
            {
             day = 1;
             month = 1;
             ++year;
            }
            else if(day == daysOfMonth(*this)) // end month
            {
             day = 1;
             ++month;
            }
            else      // not the last day of the month
            {
             day++;
            }
}
```

The main program is:

```
#include <iostream.h>
#include "date.h"

void main()
{
   Date  d1 (26, 12, 2002), d2(28,2 ,2000), d3;
   d1.display();
   ++d1;
   cout << "\nAfter adding 1 day, the date is ";
   d1.display();
   cout << endl;
   d2.display();
   d2 = d2 + 5;
   cout << "\nAfter adding 5 days to the above date";
   d2.display();
  }
```

Output of the program:

```
Date:26-12-2002
After adding 1 day, the date is
Date:27-12-2002

Date:28-2-2000
After adding 5 days to the above date
```


© Copyright Virtual University of Pakistan

Date:4-3-2000

Similarly we may have a counter-function that subtracts some number from the date. This is the same but of reverse nature. Suppose it is Jan 3rd and we have to subtract ten days from it. The month will be changed to December while the year is going to be decremented by 1. To determine the date of December, we need to know the number of days in December and count backwards. Now we don't need the number of days of current month. Rather the number of days in previous month is important. Suppose it is 3rd of March and subtract seven from it. What will be the date? Now you have to do complex arithmetic and take care of all the cases. It is very complicated but having only one time effort. Date arithmetic is very important and common in business applications. If someone applies for vacations, you just have to enter that this person is going on leave from this date for ten days and you will know his date of re-joining the duty. If someone works on daily wages and paid after a week. Someday, he comes and says that he is going on vacations. We need to calculate the number of days from the day of last payment to to-date. It is simple date arithmetic. Writing a *Date* class with these appropriate operators overloaded will be very useful exercise. It adds to your overall programming vocabulary.

There are two kinds of programming vocabulary. One is the keywords of C/C++ etc while the second is higher-level vocabulary. What sort of vocabulary we have in our tool- box. In the first part of this course, we have learned how to write loops, nested loops etc. We learn to handle matrices and vectors using those rudimentary rules. Now if you think about that we have written a matrix class and a member function *inverseOfMatrix()*. We can use this function again and again. Similarly in the *Date* class, we can put in some rudimentary calculations on date arithmetic. Add or subtract number of days from some date. These are very useful functions. In the daily wages example, you need to subtract a date from a date. Now we need to overload the minus operator again with date minus date. First we overload the minus operator with date minus some integer number. There may be two versions of minus operator. Here, you have to work in detail. Subtracting a date from another date is relatively non-trivial. As a programming idea, you can think that subtracting two dates involves huge calculations. Can we perform some logical tests here? If we want to implement *date1 – date2* while *date1* is smaller than *date2*. The first question is do we want to return a negative number. Let's say we want this, then *date1 – date2* can return a negative number. So it can return a negative number or zero (if the dates are identical) or positive number (the number of days). How we can implement this functionality? One way to do it is with the help of calendar. Under this method, we will start a loop till the other date is got. Then by reading the loop counter, we can tell the difference in days. It is a good idea. But for that, we need a calendar somewhere. If the dates are in different years, we will have to ensure the availability of calendar of next year. Think about it and try to write this function.

Now what about the plus-operator for two dates? Minus operator for strings did not make a lot of sense. Similarly, the plus operator for two dates does not make much sense. We can add some number to date. But how can we add a date to some other date. There is no logical and straight forward answer to this. So we don't define such a function. The meaning of our operator should be obvious. You can write whatever you want in the function. But it is bad idea. The idea of this exercise is to pay attention to detail. Think of all the various things that can happen. Tabulate them,

© Copyright Virtual University of Pakistan

determine the logic and then start programming. Don't start typing your program before your brain has come up to the same point. First analyze the problem, understand it, look at all the cases, draw a flow chart, write pseudo code. Once you are comfortable with this and know what you want to do then start writing your program. The time spending on analyses is arguably the best usage of your time as a programmer. The time you spend on debugging and removing errors from faulty code is huge. Spending time on good design pays off. You should debug for syntax errors like a semi-colon is missing somewhere. You should not face any logical error at debugging stage because logic errors are very hard to track. You may just not worry about the design and start writing code. The program may work for two or three cases. You may declare that you have written the program. When other starts using it on some other case which you did not cater, the program does not work or produces some strange results. There is no syntax error in the program. The compiler compiles it successfully and makes an executable file. Now we have to check the logic. Determining the logic from the code is a million times more difficult than determining code from logic. In this case, analysis will be always followed by design and then code. Please keep this in mind.

Unary Operators

Let's talk about unary operators. Unary operators take one argument like *i*++ or *i*-- (Post Increment or post decrement operators for integers) or ++i, --i (Pre increment or pre decrement operator). You can't make unary operator as binary operator or binary operator as unary. Let's overload unary operator in the *Date* class. We want to overload ++. This operator should add a day in the current date. When we say *++date1 or date1++*, it should get tomorrow's date. This is same as *date1 +=1 or date1 = date1 + 1*. We simply have to change to tomorrow's date. If this is the member function, it will get the date object automatically. The internal structure of the object is available to the function so that it takes no argument. It will return a *Date* object. Its prototype will be as:

```
Date operator ++ ( ); // pre increment operator
```

What will be in the function definition? You have to pay attention to details. The argument that we used in the plus operator, is also applicable here. What will be the next date when we add 1 to the current date. Let's work it out. If it is not the last date of the month, then simply add one to the day. If the date is the last day of the month, then change the month and date to 1st. If the date is the last date of the year, then increment the year too. Suppose we have some function available which returns the days of month given the month number. So if we say *daysOfMonth(6)* it should return 30. The function is intelligent enough that when we say *daysOfMonth(2)* it should return 28 if the year is not leap year. Otherwise, it will be 29. Therefore we have to send it year too along with the month number. We can also pass it the complete *Date* structure as *daysOfMonth(date1);* We will use this function in writing the ++ operator. In a way, the logic is same as we used in the plus operator. Suppose the object *d* is calling this ++ operator as *d*++ where *d* is an object of type *Date*. Therefore the day, month and year data members will be available to this function. In the body of the function, first of all we will check whether this is the last date of the month as:

© Copyright Virtual University of Pakistan

```
if (day == daysOfMonth ( *this ) )
{
        // this is the last day of the month
        // process accordingly
}
```

In the above condition, we have checked that day is equal to the number of days in the month or not. If the condition returns true it means that this is the last day of the month. Here we have used *this* to pass the object (current object) to the function *daysOfMonth*. '*this'* pointer is implicitly available to every member function and *this pointer* points to the current object. As per requirement of the program, we have written *d++* where *d* is the object of type *Date*. We are not using the object *d* in the program. This object is itself available.

Now the data of object *d* is available in the function as day, month or year. The object *d* is itself present either from its member data (day, month, year) or through the '*this* pointer' which points to the current object. We can also expand the definition of the function *daysOfMonth( )* as *daysOfMonth(int day, int month, int year)*. If the given day is the last day of the month, we will increment the month. Before doing this, we need to check whether this is the last month or not. Therefore we have to introduce another nested 'if' condition. The code segment will now be as:

```
if (day == daysOfMonth ( this ) )
{
        // this is the last day of the month
    if (month < 12)
        {
            day = 1;
            month++;
         }
         else // this is the last month i.e. December
         {
                day = 1;
                month = 1;
                year++;
         }
}
else        // not the last day of the month
{
            day++;
}
```

The ++ operator simply adds one to the date of the calling object. We define it as member function. Therefore, no argument is needed. We can make it non-member but have to pass it a *Date* object.

To distinguish the pre increment operator with post increment operator, an int argument is passed to it. The prototype of post increment operator for Date is:

```
Date operator ++ (int  ); // post  increment operator
```


© Copyright Virtual University of Pakistan

Here we don't need to use this int argument. The implementation is same as pre increment operator as in both cases we want to add 1 to the date.

Can we implement the plus operator using this function? We can write the plus operator in some new fashion. We pass it a positive integer number, which has to be added to the date. We can write a loop in the plus operator. The loop condition will be as *i* < *number* where *number* is the argument passed to it. So in the program if we have written as *date1+5;* the loop will run for five times and in the body of the loop we have ++*date1*. Suddenly our complicated logic has been boiled down to simple as incremented by 1. This is the classic example of code reuse.

We don't know who is going to use this code. Nobody is perfect. But we should think before writing the program about the structure, interface, the setters and getters and the operators to be overloaded. The thumb rule is if the meaning of + and ++ operator is same as in ordinary arithmetic, then + operator can be used in the ++ operator. Keep in mind that we can call a function from another function. This is a good example of code reuse. We can call the + operator as many times as needed. The *daysOfMonth* is a member function and it is used in ++ operator function. '+ operator' is a member function, used in ++ operator. We are building a hierarchy. Suppose there is some small logical error in the code and the *daysOfMonth* is not returning the correct value. This will effect the + operator as well as ++ operator. When we remove that error, then + and ++ operator both will be corrected. Moral of the story is that whenever we write some code, it is better to see whether we are rewriting some code in the same class. If we are calculating the number of months at two places or determining the leap year at two places, then try to combine this in such a way that things should be calculated at one place. That piece of code may become some utility function. This will not be called from outside the class so we will put this function in the private area. But the member functions can call it. We will make the *daysOfMonth()* as a private member function of the class. It will return the days of the month having checked whether this is leap year or not. Using this utility function, we have written + and ++ operator function. Don't repeat code inside a class. Make it a general rule. Make a function for the repeated code and call it where needed. For efficiency and speed, we can repeat the code. For this, we start using macros. It means that if you put all your logic in a single place and then reuse it. You will get lot of safety and security with this. A correction at one place will make the behavior of the whole class correct.

Let's see another interesting function of the *Date* class. Sometimes, we need to compare two dates i.e. whether a date is greater or less than the other date. In other words, the comparison operator is applied. Comparison operators <, >, <=, >=, == can also be overloaded. How do we determine whether *date1* is greater than *date2*? First of all, what will be its return type. Return type has to be either true or false. It says *date1* is greater than *date2* or *date1* is not greater than *date2*. Let's introduce another keyword *bool*. It is a new data type. It is very simple, it only takes two values true or false. So, the return type of greater than operator (>) is *bool*. The prototype of this member function is as:

```
bool operator > (Date d);
```


© Copyright Virtual University of Pakistan

The argument *d* is the *Date* object that is on the right side of the greater than sign. The left hand side *Date* object is available to this as this is the member operator of the class. Before writing the code, think abut the logic. We have to determine that the calling date is greater than the date *d* or not. If the year of current date is greater than date d, will the current date greater than date *d*? Certainly, it will be so. If the year is greater, obviously date is greater. If the years of both the dates are equal, then we have to check whether the month of the current date is greater than date *d* or not. If the month of the current date is greater than the date *d*, current date is greater. If the months are also equal, we will compare the days. It's a very simple hierarchical logic. To be able to write this logic cleanly, you should write case by case on paper. Analyze it thoroughly. The logic can be written in reverse too. If the year of the date *d* is greater than the current date, return false and so on. So we can go either true, true, true or false, false, false logic. You will find the false logic quicker. We can use if, 'else if' structures. Return type of this function is 'boolean'. Suppose that in our calling function we have two *Date* objects as *d1* and *d2*. We will write as *if(d1 > d2)*. Why should we write this? As our operator is returning true or false and 'if' also needs true or false, we can write very clean and neat code. This greater than operator is a member operator of *Date* class. In this case, we have the return type as boolean and not returning the *Date* object. Is it the violation of any rule? The answer is no. The return type can be anything. It needs not to be the same as the class. It can be anything. The same applies to difference between two dates. The difference between two dates will be an integer. It is still a member function. It simply tells us the number of days between two days. It could be negative or positive but it is an integer. There is no such rule that the member operators should return the object of the same class.

The code of the greater than operator is as follows:

```
// Definition of the greater than operator

bool Date :: operator > ( Date d )
{
    if ( year > d.year )  // if year is greater date is greater
    {
      return true;
    }
    else if ( year == d.year)     //if years are equal check month
    {
        if ( month > d.month )  // if month is greater date is greater
        {
              return true;
        }
        else if ( month == d.month)      // if months are equal check dates
        {
            if(day > d.day)
              return true;
            else                                          // otherwise return false
              return false;
        }
        else
           return false;
    }
```


© Copyright Virtual University of Pakistan

```
    else
    {
      return false;
    }
}
```

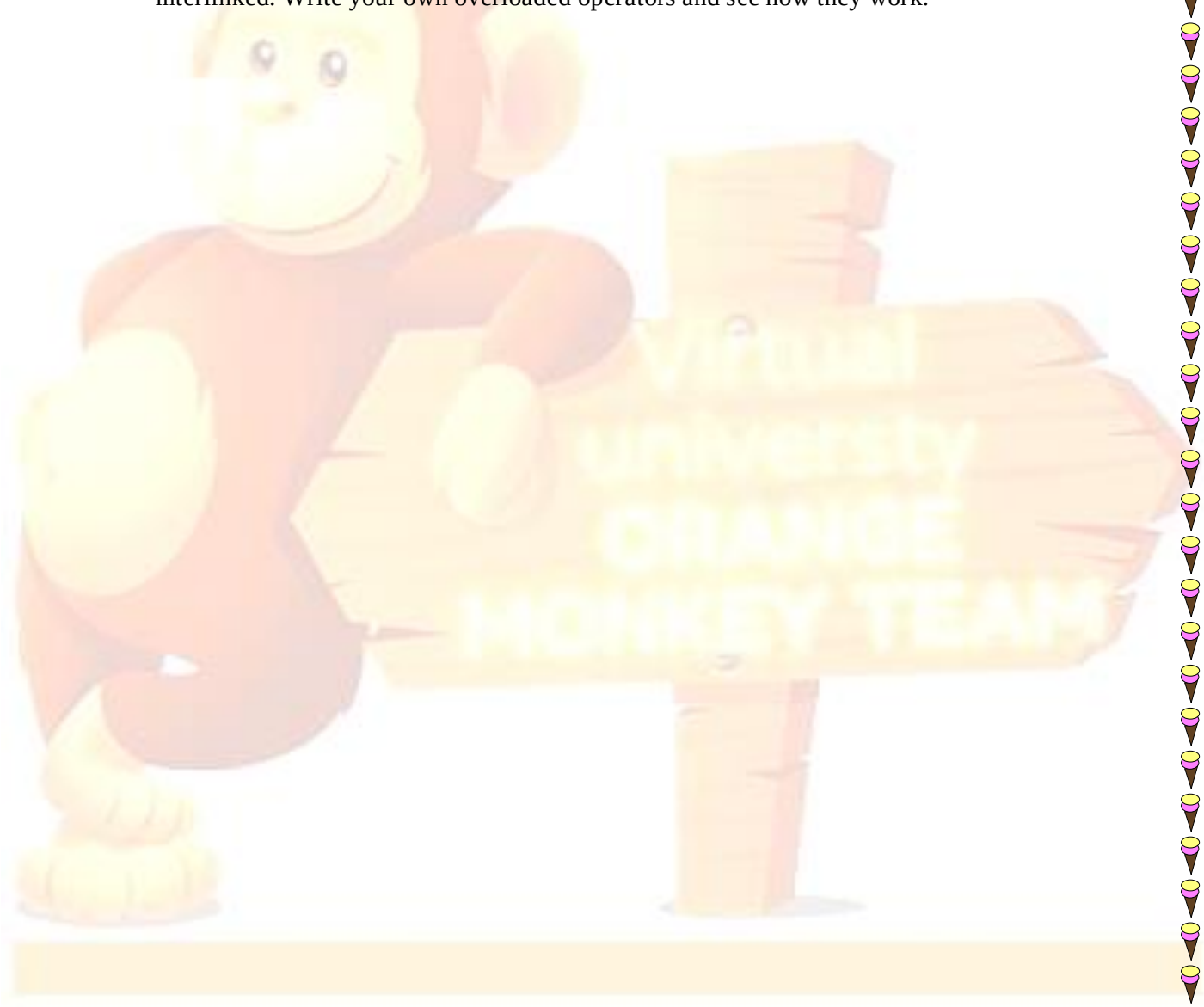
Now you can write all the comparison operator of the *Date* class. The comparison operators are greater than, greater than or equal to, equal to, less than, less than or equal to. If you are writing one, you might want to write all of them. Now we have expanded the *Date* class enough. As an exercise, write your own *Date* class. Keep in mind the principles. What should be in the *Date* class? How should we set its values? How many constructors we need? Do we need a destructor or default destructor is enough. After this, define its public interface that is the member functions that are visible from outside. What operators should be overloaded?

In the program, if we say *date1 + 5,* we know that we will get a date which is five days later. What will happen if we write *5 + date1*? The situation like this may happen. You have published your *Date* class and someone wants to use it in this fashion. Here we have an integer, the plus operator and a *Date* object. It should return an object of type *Date*. To make this work properly, we need to have another operator. You will have to look at the set of operators needed for this class. List them out and write down their behavior. Be very clear what you expect them to do and start writing the class. How can we implement *integer + date*? On the left hand side, we have an integer. If the integer is at the left side, it can't be a member function. Member function is always called by object. Here object is not calling the function. Rather integer is calling. So it has to be a friend function that is sitting outside. It will get two arguments, integer and *Date.* As this is the friend function, the internal structure of the *Date* will be available to it. You will create a new *Date* object based on the given *Date* object and the integer and return it. We have seen that member functions are returning integers or Boolean. Here, a non-member function is returning an object of *Date* class. When we have listed out comprehensively that what will be the interface of our class. Which functions and operators will be visible from outside? When we have written the behavior of our class on paper, it is good to start writing the code. You may have to write a lot of code for this class. Once we have compiled the code and have object file, then anyone can use *Date* object. There will be no problem. We will include the "date.h" file in the program and use and manipulate the *Date* objects. We can use its operators, member functions etc very easily. The effort we put in writing this code does not go waste. It will provide a lot of ease in the main program. The biggest advantage is the encapsulation that has happened. All of the logic that was needed to manipulate the object of class *Date* is now encapsulated in that class. In case of any problem in the behavior of the class, we will need to correct the class, compile it. In the conventional function programming or structured programming, this logic has been split at different locations of the program. It was everywhere. Different things have been embedded at different points. In the function oriented programming, we have written a lot of functions at different locations. Here we have a new data type as *Date*. All the date related functions are at one place. We are encapsulating all the functionalities in the *Date* class that is another reason for doing all of the homework, all the thinking before we write the code. No one can determine all the usage of the *Date* class. If you start determining all the usage of *Date* class and writing the definition of the *Date* class for the last six months, this will be impractical. You

© Copyright Virtual University of Pakistan

would want to keep it within limits but do the homework then you write it. Now you can reuse it as many times as you want.

We need a friend operator when the driving thing is not the object of the class like *integer + date*. The operator is derived by integer. Here, we use a friend function. There are instances where friend operators are used to manipulate two different classes. A classic example in the mathematics is of the multiplication of vector and matrix. If we have to multiply a vector with a matrix, the multiplication operator will be friend of both the classes. It will get both vector and matrix as arguments and manipulate them. Keep in mind that friend operators in a way can also be used to glue two classes. The disadvantage of clubbing them together is that they become interlinked. Write your own overloaded operators and see how they work.



© Copyright Virtual University of Pakistan

Lecture No. 33

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 7, 8

7.5, 8.1, 8.2, 8.7, 8.9

Summary

10) Operator Overloading
11) Assignment Operator
12) Example
13) this Pointer
14) Self Assignment
15) Returning this Pointer from a Function
16) Conversions
17) Sample Program (conversion by constructor)

Operator Overloading

As earlier discussed, overloading of operators is carried out in classes on some occasions to enable ourselves to write a code that looks simple and clean. Suppose, there is a class and its two objects, say *a* and *b*, have been defined. Addition of these objects in the class by writing *a + b* will mean that we are adding two different objects (objects are instance of a class which is a user defined data type). We want our code to be simple and elegant. In object base programming, more effort is made in class definitions, as classes are data types that know how to manipulate themselves. These know how to add objects of their own type together, how to display themselves and do many other manipulations. While discussing *date* class in the previous lecture, we referred to many examples. In an example, we tried to increment the 'date'. The best way is to encapsulate it in the class itself and not in the main program when we come around to use the *date* class.

Assignment Operator

At first, we ascertain whether there is need of an assignment operator or not? It is needed when we are going to assign one object to the other, that means when we want to have expression like *a = b*. C++ provides a default assignment operator. This operator does a member-wise assignment. Let's say, we have in a structure of a class three integers and two floats as data members. Now we take two objects of this class *a* and *b* and write *a = b*. Here the first integer of *a* will have the value of first integer of *b*. The second will have the value of second integer and so on. This means that it is a

© Copyright Virtual University of Pakistan

member-wise copy. The default assignment operator does this. But what is to do if we want to do something more, in some special cases?

Now let's define a *String* class. We will define it our self, without taking the built in *String* class of C. We know that a string is nothing but an array of characters. So we define our *String* class, with a data member buffer, it is a pointer to character and is written as **buf* i.e. a pointer to character (array). There are constructors and destructors of the class. There is a member function *length* that returns the length of the string of the calling object. Now we want an assignment operator for this class. Suppose we have a constructor that allows placing a string into the buffer. It can be written in the main program as under:

String s1 ("This is a test") ;

Thus, an object of *String* has been created and initialized. The string "This is a test" has been placed in its buffer. Obviously, the buffer will be large enough to hold this string as defined in our constructor. We allocate the memory for the buffer by using *new* operator. What happens if we have another *String* object, let's say s2, and want to write *s2 = s1* ; Here we know that the buffer is nothing but a pointer to a memory location. If it is an array of characters, the name of the array is nothing but a pointer to the start of the memory location. If default assignment operator is used here, the value of one pointer i.e. *buf* of one object will be assigned to *buf* of the other object. It means there will be the same address in the both objects. Suppose we delete the object *s1*, the destructor of this object will free the allocated memory while giving it back to the free store. Now the *buf* of *s2* holds the address of memory, which actually has gone to free store, (by the destructor of s1). It is no longer allocated, and thus creates a problem. Such problems are faced often while using default assignment operator. To avoid such problems, we have to write our own assignment operator.

Before going on into the string assignment operator, let's have a look on the addition operator, which we have defined for strings. There is a point in it to discuss. When we defined addition operator for strings, we talked about that what we have to do if we want to add (concatenate) a string into the other string. There we had a simple structure i.e. there is a string defined *char buf* with a space of 30 characters. If we have two string objects and the strings are full in the both objects. Then how these will be added? Now suppose for the moment that we are not doing memory allocation. We have a fixed string buffer in the memory. It is important that the addition operator should perform an error check. It should take length of first string , then the length of second string, before adding them up, and check whether it is greater than the length defined in the String (i.e. 30 in this case). If it is greater than that, we should provide it some logical behavior. If it is not greater, then it should add the strings. Thus, it is important that we should do proper error checking.
Now in the assignment operator, the problem here is that the *buf*, that we have defined, should not point to the same memory location in two different objects of type *String*. Each object should have its own space and value in the memory for its string. So when we write a statement like *s2 = s1*, we want to make sure that at assignment time, the addresses should not be assigned. But there should be proper space and the strings should be copied there. Let's see how we can do this.

© Copyright Virtual University of Pakistan

Now take a look on the code itself. It is quiet straight forward what we want to do is that we are defining an assignment operator (i.e. =) for the String class. Remember that the object on left side will call the = operator. If we write a statement like *s2 = s1*; *s2* will be the calling object and *s1* will be passed to the = operator as an argument. So the data structure of *s2* i.e. *buf* is available without specifying any prefix. We have to access the *buf* of *s1* by writing *s1.buf*.

Suppose *s2* has already a value. It means that if *s2* has a value, its buffer has allocated some space in the memory. Moreover, there is a character string in it. So to make an assignment first empty that memory of *s2* as we want to write something in that memory. We do not know whether the value, we are going to write, is less or greater than the already existing one. So the first statement is:

delete buf ;

Here we write *buf* without any prefix as it is *buf* of the calling object. Now this buffer is free and needs new space. This space should be large enough so that it can hold the string of *s1*. First we find the length of the string of *s1* and then we use the *new* operator and give it a value that is one more than the length of the buffer of *s1*. So we write it as:

buf = new char[length + 1] ;

where *length* is the length of *s1*. Here *buf* is without a prefix so it is the *buf* of object on the left hand side i.e. *s2*. Now, when the *buf* of *s2* has a valid memory address, we copy the *buf* of *s1* into the *buf* of *s2 with the use of string copy function (strcpy).* We write it as:

strcpy (buf, s1.buf) ;.

Now the *buf* of string of *s1* has been copied in the *buf* of *s2 that* are located at different spaces in the memory. The advantage of it is that now if we delete one object *s1* or *s2*, it will not affect the other one. The complete code of this example is given below.

Example

```
/*This program defines the assignment operator. We copy the string of one object
into the string of other object using different spaces for both strings in the memory.
*/

#include <iostream.h>
#include <string.h>
#include <stdlib.h>
// class definition

class String
{
        private :
```


© Copyright Virtual University of Pakistan

```
        char *buf ;
    public:
        // constructors
        String();
        String( const char *s )
        {
            buf = new char [ 30 ];
            strcpy (buf,s);
        }
        // display the string
        void display ( )
    {
            cout << buf << endl ;
    }
        // getting the length of the string
        int length ()const
        {
            return strlen(buf);
        }
        // overloading assignment operator
        void operator = ( const String &other );
};
// ------------Assignment operator
void String::operator = ( const String &other )
{
    int length ;
    length = other.length();
    delete buf;
    buf = new char [length + 1];
    strcpy( buf, other.buf );
}

//the main program that uses the new String class with its assignment operator:
main()
{
    String myString( "here's my string" );
    cout << “My string is = ” ;
    myString.display();
    cout << '\n';

    String yourString( "here's your string" );
    cout << “Your string is = ” ;
    yourString.display();
    cout << '\n';

    yourString = myString;
    cout << “After assignment, your string is = ” ;
    yourString.display();
    cout << '\n';
    system ("pause");
```


© Copyright Virtual University of Pakistan

```
}
```

Following is the output of the program.

```
My string is = here's my string

Your string is = here's your string

After assignment, your string is =  here's my string
```

The above example is regarding the strings. Yet in general, this example pertains to all classes in which we do memory manipulations. Whenever we use objects that allocate memory, it is important that an assignment operator (=) should be defined for it. Otherwise, the default operator will copy the values of addresses and pointers. The actual values and memory allocation will not be done by it.

Let's go on and look what happens when we actually do this assignment? In the assignment of integers, say we have three integers i, j and k with some values. It is quiet legal to write as *i = j* ; By this ,we assign the value of *j* to *i*. After this we write *k = i* ; this assigns the value of *i* to *k*. In C, we can write the above two assignment statements in one line as follows

$$k = i = j ;$$

This line means first the value of *j* is assigned to *i* and that value of *i* is assigned to *k*. The mechanism that makes this work is that in C or C++ every expression itself has a value. This value allows these chained assignment statements to work. For example, when we write *k = i = j* ; then at first *i = j* is executed. The value at the left hand side of this assignment statement is the value of the expression that is returned to the part '*k ='* .This value is later assigned to *k*. Now take another example. Suppose we have the following statement:

$$k = i = ++j ;$$

In this statement, we use the pre-increment operator, as the increment operator (++) is written before *j*. This pre-increment operator will increment the value of *j* by 1 and this new value will be assigned to *i*. It is pertinent to note that this way *++j* returns a value that is used in the statement. After this, the value of *i* is assigned to *k*. For integers, it is ok. Now, how can we make this mechanism work with our *String* objects. We have three *String* objects *s1*, *s2* and *s3*. Let's say there is a value (a string) in the buffer of *s1*. How can we write *s3 = s2 = s1*;. We have written *s2 = s1* in the previous example in assignment operator. We notice that there is no return statement in the code of the assignment operator. It means that operator actually returns nothing as it has a void return type. If this function is not returning any thing then *s3 = s2 = s1* ; cannot work. We make this work with the help of *this* pointer.

this Pointer

Whenever an object calls a member function, the function implicitly gets a pointer from the calling object. That pointer is known as *this* pointer. '*this'* is a key word. We cannot use it as a variable name. '*this'* pointer is present in the function, referring to

© Copyright Virtual University of Pakistan

the calling object. For example, if we have to refer a member, let's say *buf*, of our *String* class, we can write it simply as:

b*uf ;*

That means the *buf* of calling object is being considered. We can also write it as

this->buf ;

i.e. the data member of the object pointed by *this* pointer is being called. These (*buf* and *this->buf*) are exactly the same. We can also write it in a third way as:

*(*this).buf ;.*

So these three statements are exactly equivalent. Normally we do not use the statements written with *this* key word. We write simply *buf* to refer to the calling object.

In the statement *(*this).buf* ; The parentheses are necessary as we know that in *object.buf* the binding of dot operator is stronger than the *. Without parentheses the *object.buf* is resolved first and is dereferenced. So to dereference *this* pointer to get the object, we enforced it by putting it in parentheses.

Self Assignment

Suppose, we have an integer '*i*. In the program, somewhere, we write *i = i* ; It's a do nothing line which does nothing. It is not an error too. Now think about the String object, we have a string *s* that has initialized to a string, say, 'This is a test'. And then we write *s = s* ; The behavior of equal operator that we have defined for the String object is that it, at first deletes the buffer of the calling object. While writing *s = s* ; the assignment operator frees the buffer of *s*. Later, it tries to take the buffer of the object on right hand side, which already has been deleted and trying to allocate space and assign it to *s*. This is known as self-assignment. Normally, self -assignment is not directly used in the programs. But sometimes, it is needed. Suppose we have the address of an object in a pointer. We write:

*String s, *sptr ;*

Now *sptr* is a pointer to a *String* while *s* is a *String* object. In the code of a program we can write

sptr = &s ;

This statement assigns the address of *s* to the pointer *sptr*. Now some where in the program we write *s = *sptr* ; that means we assign to *s* the object being pointed by *sptr*. As *sptr* has the address of *s* , earlier assigned to it. This has the same effect as *s = s* ;. The buffer of *s* will be deleted and the assignment will not be done. Thus, the program will become unpredictable. So self-assignment is very dangerous especially at a time when we have memory manipulation in a class. The *String* class is a classic example of it in which we do memory allocation. To avoid this, in the equal operator (operator=), we should first check whether the calling object (L.H.S.) and the object

© Copyright Virtual University of Pakistan

being gotten (R.H.S.) are the same or not. So we can write the equal operator (operator=) as follows

```
void String::operator=( const String &other )
{
    if( this == &other )
        return;
delete buf;
length = other.length;
buf = new char[length + 1];
strcpy( buf, other.buf );
}
```

Here above, the statement *if (this == &other)* checks that if the calling object (which is referred by *this*) is the same as the object being called then do nothing and return as in this case it is a self assignment. By doing this little change in our assignment operator, it has become safe to use. So it is the first usage of *this* pointer that is a check against self assignment.

Returning *this* Pointer From a Function

Now lets look at the second use of *this* pointer. We want to do the assignment as

```
s3 = s2 = s1 ;
```

In this statement the value of *s2 = s1* is assigned to *s3*. So to do this it is necessary that the assignment operator should return a value. Thus, our assignment operator will expand so that it could return a value. The assignment operator, till by now copies the string on right hand side to the left hand side. This means *s2 = s1* ; can be done by this. Now we want that this *s2* should be assigned to *s3*, which can be done only if *s2 = s1* returns *s2* (an object). So we need to return a *String* object. Here becomes the use of *this* pointer, we will write as

```
return *this ;
```

Here, in the assignment operator code *this* is referring to the calling object (i.e. s2 in this case). So when we write *return *this* ;* it means return the calling object (object on L.H.S.) as a value. Thus *s3* gets the value of *s2* by executing *s3 = s2* where *s2* is the value returned by the assignment operator by *s2 = s1*; Thus the complete assignment operator (i.e. operator= function) that returns a reference to an object will be written as under

```
String &String::operator=( const String &other )
{
    if( &other == this )    //if calling and passed objects are
        return *this;       // same then do nothing and return
    delete buf;
    length = other.length;
buf = new char[length + 1];
strcpy( buf, other.buf );
```


© Copyright Virtual University of Pakistan

```
    return *this;
}
```

Now, here the first line shows that this *operator*= function returns a reference to an object of type *String* and this function takes a reference as an argument. The above version of *operator*= function takes a reference as an argument. First of all it checks it with the calling object to avoid self assignment. Then it deletes the buffer of calling object and creates a new buffer large enough to hold the argument object. And then at the last it returns the reference of the calling object by using *this* pointer.
With this version of the assignment operator (operator= function) we can chain together assignments of *String* objects like *s3 = s2 = s1* ;
Actually, we have been using *this* pointer in chained statements of *cout*. For example, we write

cout << a << b << c ;

Where *a*, *b*, and *c* are any data type. It works in the way that the stream of *cout* i.e. << is left associative. It means first *cout* << *a* is executed and to further execute it, the second << should have *cout* on its left hand side. So here, in a way, in this operator overloading, a reference to the calling object that is *cout* is being returned to the stream insertion <<. Thus a reference of *cout* is returned, and (as a reference to *cout* is returned) the << sees a *cout* on left hand side and thus the next << *b* is executed and it returns a reference to *cout* and with this reference the next << *c* works. This all work is carried out by *this* pointer.

We have seen that value can be returned from a function with *this* pointer. We used it with assignment operator. Lets consider our previous example of *Date* class. In that class we defined increment operator, plus operator, plus equal operator, minus operator and minus equal operator. Suppose we have *Date* objects *d1*, *d2* and *d3*. When we write like *d2 = d1++* ; or *d2 = d1 + 1* ; here we realize that the + operator and the ++ operator should return a value. Similarly, other operators should also return a value. Now let's consider the code of *Date* class. Now we have rewritten these operators. The difference in code of these is not more than that now it returns a reference to an object of type *Date*. There are two changes in the previous code. First is in the declaration line where we now use & sign for the reference and we write it like

Date& Date::operator+=(int days)

We write this for the all operators (i.e. +, ++, - and -=). Then in the function definition we return the reference of the left hand side *Date* object (i.e. calling object) by writing

*return *this* ;*

Now we can rewrite the *operator+=* of the *Date* class as follows.
The declaration line in the class definition will be as

Date& operator+=(int days);

And the function definition will be rewritten as the following.

© Copyright Virtual University of Pakistan

```
Date& Date::operator+=(int days)  // return type  reference to
object
{
        for (int i=0; i < days; i++)
        *this++;
        return *this;    // return reference to object
}
```

This concludes that whenever we are writing arithmetic operator and want that it can be used in chained statements (compound statements) then we have to return a value. The easiest and most convenient way of returning that value is by returning a reference to the calling object. So, by now we can easily write the statements of *date* object, just like we write for integers or floats. We can write

```
        date2 = date1 + 1 ;
Or      date2 = date1++ ;
```

and so on.

Conversions

Being in the C language, suppose we have an integer *i* and a float *x*. Now in the program we write *x = i* ; As we know that the operations of *int* and *float* are different in the memory. Therefore, we need to do some kind of conversion. The language automatically converts *i* (int) to a float (or to whatever type is on the L.H.S.) and then does the assignment.

Both C and C++ have a set of rules for converting one type to another. These rules are used in the following situations

- When assigning a value. For example, if you assign an integer to a variable of type long, the compiler converts the integer to a long.
- When performing an arithmetic operation. For example, if you add an integer and a floating-point value, the compiler converts the integer to a float before it performs the addition.
- When passing an argument to a function; for example, if you pass an integer to a function that expects a long.
- When returning a value from a function; for example, if you return a float from a function that has double as its return type.

In all of these situations, the compiler performs the conversion implicitly. We can make the conversion explicit by using a cast expression.

Now the question arises that can we do conversion with objects of our own classes. The answer is yes. If we go to the basic definition of a class it is nothing but a user defined data type. As it is a user defined data type, we can also define conversion on it. When we define a class in C++, we can specify the conversions that the compiler can apply when we use instances of that class. We can define conversions between classes, or between a class and a built-in type

© Copyright Virtual University of Pakistan

There is an example of it. Suppose we have stored date in a serial number form. So now it is not in the form of day, month and year but it is now a long integer. For example if we start from January 1, 1900 then 111900 will be the day one, second January will be the day 2, third January will be the day 3 and so on. Going on this way we reached in year 2000. There will be a serial number in year 2000. This will be a single long integer that represents the number of days since a particular date. Now if we have a *Date* object called *d* and an integer *i*. We can write *d = i* ; which means we want that *i* should go in the serial part of *d*. This means convert the integer *i* into *date* object and then the value of *i* should go into the serial number and then that object should be assigned to *d*. We can make this conversion of *Date* object. We do this in the constructor of the class. We pass the integer to the constructor, here convert it into a date, and the constructor then returns a *Date* object. On the other hand, if there is no constructor then we can write conversion function. We have used the conversion operator with *cast*. The way of casting is that we write the name of the cast (type to which we want to convert) before the name of variable. Thus if we have to write *x = i* ; where *x* is a float and *i* is an integer then we write it with conversion function as *x = (float) i* ; The (float) will be the conversion operator. Normally this conversion is done by default but sometimes we have to force it. Now we want to write a conversion operator, which converts an integer to a *Date* object. (here Date is not our old class, it's a new one). We can write a conversion function. This function will be a member of the *Date* class. This function will return nothing, as it is a conversion function. The syntax of this function will be *Date ()* that means now this is a conversion function.

In the body of this function we can write code of our own. Thus we can define the operator, and it will work like that we write within the parentheses the name of the conversion operator. The conversion functions are quiet interesting. They allow us to manipulate objects of different classes. For example, we have two classes one is a truck and other is a car. We want to convert the car to a truck. Here we can write a conversion function, which says take a car convert it into a truck and then assign it to an object of class truck.

Sample Program (conversion by constructor)

Lets take a look at an example in which the conversion functions are being used. There is a class *Fraction*. Lets talk why we called it *Fraction* class. Suppose we have an assignment

double x = 1/3 ;

When we store 1/3 in the computer memory, it will be stored as a double precision number. There is a double precision division of 1 by 3 and the answer 0.33333… is stored. Here the number of 3s depends upon the space in the memory for a double precision number. That value is assigned to a *double* variable. What happens if we multiply that *double* variable by 3, that means we write *3 * x*; where x was equal to 1/3 (0.33333…). The answer will be 0.99999…, whatever the number of digits was. The problem here is that the answer of 3 * 1 / 3 is 1 and not 0.99999. This problem occurs, when we want to represent the numbers exactly. The fraction class is the class in which we provide numerator and denominator to the object and it stores them separately. It will always keep them as an integer numerator and an integer denominator and we can do all kinds of arithmetic with it. Now if 1 and 3 are stored

© Copyright Virtual University of Pakistan

separately as numerator and denominator, how we can add them to some other fraction. We have enough tools at our disposal. One of which is that we can overload the addition operator. We can add 1/3 and 2/5, the result of which is another fraction i.e. numerator and denominator. In this way we have no worries of round off errors, the truncation and conversions of int and floats.

Considering the *Fraction* class we can think of a constructor which takes an integer and converts it into a fraction. We do not want that as a fraction there should be some value divided by zero. So we define the default constructor that takes two integers, one for numerator and one for denominator. We provide a default value for the denominator that is 1. It means that now we can construct a fraction by passing it a single integer, in which case it will be represented as a fraction with the passed integer as a numerator and the default vale i.e. 1 as the denominator. If we have a fraction object *f* and we write *f = 3*; Then automatically this constructor (i.e. we defined) will be called which takes a single integer as an argument. An object of type fraction will be created and the assignment will be carried out. In a way, this is nothing more than a conversion operation. That is a conversion of an integer into a fraction. Thus a constructor that takes only one parameter is considered a conversion function; it specifies a conversion from the type of the parameter to the type of the class.

So we can write a conversion function or we can use a constructor of single argument for conversion operation. We cannot use both, we have to write one or the other. Be careful about this. We don't use conversion functions often but sometimes it is useful to write them. The conversion operators are useful for defining an implicit conversion from the class to a class whose source code we don't have access to. For example, if we want a conversion from our class to a class that resides within a library, we cannot define a single-argument constructor for that class. Instead, we must use a conversion operator.

It makes our code easier and cleaner to maintain. It is important that pay more attention while defining a class. A well-defined class will make their use easy in the programming.
Following is the code of the example stated above.

```
/* This program defines a class Fraction which stores numerator and
denominator of a fractional number separately. It also overloads the
addition operator for adding the fractional numbers so that exact results
can be obtained.
*/

#include <stdlib.h>
#include <math.h>
#include <iostream.h>

// class definition
class Fraction
{
        public:
                        Fraction();
                        Fraction( long num, long den );
```


© Copyright Virtual University of Pakistan

```
                    void display() const;
                    Fraction operator+( const Fraction &second ) const;
        private:
                    static long gcf( long first, long second );
                    long numerator, denominator;
};

// ------------Default constructor
Fraction::Fraction()
{
        numerator = 0;
        denominator = 1;
}
// ------------Constructor
Fraction::Fraction( long num, long den )
{
        int factor;
        if( den == 0 )
          den = 1;
        numerator = num;
        denominator = den;
        if( den < 0 )
        {
                    numerator = -numerator;
                    denominator = -denominator;
         }
        factor = gcf( num, den );
        if( factor > 1 )
        {
                    numerator /= factor;
                    denominator /= factor;
        }
}
// ------------Function to print a Fraction
void Fraction::display() const
{
        cout << numerator << '/' << denominator;
}

// ------------Overloaded + operator
Fraction Fraction::operator+( const Fraction &second ) const
{
        long factor, mult1, mult2;
        factor = gcf( denominator, second.denominator );
        mult1 = denominator / factor;
        mult2 = second.denominator / factor;
        return Fraction( numerator * mult2 + second.numerator * mult1,
                                denominator * mult2 );
}
```


© Copyright Virtual University of Pakistan

```
//------------ Greatest common factor
// computed using iterative version of Euclid's algorithm
long Fraction::gcf( long first, long second )
{
        int temp;
        first = labs( first );
        second = labs( second );
        while( second > 0 )
        {
                temp = first % second;
                first = second;
                second = temp;
        }
        return first;
}

//main program
void main()
{
        Fraction a, b( 23, 11 ), c( 2, 3 );
        a = b + c;
        a.display();
        cout << '\n';
        system("pause");
}
```

The output of the program is as follows

```
91/33
```

Here is an example from the real world. What happens if we are dealing with currency? The banks deal with currency by using computer programs. These programs maintain the accounts by keeping the track of transactions and manipulating deposits and with drawls of money. Suppose the bank declares that this year the profit ratio is 3.76 %. Now if the program calculates the profit as 3.67 % of the balance, will it be exactly in rupees and paisas or in dollars and cents? Normally, all the currencies have two decimal digits after decimal point. Whenever we apply some kind of rates in percentage the result may become in three or four decimal places. The banks cannot afford that the result of 90 paisas added to 9 rupees and 10 paisas become 10 rupees and 1 paisa. They have to accurate arithmetic. So they do not rely on programs that use something like *double* precisions to represent currencies. They would have rather written in the program to treat it as a string. Thus 9 is a string, 10 is a string and the program should define string addition such that the result of addition of the strings .10 and .90 should be 1.00. Thus, there are many things that happen in real world that force us as the programmer to program the things differently. So we do not use native data types.

The COBOL, COmmon Business Oriented Language has a facility that we can represent the decimal numbers exactly. Internally it (the language) keeps them as strings in the memory. There is no artificial computer representation of numbers. Now a day, the languages provide us the facility by which we can define a data type

© Copyright Virtual University of Pakistan

according to a specific requirement, as we defined *fraction* to store numerator and denominator separately. By using this *fraction* data type, we never loose precision in arithmetic. The same thing applies to the object like currency, where we can store the whole number part and the fractional part both as separate integers and never loose accuracy. But whenever we get into these classes, it is our responsibility to start writing all of the operators that are required to make a complete class.

Lecture No. 34

Reading Material

Deitel & Deitel - C++ How to Program ***Chapter 8***

Summary

- Arrays of Objects
- Dynamic Arrays of Objects
- Overloading new and delete Operators
- Example of Overloading new and delete as Non-members
- Example of Overloading new and delete as Members
- Overloading [] Operator to Create Arrays

Arrays of Objects

A class is a user-defined data type. Objects are instances of classes the way *int* variables are instances of *int*s. Previously, we have worked with arrays of *int*s. Now, we are going to work with arrays of objects.

The declaration of arrays of user-defined data types is identical to the array of *primitive* data types.

Following is a snapshot of our veteran *Date* class:

```
/* Snapshot of Date class discussed in previous lectures */
class Date
{
    private:
      int  day, month, year;

    public:
     /* Parameterless constructor, it is created by the compiler automatically when we
```


© Copyright Virtual University of Pakistan

```
    don't write it for any of our class. */
  Date( )
  {
      cout << "\n Parameterless constructor called ...";
      month = day = year = 0;
  }
  /* Parameterized constructor; has three ints as parameters. */
  Date(int month, int day, int year)
  {
      cout << "\n Constructor with three int parameters called ...";
      this->month = month;
      this->day = day;
      this->year = year;
  }

  ~Date ( )
  {
      cout << "\n Destructor called ...";
  }
  . . .
  . . .
};
```

Consider the example of declaring an array of *10* date objects of *Date* class. In this case, the declaration of arrays will be as under:
Following is the declaration:

```
Date myDates [10] ;
```

With this line (when this line is executed), we are creating 10 new objects of *Date* class. We know that a constructor is called whenever an object is created. For every object like *myDate[0], myDate[1], ... myDate[9]*, the constructor of the *Date* class is called. Theimportant thing to know here is that which constructor of Date class is being called to construct objects of the array *myDates*. As we are not doing any initialization of the array objects explicitly, the default constructor (parameterless constructor) of the Date class is called. Remember, the default constructor is defined by the C++ compiler automatically for every class that has no parameterless constructor defined already. In our case of *Date* class, we have defined a parameterless constructor, therefore, the compiler will not generate default constructor automatically.

We can also initialize the array elements at the declaration time. This initialization is similar to that done for native data types. For *int* array, we used to do initialization in the following manner:

```
int array [10] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 } ;
```

Similarly, we initialize *Date* array while declaring it:

```
Date yourDate [3] = { Date(10, 24, 1980), Date(06, 14, 1985), Date(07, 09,1986) };
```


© Copyright Virtual University of Pakistan

The above statement will call the parameterized constructor *Date (int month, int day, int year)* of *Date* class to create three objects of *myDate* array. This parameterized constructor carries out initialization of the objects data member (*month, day, year*) with the values supplied as arguments to it.

It will be interesting to know, how the following statement works:

Date myDate [10] = { Date(09, 03, 1970), Date(08, 23, 1974) } ;

We are trying to declare an array of 10 Date objects while supplying only initialization values for the first two elements. At first, we might be doubtful if the statement is compiled successfully. Not only it compiles successfully but also does the initialization of the first two objects (*myDate[0], myDate[1]*). What will happen to the remaining objects in the array? Actually, all the *10* objects are created successfully by the above statement. The parameterized constructor is called for the first two objects (*myDate[0], myDate[1]*) and parameterless constructor is called for the remaining objects (*myDate[2], myDate[3], …, myDate[9]*).

You might have noticed that at the array initialization stage, we have explicitly called parameterized constructor of *Date* for every object. We may specify only the argument when a constructor with only one parameter is called.

```
/* A snapshot of String class discussed in previous lectures */

class String
{
private :
        char  *buf ;
public:
        // Constructors
        String ();
        String( const char  *s )
        {
                buf  =  new  char [ 30 ];
                strcpy (buf,s);
        }
        . . .
        . . .
};
```

For example, in the above-mentioned case of *String* class, we have a constructor that is accepting one argument of type *char* *. While writing our code, we can declare and initialize an array of *String*s as follows:

```
String message [10] = {     "First line of message\n",
                            "Second line of message\n",
                            String( "Third line of message\n"),
                            String( )
```


© Copyright Virtual University of Pakistan

```
};
```

See the initializing arguments for first two objects i.e, (*message[0], message[1]*) in the array. Here only one string is being passed. Therefore, for the first two objects, constructor with one parameter of type *char* * of *String* class is called automatically. That constructor is *String (char * str)*. For the third object (*message[2]*), the same constructor with one *char* * as parameter is being called explicitly. For fourth object (*message[3]*), parameterless constructor i.e., *String ( )* is being called explicitly, though, this was optional as parameterless constructor is called up automatically when no initialization is made. As there is no explicit initialization for the remaining six objects, the parameterless constructor is called up automatically.

Can we create arrays of objects dynamically? As usual, the answer is yes. Let's discuss it in detail.

Dynamic Arrays of Objects

Consider the following statement:

```
1.  String *text ;
2.  text = new String [5] ;
```

In line 1, we have declared a pointer *text* of *String* type.

In line 2, we are creating an array of 5 objects of *String* type. This statement allocates space for each object of the array, calls the parameterless constructor for each object and starting address of the first object is assigned to the pointer *text*.

The important point to be noted here is that in line 2, we can't initialize objects because there is no way to provide initializers for the elements of an array allocated with *new*.

The default constructor (parameterless constructor) is called for each element in the array allocated with *new*. Remember, the default constructor for a class is generated by C++ compiler automatically if it is not defined already in the class definition.

To deallocate these arrays of objects, the *delete* operator is used in the same way as it is used for the native data types.

There are few cautions that should be taken care of while performing these operations of allocation and deallocation with arrays of objects.

Firstly, while deallocating an array allocated with *new* operator, it is important to tell the compiler that an array of objects is being deleted. The brackets (*[]*) are written in our *delete* statement after the *delete* keyword to inform the *delete* operator that it is going to delete an array. The consequences of using the wrong syntax are serious. For example, if we want to delete previously created array of five *String* objects using the following statement:

```
delete  text;  // Incorrect syntax of deleting an array
```

The delete operator in this case will not be aware of deleting (deallocating) an array of objects. This statement will call the destructor only for the object pointed by the ***text*** pointer i.e. *String[0]* and deallocate the space allocated to this object. The requirement is to call the destructor for all the objects inside the array and deallocate the space allocated to all of these objects. But on account of the wrong syntax, only the first object is deleted and the remaining four objects (*String[1], String[2], String[3], String[4]* pointed by *text[1], text[2], text[3], text[4]* respectively) remain

© Copyright Virtual University of Pakistan

in the memory intact. The memory space occupied by these four objects results in *memory leak* as the same program or any other program on the same computer cannot use it unless it is deallocated.

Calling the destructor while destroying an object becomes essential when we have allocated some memory in free store from inside the object (usually from within the constructor).

To destroy an array of objects allocated on free store using the *new* operator, an array equivalent of *delete* operator is used. The array equivalent of *delete* operator is to write empty square brackets after the *delete* keyword (*delete []*). So the correct statement is:

```
delete [] text ;
```

This statement destroys the whole array properly. It calls destructor for each object inside the array and deallocates the space allotted to each object. Actually, by looking at the brackets (*[]*) after *delete*, the compiler generates code to determine the size of the array at runtime and deallocate the whole array properly. Here, it will generate code to deallocate an array of *5* objects of *String* type.

If we create an array of *Date* objects and want to delete them without specifying array operator: It will look as under:

```
// Bad Technique: deleting an array of objects without []
// for a class that is not doing dynamic memory allocation internally
Date * ppointments;
appointments = new Date[10];
. . .
delete appointments; // Same as delete [] appointments;
```

Although, this is good to deallocate an array of objects without specifying array operator ([]) as there is no dynamic memory allocation occurring from inside the *Date* class. But this is a bad practice. In future, the implementation of this class may change. It may contain some dynamic memory allocation code. So it is always safer to use array operator (*[]*) to delete arrays.

Can we overload *new* and *delete* operators? Yes, it is possible to overload *new* and *delete* operators to customize memory management. These operators can be overloaded in global (non-member) scope and in class scope as member operators.

Overloading of new and delete Operators

Firstly, we should know what happens when we use *new* operator to create objects. The memory space is allocated for the object and then its constructor is called. Similarly, when we use *delete* operator with our objects, the destructor is called for the object before deallocating the storage to the object.

When we overload *new* or *delete* operators, it can only lead to a change in the allocation and deallocation part. The call to the constructor after allocating memory while using *new* operator and call to the destructor before deallocating memory while

© Copyright Virtual University of Pakistan

using *delete* operator will be there. These calls to constructors and destructors are controlled by the language itself and these cannot be altered by the programmer.

One of the reasons of overloading *new* and *delete* operators can be their limited current functionality. For example, we allocate space on free store using the *new* operator for *1000 int*s. It will fail and return *0* if there is no contiguous space for *1000 int*s in free store. Rather the free store has become fragmented. The total available memory is much more than *1000 int*s , but in fragments. There is no contiguous segment of at least *1000 int*s. The built-in *new* operator will fail to get the required space but we can overload our own *new* operator to de-fragment the memory to get at least *1000 int*s space. Similarly, we can overload *delete* operator to deallocate memory.

In the embedded and real-time systems, a program may have to run for a very long time with restricted resources. Such a system may also require that memory allocation always takes the same amount of time. There is no allowance for heap exhaustion or fragmentation. A custom memory allocator is the solution. Otherwise, programmers will avoid using *new* and *delete* altogether in such cases and miss out on a valuable C++ asset.

There are also downsides of this overloading. If we overload *new* or *delete* operator at global level, the corresponding built-in *new* or *delete* operator will not be visible to whole of the program. Instead our globally written overloaded operator takes over its place all over. Every call to *new* operator will use our provided *new* operator's implementation. Even in the implementation of *new* operator, we cannot use the built-in *new* operator.
Nonetheless,when we overload *new* operator at a class level then this implementation of *new* operator will be visible to only objects of this class. For all other types (excluding this class) will still use the built-in *new* operator. For example, if we overload *new* operator for our class *Date* then whenever we use *new* with *Date*, our overloaded implementation is called.

```
Date* datePtr = new Date;
```

This statement will cause to call our overloaded *new* operator. However, when we use *new* with any other type anywhere in our program as under:

```
int* intPtr = new int [10];
```

The built-in *new* operator is called. Therefore, it is safer to overload *new* and *delete* operators for specific types instead of overloading it globally.

An important point to consider while overloading *new* operator is the return value when the *new* operator fails to fulfill the request. Whether the operator function will return *0* or throw an exception.

Following are the prototypes of the *new* and *delete* operators:

```
void * operator new ( size_t size ) ;
void operator delete ( void * ptr) ;
```

The *new* operator returns a *void* * besides accepting a parameter of whole numbers *size_t*. This prototype will remain as it is while overloading *new* operator. In the implementation of overloaded *new* operator, we may use *calloc( )* or *malloc( )* for memory allocation and write some memory block's initialization code.

© Copyright Virtual University of Pakistan

The delete operator returns nothing (*void*) and accepts a pointer of *void* * to the memory block. So the same pointer that is returned by the *new* operator, is passed as an argument to the *delete* operator. Remember, these rules apply to both if operators (*new* and *delete*) are overloaded as member or non-member operators (as global operators). Importantly, whenever we use these operators with classes, we must know their sequence of events that is always there with these operators. For *new* operator, memory block is allocated first before calling the constructor. For *delete* operator, destructor for the object is called first and then the memory block is deallocated. Importantly, our overloaded operators of *new* and *delete* only takes the part of allocation and deallocation respectively and calls to constructors and destructors remain intact in the same sequence.

Because of this sequence of events, the behavior of these *new* and *delete* operators is different from the built-in operators of *new* and *delete*. The overloaded *new* operator returns *void* * when it is overloaded as non-member (global). However, it returns an object pointer like the built-in *new* operator, when overloaded as a member function. It is important to understand that these operator functions behave like *static* functions when overloaded as member functions despite not being declared with *static* keyword. *static* functions can access only the *static* data members that are available to the class even before an object is created. As we already know that *new* operator is called to construct objects, it has to be available before the object is constructed. Similarly, the *delete* operator is called when the object has already been destructed by calling destructor of the object.

Example of Overloading new and delete as Non-members

Suppose we want *new* to initialize the contents of a memory block to zero before returning it. We can achieve this by writing the operator functions as follows:

```
/* The following program explains the customized new and delete operators */

#include <iostream.h>
#include <stdlib.h>
#include  <stddef.h>

// ---------- Overloaded new operator
void *  operator  new ( size_t   size )
{
  void * rtn = calloc( 1, size ); // Calling calloc() to allocate and initialize memory
  return  rtn;
}


// ---------- Overloaded delete operator
void  operator  delete  ( void *   ptr )
{
  free( ptr );  // Calling  free() to deallocate memory
}
void  main()
```


© Copyright Virtual University of Pakistan

```
{
   // Allocate a zero-filled array
   int *ip = new int[10];
   // Display the array
   for ( int i = 0; i < 10; i ++ )
      cout << " " << ip[i];
   // Release the memory
   delete [] ip;
}
```

The output of the program is as follows.

```
0 0 0 0 0 0 0 0 0 0
```

Note that the *new* operator takes a parameter of type *size_t*. This parameter holds the size of the object being allocated, and the compiler automatically sets its value whenever we use *new*. Also note that the *new* operator returns a *void* pointer. Any *new* operator we write must have this parameter and return type.

In this particular example, *new* calls the standard C function *calloc* to allocate memory and initialize it to zero.

The *delete* operator takes a *void* pointer as a parameter. This parameter points to the block to be deallocated. Also note that the *delete* operator has a *void* return type. Any *delete* operator we write, must have this parameter and return type.

In this example, *delete* simply calls the standard C function *free* to deallocate the memory.

Example of Overloading new and delete as Members

```
// Class-specific new and delete operators

#include <iostream.h>
#include <string.h>
#include <stddef.h>
const int MAXNAMES = 100;

class Name
{
public:
  Name( const char *s ) { strncpy( name, s, 25 ); }
  void display() const { cout << '\n' << name; }
  void * operator new ( size_t size );
  void operator delete( void * ptr );
  ~Name() {};  // do-nothing destructor
private:
  char name[25];
};
```


© Copyright Virtual University of Pakistan

```
//--------- Simple memory pool to handle fixed number of Names
char  pool[MAXNAMES] [sizeof( Name )];
int inuse[MAXNAMES];

//--------- Overloaded new operator for the Name class
void *  Name :: operator  new( size_t size )
{
   for( int p = 0; p < MAXNAMES; p++ )
         if( !inuse[p] )
         {
            inuse[p] = 1;
            return  pool + p;
         }
   return  0;
}

//---------- Overloaded delete operator for the Names class
void  Name :: operator  delete( void *ptr )
{
   inuse[((char *)ptr - pool[0]) / sizeof( Name )] = 0;
}

void main()
{
   Name *  directory[MAXNAMES];
   char name[25];
   for( int i = 0; i < MAXNAMES; i++ )
   {
      cout  <<  "Enter name # "  <<  i+1  <<  ": ";
      cin  >>  name;
      directory[i]  =  new   Name( name );
   }
   for( i = 0; i < MAXNAMES; i++ )
   {
      directory[i]->display();
      delete directory[i];
   }
}
```

The output of the above program is given below.

```
Enter name # 1: ahmed
Enter name # 2: ali
Enter name # 3: jamil
Enter name # 4: huzaifa
Enter name # 5: arshad
Enter name # 6: umar
Enter name # 7: saleem
Enter name # 8: kamran
```


© Copyright Virtual University of Pakistan

```
Enter name # 9: babar
Enter name # 10: wasim

ahmed
ali
jamil
huzaifa
arshad
umar
saleem
kamran
babar
wasim
```

This program declares a global array called *pool* that can store all the *Name* objects expected. There is also an associated integer array called *inuse*, which contains *true/false* flags that indicate whether the corresponding entry in the *pool* is in use.
When the statement *directory[i] = new Name(name)* is executed, the compiler calls the class's *new* operator. The *new* operator finds an unused entry in *pool*, marks it as used, and returns its address. Then the compiler calls *Name*'s constructor, which uses that memory and initializes it with a character string. Finally, a pointer to the resulting object is assigned to an entry in directory.
When the statement *delete directory[i]* is executed, the compiler calls *Name* 's destructor. In this example, the destructor does nothing; it is defined only as a placeholder. Then the compiler calls the class's *delete* operator. The *delete* operator finds the specified object's location in the array and marks it as unused, so the space is available for subsequent allocations.
Note that *new* is called before the constructor, and that *delete* is called after the destructor.

Overloading [ ] Operator to Create Arrays

We know that if we overload operators *new* and *delete* for a class, those overloaded operators are called whenever we create an object of that class. However, when we create an array of those class objects, the global *operator new( )* is called to allocate enough storage for the array all at once, and the global *operator delete( )* is called to release that storage.
We can control the allocation of arrays of objects by overloading the special array versions of *operator new[ ]* and *operator delete[ ]* for the class.

Previously, while employing global *new* operator to create an array of objects, we used to tell the *delete* operator by using the array operator(*[]*) to deallocate memory for an array. But it is our responsibility to provide or to overload different type of *new* and different type of *delete*.

There is a common problem when working with arrays. While traversing elements from the array, we might run off the end of the array. This problem might not be caught by the compiler. However, some latest compilers might be able to detect this.

© Copyright Virtual University of Pakistan

```
int  iarray [10] ;
for ( int  i = 0;  i  <  100;   i++ )
{
        // Some code to manipulate array elements
}
```

If a variable is used in the condition of the loop instead of the constant value, the probability of that error increases. Variable might have an entirely different value than that anticipated by the programmer.

We can overcome this problem of array bound by overloading array operator '*[]*'. As usual before overloading, we should be clear about the functionality or semantics of the array operator. We use array operator to access an element of array. For example, when we write *iarray[5]*, we are accessing the 6th element inside array *iarray*. As we want to check for validity of index every time, an array element is accessed. We can do this by declaring the size of the array using *#define* and checking the *index* against the size every time the array is accessed.

```
#define  MAXNUM  1000
int  iarray [MAXNUM];
```

Below is the syntax of declaration line of overloaded array operator:

```
int& operator  [] ( int   index ) ;
```

In the body of this operator, we can check whether the *index* is greater or equal to the *MAXNUM* constant. If this is the case, the function may throw an exception. At the moment, the function only displays an error message. If *index* is less than *MAXNUM* and greater than or equal to zero, a reference to the value at the *index* location is returned.

Let's write a class *IntArray* and see the array manipulation.

```
/*
The following example defines the IntArray class, where each object contains
an array of integers. This class overloads the [] operator to perform
range checking.
*/

#include <iostream.h>
#include  <string.h>

class  IntArray
{
public:
    IntArray( int len );
    int  getLength( ) const;
    int &  operator[] ( int index );
    ~IntArray( );
private:
    int  length;
    int  *aray;
```


© Copyright Virtual University of Pakistan

```
};


// -------------Constructor
IntArray :: IntArray( int len )
{
   if( len  >  0 )
   {
      length  =  len;
      aray  =  new  int[len];
      // initialize contents of array to zero
      memset( aray, 0, sizeof( int ) * len );
   }
   else
   {
      length  =  0;
      aray  =  0;
   }
}

// -------------Function to return length
inline int  IntArray :: getLength()  const
{
   return  length;
}
// -------------Overloaded subscript operator
// Returns a reference
int  & IntArray :: operator []( int index )
{
   static int dummy  =  0;
   if( (index  >=  0)  &&
      (index < length) )
      return aray[index];
   else
   {
      cout  <<  "Error: index out of range.\n";
      return  dummy;
   }
}

// -------------Destructor
IntArray :: ~IntArray()
{
   delete  aray;
}
void  main()
{
   IntArray numbers( 10 );
   int  i;
   for( i = 0; i < 10; i ++ )
```


© Copyright Virtual University of Pakistan

```
    numbers[i]  =  i;        // Use numbers[i] as lvalue
  for( i  =  0; i  <  10; i++ )
    cout  <<  numbers[i]  <<  '\n';
}
```

This program first declares *numbers* of type *IntArray* object that can hold ten integers. Later, it assigns a value to each element in the array. Note that the array expression appears on the left side of the assignment. This is legal as the *operator[]* function returns a reference to an integer. This means the expression *numbers[i]* acts as an alias for an element in the private array and it can be the recipient of an assignment statement. In this situation, returning a reference is not simply more efficient but also necessary.

The *operator[]* function checks whether the specified *index* value is within range or not. If it is within the range, the function returns a reference to the corresponding element in the private array. If it is not, the function prints out an error message and returns a reference to a *static* integer. This prevents out-of-range array references from overwriting other regions of memory while causing unexpected program behavior.

Tips

- The *default constructor* is defined by the C++ compiler automatically for every class that has no default constructor (parameterless constructor) defined already.
- The default constructor (parameterless constructor) is called for each element in the array allocated with *new*.
- The *new* operator returns a *void* *, accepts a parameter of type *size_t*.
- The *delete* operator returns nothing (*void*) and accepts a pointer of *void* * to the memory block.
- With *new* operator function, a block of memory is allocated first and then constructor is called.
- With *delete* operator, destructor of the object is called first and then memory block is deallocated.
- By overloading *new* and *delete* operators, only allocation and deallocation part can be overridden.
- The same pointer that is returned by the *new* operator, is passed as an argument to the *delete* operator. These rules apply to both, if operators (*new* and *delete*) are overloaded as member or non-member operators (as global operators).

© Copyright Virtual University of Pakistan

- By overloading the array operator ([]), one can implement mechanism to check for array bound.



© Copyright Virtual University of Pakistan

Lecture No. 35

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 11

11.1, 11.2, 11.3, 11.3.1, 11.3.2,

11.4, 11.4.1, 11.4.2, 11.4.3, 11.5

Summary

18) Streams
19) Source and Destination of streams:
20) Formatted Input and Output
21) Recap Streams
22) Buffered Input/Output
23) Methods with streams
24) Examples using streams

Streams

We have been discussing the concept of 'Streams' from the very beginning of the course. In this lecture, various aspects of this concept will be discussed. There are two types of streams i.e. input streams and output streams. Before going into minute details, we will see what actually these two types are. You are well aware of the terms 'cin' and 'cout', used several times in our previous programs. We have used *cout* for output and *cin* for input. Similarly, the terms of file input and file output are very much known to us. We leaned how to write in files and how to read from files. These are also streams. Let's have a look on the things and functions we can do with streams. There are some specific functions for input and output in C. *printf* and *scanf* are normally used in C. In these functions, we have to tell what type of data we are using and in which variable. Streams are counterpart of this in C++. The input output system in C++ is streams. As the name applies, it's a stream of bytes. As told earlier, it may also be termed as a door through which program can communicate with the outside world. For entering some data, we use *cin* stream while the data is read from the keyboard and stored in some variable. To display data on the screen, we can take help of *cout*.

For making the things more comprehensive, we will consider the example of a class-room. Suppose you are sitting in the class and listening to this lecture. All of a

© Copyright Virtual University of Pakistan

sudden, the students are asked by the instructor to go out of the classroom. How would you do it? You will get up from the seat and walk through the door. But you can see that all of you cannot go through the door simultaneously. You would go through the door one by one. Similarly if you have to come to the classroom, you will enter one by one and sit on the seats. In a way you are forming a sequence of people in this case. Stream is a sequence of bytes. It is an ordered sequence. Let's compare it with the door example. The person who enters first will go out of the door first. The person who enters behind someone will go out behind that person. Similarly streams are also ordered sequence. The thing that enters first into the stream will go out first. You should think streams as ordered sequence of bytes. Byte is a unit of measure. A byte can store one character, so you can think of an ordered sequence of characters.

As programmers, we communicate with our programs in English through the keyboard. We may be typing the letters, abc or the numbers, 012 on the keyboard. These all are the characters. In the programs, we store these characters in variables of different data types. Sometimes, these may be in some of our objects. On the keyboard, we type the character 'a' that is stored in some variable *c* in our program. How these two are linked? This link is formed in *cin* stream. Consider *cin* as a pipe or a door. The character 'a' is entered from one side and then a conversion takes place i.e. character is converted into its binary representation, stored in the character variable named *c*. So there is an implicit conversion happening. The same thing happens, if we have an integer *i* and press the key 1 from the keyboard. The digit 1 travels as a character but inside it is stored as number. You have to be careful while dealing with this concept. We have talked about the ASCII characters. On the keyboard, we have alphabets, numbers and symbols. When you press the number key from the keyboard, it goes at number inside the computer. In general terms, it is a character. It means that when you enter a key on your keyboard, a character sequence is generated that later goes into the computer. This character sequence has some binary representation. It does not mean a sequence of characters, but a code that goes inside the computer on pressing a key on the keyboard. This code is called as ASCII code. It is the binary representation of a character. Here both the characters 'A' and 'B' have some binary representation. Similarly '0','1','2' have some binary representation. It does not mean that the character representation of '1' is also '1'. If you are aware of the ASCII table (you have already written a program to display the ASCII table), it will be evident that the binary representation of character '1' is some other value. Similarly all the numbers 1, 2, 3 etc have some ASCII value. So whenever you press any key on the keyboard, its ASCII code goes inside the computer.

Now when we use *cin* stream to read some number from the keyboard and store it in the integer variable, its binary representation is ignored and the value is stored. So *cin* is performing this transformation operation. It is taking a character ASCII code and knows that it is supposed to represent some number. It has the ability to convert it into the appropriate number before putting it into the integer variable *i*. What happen if we use *cin* to read some value and store it in some integer variable and press some alphabet key instead of numeric keys. Some error will occur. So in the *cin* stream, an error can be detected.

Let's us look at the collection of input and output classes. These are objects having functions. This collection of classes and their functions are known as input-output

© Copyright Virtual University of Pakistan

streams in the C++. The most common things that we have been using are *cin*, used to get input from the keyboard and *cout* employed to display something on the screen. So one is input stream and the other one is output stream. Since we are not going to discuss the object- oriented programming in this course. We will not discuss the hierarchy through which these classes are derived. We have some objects for input stream and out put stream e.g. *cin* and *cout* respectively. Being objects, they have member methods that we can call. They also have the operators like '<<' and '>>', as used in the earlier programs. These operators '<<', '>>' are heavily overloaded. What does this mean? If we write *cin >> i;* here *i* is an integer. Automatically *cin* will take a character in ASCII code from the keyboard, convert it into number and store it into the integer variable. On the other hand, if we write *cin >> c;* where *c* is a character data type. When we press a key from the keyboard, it will be stored in *c* as a character. So the stream extractor operator of cin (i.e. >>, which gets the data from the stream and stores it into the variable) is already overloaded. It knows how to behave with int, char, float etc data type and what sort of conversion is required. In case of float number, we have decimal point, *cin* knows how to treat it besides converting and storing it into a float variable. Similarly if we use any character pointer i.e. string, the same >> operator has the capability of reading strings too. Obviously, one operator can't perform all these functions. It seems that we are using the same operator. Internally, this operator is overloaded. It is not the same for int, char, float, string and so on. But due to the operator overloading, its usage is very simple. We just write *cin >> i;* it works perfectly.

Source and Destination of streams

As earlier said that streams are sort of door or pipe between two things. What are these two things? For every stream, there must be some source and some destination. For *cin*, the source is normally keyboard and the destination can be an ordinary variable i.e. native-data type variable. It could be some area of memory or our own data type, i.e. object for which we have overloaded the operator and so on. So always there is a source and there is a destination.

cout is output stream. It takes the data from the program and presents it in human readable form. It also has some source and destination. The source may be some file, or the region in memory or the processor or a simple variable or our own object of our data type. The destination is normally screen. The destination can be a file, screen, or printer etc. You have used file input and file output so you know how it works. When we talk about area in memory, it may be an array that we read or write. It could also be a character string which is itself an area in the memory.

"Every stream has an associated source and a destination"

Now we will talk about yet another concept i.e. the state of stream. What does it mean? We say that *cin >> i* ; where *i* is an integer. When we give it the input 'a' and press the key 'enter', the stream knows that it is a bad input. So it is capable of signaling and setting its state specifying that some thing not good has been done. So from a program, we can always test whether the state of stream is right or not. We should carry out all kinds of error checking, debugging and error handling while writing programs. We don't want to manipulate bad data. So checking for this everywhere will be good. For example, if we write a simple program that takes two

© Copyright Virtual University of Pakistan

integers from key-board, divides one number by the other and displays the result. Following will be the code segment.

```
int i, j ;
cin >> i ;
cin >> j ;
cout << i / j ;
```

Now we have to see what happens if the user gives a value 0 for j. We don't want to divide it by zero as even the computer does not know how to do it. When we have zero in j, our program probably will work through an exception or error before coming to a halt. If we trap this error inside the program, it would be much nicer. We can say if j is not zero, it will be good to carry out the division. So error checking and handling is always important. The same thing applies to I/O streams. When we execute input or output operation, we should check whether the operation has been carried out correctly or not. To do this, we can check the state of the stream.

Here is a simple example showing the simple use of streams.

```
/* Avoiding a precedence problem between the stream-insertion operator and the
conditional operator. */

#include<iostream>

int main()
{
        int x,y;
        cout<< "Enter two integers: ";
        cin>>x>>y;
        cout<<x << (x ==y ? " is" : " is not") <<" equal to "<< y;
        return 0;
 }
```

The output of the program:

```
Enter two integers: 3  3
3 is equal to 3
```

Formatted Input and Output

Other things that the streams provide us are a capability of formatted input and output. We have been using *cin* and *cout* very simply without any formatting. As the output of a program, we do not want that numbers should be printed in a way that makes it difficult to read and understand. We want to format the output in a way that the numbers are placed correctly at the correct position on the paper. You might have seen the electricity bills or telephone bills printed by the computers. First the empty bills are printed in the printing press containing headings with blank boxes to put in the bill entries. Then the computer prints the entries in these boxes from the system. These entries (data regarding the bill) are printed at correct places on the bill. Sometimes, you see that the entries are properly printed. That is not a computer-fault

© Copyright Virtual University of Pakistan

but only due to poor paper adjustment in the printer. The printing of these entries is carried out by with the use of formatted output.

The second example is the display of a matrix on the screen. Suppose, we want that the numbers of a column are displayed up and down in a column. Similarly the second column should be displayed and so on. At first, we will format these things.

When we do word processing, we type a paragraph. The lines of the paragraph are left justified but ragged on right hand side. In word processing, we have a choice to justify the paragraph. By doing this, the left and right margins of the paragraph are put in a straight line while adjusting the space between the words. Now look what happens if we want to print a string with *cout* and want it left or right justified. This is what we call formatting the output. Similarly, we want to print the value of *pi* which is stored in a variable as 3.1415926. But we want that it should be printed as 3.141 that means up to three decimal places.
There should be a method of formatting it. Thus by formatting the output, the presented representation (which we read as human being) can be different from the internal representation. So we can do a lot of formatting with these streams.

Besides, there are member functions with the streams. Let's look at the member functions of *cin*. The first one is the *get* function. We can use it by writing:

cin.get();

The notation explains that *cin* is an object (cin is an object of input stream) and *get* is a member function of it. This function reads a character. In this case, it reads a single character from key board and returns it as *cin* is calling it. We have two variants of this get function with *cin*. One is that *cin.get* returns a character. W e can write it as under:

c = cin.get() ;

The second method is *cin.get(character variable) i.e.* one character at a time. It works with characters, not through number or string. It is one character at a time.

The second function of *cin* is the *read* function. This function differs from the *get* function in the way that it returns a buffer instead of a single character. So we can point to a buffer and tell the number of characters to be read. We normally provide a delimiter, a specific character up to which we want to read. Normally we use the new line character as a delimiter and read a single line at a time.

Thus, we have three ways of obtaining input with *cin,* which is an object of type input stream (istream). If we create an object of type *istream*, it will also get these functions as it is derived from the same class.

We have seen that there are many methods and operators associated with *cin*. The same thing applies to *cout*. *cout* is the output stream which usually, presents the data from the computer in human readable form. The operator associated with *cout* is the stream insertion (<<). That means we insert the operator in the stream and the stream displays the output on the screen. So the operator with cout is << and it displays the

© Copyright Virtual University of Pakistan

value of the data variable that we provide it after the << sign. Thus to display the value of an integer variable i, we can write *cout << i* ; If we want to format the output, it can also be done here.

The cout has a function *write* with it. This function can be used if we want to write a chunk of data from the buffer. Similarly, to output a single character, *cout* has the function named *put*. It can be written as:

cout.put(character variable) ;

Here, it will display the value of the character variable.

Recap streams

Streams are nothing but an ordered sequence of bytes.
They allow data to move from one part of the computer to another which may be the screen or key board from and to, or from memory or files on disc and so on.
Byte stream is used to connect the source and the destination.
These byte streams are implemented as objects. Being objects, they do have their member functions and have their member operators. The member operators are heavily overloaded to allow these streams to handle a variety of data types.
The streams have a state that can be checked by us. We have used eof (end of file) with the file reading. This is a way to check the state of the stream.

While using I/O streams, we have to include some header files. Whenever we use *cin* and *cout*, the file *iostream.h*, is included in which all these classes and objects have been defined. For the formatted input and output, we manipulate the streams. To do stream manipulations, we have to include a header file having the name *iomanip.h*. We can understand that *iomanip* is a short hand for input output manipulation.

Now let's take a look at the standard streams which are provided to our programs. Whenever we write a C++ program and include *iostream.h* in it, we get a stream for input (reading) that is *cin.* This is a built in thing. We can use this object. Similarly, for output (writing), we get *cout*. Other than these, we get some other streams by default. These include *cerr* (read as c error) and clog. To understand these streams, we have to talk about buffered input and output.

Buffered Input/Output

In computers, most of the components relate to electronics like chips, memory, micro processor etc. There are also electro-mechanical things like disc. The key board itself is an electro mechanical accessory. The electro mechanical parts of the computer are normally very slow as compared to the electronic components. So there is a difference between the two in terms of speed. Secondly, every input/output operation costs computer time. Input/output costs and the I/O devices (keyboard, monitor and disc etc) are slower as compared to the speed of the microprocessor and the memory being used. To overcome this speed difference, we use the mechanism, called buffered input/output. Suppose, we have a program which executes a loop. That loop outputs a number in each of iteration to store in a file on the disc. If we write the output number

© Copyright Virtual University of Pakistan

to the disc in each iteration, it will be the horrendously wastage of computer time. It means that the disc is electro mechanical device. Similarly in each iteration, the mechanical movement takes time. But if we gather the data and write it to the disc, there will be one mechanical movement. The heads of the disc will move mechanically once to a point and the whole chunk of data will be written on the disc. This is the more efficient way of using the disc. So whenever we have a program that writes the output data to the disc, it will be nice to collect the output data (numbers) and write it on the disc in one write operation instead of writing the numbers one by one. The area where we gather the numbers is known as buffer. The example stated in this case is the buffered output. In this case, the output does not go directly to the disc. We first gather the data in a buffer and then write it on the disc.

Now think about another situation. Suppose we have a program that performs very complex calculations. It means that there is a *while* loop that performs so heavy calculations that each of the iteration takes, say one minute and then provides the result. Now we want to write that output to a file on the disc and see the iteration number of the loop on the screen. We do not want to write the output to the disc after each iteration. We gather the data in a buffer. In the meantime, we want to see the loop counter on the screen. If we gather this output of counter number in a buffer, it may happen that the buffer gathers the iteration numbers for 250 iterations before displaying it on the screen. Thus, we see on the screen the numbers 1, 2, 3 …..250, when 250 iterations have been performed. There are again 250 numbers gathered at one time. We see numbers 251, 252 ……500, when 500 iterations have been performed. When we start the program, there will be two buffers gathering data. One buffer gathers the data to write to the disc and the other gets the data of iteration numbers to display on the screen. As we said that each iteration takes one minute, meaning that the iteration numbers will not be seen for a long time. Rather, these will be shown after 250 iterations (i.e. 250 minutes). During this period, we do not show any thing on the screen. Here, we are not sure whether the program is executing properly as we do not see any iteration number on the screen. We want to know after every minute that loop has executed once. So after every minute, the loop counter should be displayed on the screen, so that at any time we could see how many iterations have been performed. For this, we need unbufffered output on the screen. Thus, in the same program we require buffered and unbuffered output.

Now these requirements are contradictory with different issues. These are met in our system with the *cerr* object. *cout* is a buffered output. We cannot see it as nowadays compilers are very intelligent. If you know UNIX, or command prompt and input output redirection, we can actually see this in operation. Perhaps, you can create an example to understand this. For the moment just try to understand that *cout* is buffered output. It gathers data and sends it to the screen. On the other hand, *cerr* is unbuffered output. It will show the data on the screen at the same time when it gets it. So *cerr* is an output stream, an *ostream* object but unbuffered. It shows data immediately. So we can use something like the *cerr* object that will show how many loops have been executed. It will use something like *cout* to write on the disk to buffer the output. In this case, we are getting efficiency in addition to information. Normally we use *cerr* object in C++ for this purpose. Besides, we also have *clog*. It is also known as standard log having detailed information of the log. To collect information of the program, we write it with *clog*. Normally when we execute our programs- *cout*, *cerr* and *clog*, all are connected to screen. We have ways to direct them at different

© Copyright Virtual University of Pakistan

destinations. That depends on the operating system. Suppose, we specify the buffer size, normally the operating system or compiler does this for us. A typical size of buffer is 512 bytes. When the information is of 512 byte size, output will take place. But in the program, we may want at some point that whatever is in the buffer, show them. Is there a way of doing that? The normal mechanism is flush. Flush the stream. The flush command forces the data from the buffer to go to its destination which is normally a screen or file and make the buffer empty.

Uptil now, we have been using two things to end the line. One is new line character i.e. "\n". When we are displaying something on the screen, it makes the next output to start from the next line. The cursor moves to the left margin of the next line on the screen. New line is just a character. The other one was *endl*. If we write *cout << endl;* It seems that the same thing happens i.e. the cursor moves to the left margin of the new line. But *endl* actually does something else. It flushes the output too. As a result, it seems that *cout* is unbuffered i.e. its output is immediately available on the screen. Depending on the compiler and operating system, you may or may not see the buffered effect. But one thing is clear that while reading the source code, you will know where *cout* is used and where *cerr*. Typically, that is also the programming style where the output of *cerr* is informative and small. It shows that where is the control in the program. The output of *cout* is more detailed and the actual output of the program. There are benefits of these things in the code. In case of *cin*, it is alone. For output, we have *cout*, *cerr* and *clog*. In DOS, we have two more output streams i.e. *caux* (auxiliary input output stream) and *cprn* (printer output). These are no more relevant now.

Predefined Stream Objects:

Object	**Meaning**
cin	Standard input
cout	Standard output
cerr	Standard error with unbuffered output.
clog	Standard error with buffered output
caux	Auxiliary (DOS only)
cprn	Printer(DOS only)

Now let's take a look at the operators associated with these streams. We have been using the stream insertion operators '<<' with *cout*. We need to understand that how these operators are implemented. Using *cout*, we can chained the output. It means that we can write as *cout << "The value of the first integer is " << i;* This is the single *cout* statement. How does that work? What happens is the first part goes to *cout*. In this case, it is the string "The value of the first integer is". The data travels in the direction of the arrows. This string is inserted in the stream and displayed on the screen. What about the rest of the statement i.e. *<< i;* this again needs *cout* on the left side to be executed. It should look like as *cout << i;* Once, we have defined that this is the behavior expected by us. Then, we understand that this is exactly the way it has been programmed. The stream insertion operator '<<' is overloaded for the output stream and it returns the reference of the output stream. The syntax of stream insertion operator is:

```
ostream& ostream::operator << (char *text);
```


© Copyright Virtual University of Pakistan

The important thing to note is that this operator returns the reference to the *ostream* object itself. Whenever we write a chained output statement, it is executed from left to right. So *cout << " The value of the first integer is"* is processed first from left to right. The process is that this character string is displayed on the screen. As per the prototype and definition, it returns the reference to the *ostream* object. In this case, the object was *cout*, so a reference to the *cout* object is returned. Now the rest of the statement becomes as *cout << i;* It is processed quite nicely. This allows the stream insertions to be chained. It is also applicable to the input. So if we say something like *cin >> i >> j;* both *i* and *j* are integers. Now again, it is processed from left to right. This is the *istream* and the extraction operator will return the reference to the *istream* object i.e. *cin*. So at first, the *cin >> i* is processed which will return the reference to the *cin* object. The rest of the statement seems as *cin << j;* It is important to understand how these operators work. You can see their prototypes that they return *iostream* objects themselves. That is the why, we can chain them. Now let's see what are the other various methods associated with these input output streams.

Methods of streams

There are some other functions associated with *cin* stream. We have used some of them. We have used *get()* and *read()* methods with input stream. Another member function of *cin* is *getline()*. It reads a complete buffer i.e. the number of character specified up to a delimiter we specify. We can write something like:

```
cin.getline(char *buffer, int buff_size, char delimiter = '\n')
```

The character data is stored in **buffer*. *buff_size* represents the number of characters to be read. If we specify its value 100, then *getline* will read 99 characters from the keyboard and insert a null character in the end. As you know, in C++ every character string ends with a null character. We can also give it a delimiter. Sometimes, we may want to read less character. Normally, the delimiter is the new line character. So while typing on the keyboard, if we press the enter key then it should stop reading further and put the data into the variable *buffer*. So there is a *getline* fiunction.

There are some other interesting functions also. When we use *cin.get()*, a character is read. We can throw back the character gotten by this *get* function by using the *unget()* function. So we can use *cin.unget() that* will return the most recently (last) gotten single character.

We have a function *peek()*, also written as *cin.peek()*; The purpose of this function is that we can see the next character that we are going to get. This function returns the next character that would be read if we issue *cin.get()*.

All these functions (getline, get, read, unget and peek) are implemented as member functions of the input class.

Similarly, there are functions associated with *cout*. We have *cout.putline()*; which outputs a buffer. Actually we have no need of this function because *cout*, itself, knows how to handle character strings. Then we have *cout.write()*; which can perform a raw, unformatted output. The function *cout.put()*; is like a formatted output. It performs character by character output. We can do many formatting conversions by using the stream insertion operator (i.e. <<) with *cout*. We can write an overloaded function of

© Copyright Virtual University of Pakistan

stream insertion (<<) to input or output a complex number. We know that a complex number has two parts i.e. real and imaginary. We can write the overloaded function such that if we give two numbers with space between them it could read it. We can also write it as that it could read two numbers (that are real and imaginary parts of a complex number) separated by comma. Thus there may be different ways to write the overloaded operator.

The white space character is very significant. We can show it by a simple example. Suppose we have an array *name* of 60 characters. We get a name from the user in this array by using *cin* and then display this string by *cout*. The code segment for this purpose can be written as:

```
char name [60] ;
cin >> name ;
cout << name ;
```

Now when the user enters the name, suppose it enters 'naveed malik' that is a name containing two words with a space between them. When we display this name by using *cout*, only 'naveed' is displayed on the screen. It means that only one word 'naveed' was stored in the array. The reason for it that the streams (cin, cout) are sensitive to white space character that is treated as a delimiter. Now where is the second word 'malik'. It has not got deleted yet. It is in the buffer of the stream. This example will read like the following:

```
char nam1 [30], name2 [30] ;
cin >> name1 >> name2 ;
```

Thus, we have two character arrays now. We can write 'naveed malik' and press enter. The first part before space (naveed) will go to the first array *name1* when that array is used with *cin*. We will write another *cin* with *name2* and the second part (malik) will go to the second array *name2*. So things don't disappear. They stay in the buffer till you actually expect them. We have to be careful about that.

Examples using streams

A simple example showing the use of getline function.

```
// A simple example showing the use of getline function.
#include <iostream.h>

int main()
{
   const int SIZE = 80;
   char buffer[SIZE];

   cout << " \n Enter a sentence: \n" ;
   cin.getline(buffer, SIZE);

   cout << " The sentence entered is: \n" << buffer << endl;
   return 0;
}
```


© Copyright Virtual University of Pakistan

Output of the program.

```
Enter a sentence:
this is a test
The sentence entered is:
this is a test
```

A simple example showing the use of read and write functions.

```
// A simple example showing the use of read and write functions.
#include <iostream.h>

int main()
{
   const int SIZE = 80;
   char buffer[SIZE];

   cout << " \n Enter a sentence: \n" ;
   cin.read(buffer, 20);

   cout << " The sentence entered was: \n";
   cout.write(buffer, cin.gcount());
   cout << endl;
   return 0;
}
```

Output of the program.

```
Enter a sentence:
This is a sample program using read and write functions
The sentence entered was:
This is a sample pro
```


© Copyright Virtual University of Pakistan

Lecture No. 36

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 11

11.6, 11.7

Summary

25) Stream Manipulations
26) Manipulators
27) Non Parameterized Manipulators
28) Parameterized Manipulators
29) Format State Slags
30) Formatting Manipulation
31) Showing the Base
32) Scientific representation
33) Exercise

Stream Manipulations

After having a thorough look into the properties and object definition of I/O streams, we will discuss their manipulations. Here, there is need of some header files to include in our program the way, *cin* and *cout* are used. We include *iostream.h* and *fstream.h* while using file manipulations. In case of manipulation of the I/O streams, the header file with the name of *iomanip.h* is included. It is required to be included whenever there is need of employing manipulators.

As discussed earlier, we can determine the state of a stream. The states of the stream can be determined. For example, in case of *cin*, we can check where the end of file comes.

For state- checking, these stream objects have set of flags inside them. These flags can be considered as an integer or long integer. The bit position of these integers specifies some specific state. There is a bit for the end of file to test. It can be written as under:

```
cin.eof() ;
```

It will return the state of end of file. The bit will be set if the file comes to an end. Similarly, there is a *fail* bit. This bit determines whether an operation has failed or not.

© Copyright Virtual University of Pakistan

For example, an operation could be failed due to a formatting error. The statement *cin.fail*; will return the value of the *fail* bit. As this statement returns a value, we can use it in an 'if statement' and can recover the error. Then there is a *bad* bit. This bit states that data has lost. The presence of this bit means that some data has lost during I/O operation. So we can check it in the following manner.

```
cin.bad();
```

It can be used in parentheses as a function call that will allow us to check whether the operation failed or successful. Similarly we can also check for' *good',* that is a bit showing that everything is good. This bit will be set if *fail* and *bad* bits are not set. We can check this bit as *cin.good* ; and can find out whether the input operation was successful. If some bit like *bad* has been set, there should also be a mechanism to clear it. For this, we have

```
cin.clear() ;
```

as a member function for these objects. This will reset the bits to their normal good state. This is a part of checking input stream.

Manipulators

Whenever carrying out some formatting, we will want that the streams can manipulate and a number should be displayed in a particular format. We have stream manipulators for doing this. The manipulators are like something that can be inserted into stream, effecting a change in the behavior. For example, if we have a floating point number, say pi (л), and have written it as *float pi = 3.1415926* ; Mow there is need of printing the value of pi up to two decimal places i.e. 3.14 . This is a formatting functionality. For this, we have a *manipulator* that tells about width and number of decimal points of a number being printed. Some manipulators are parameter less. We simply use the name of the manipulator that works. For example, we have been using *endl*, which is actually a manipulator, not data. When we write *cout << endl* ; a new line is output besides flushing the buffer. Actually, it manipulates the output stream. Similarly *flush* was a manipulator for which we could write *cout << flush* that means flushing the output buffer. So it manipulates the output.

A second type of manipulators takes some argument. It can be described with the help of an example. Suppose we want to print the value of pi up to two decimal places. For this purpose, there should be some method so that we can provide the number i.e. two (2) up to which we want the decimal places. This is sent as a parameter in the manipulators. Thus we have the parameterized manipulators.

Let's have a look on what streams do for us. We know that streams are like ordered sequence of bytes and connect two things i.e., a source and a destination. In the middle, the stream does some conversion. So it may take some binary representation of some information and convert it into human readable characters. It may also take characters and convert them into an internal representation of data. With in the conversion of data, we can do some other things. For example, you might have seen that if a system prints computerized cheques, it puts some special characters with the numbers. If there is a cheque for four thousand rupees, this amount would be written on it as ****4000.00. The idea of printing * before the amount is that no body could

© Copyright Virtual University of Pakistan

insert some number before the actual amount to change it. As we don't want that somebody has increased the amount on the cheque from Rs 4000 to Rs 14000.The printing of * before the amount is a manipulation that we can do with input or output objects. We can also tell the width for a number to be printed. So there are many conversions that we can do. We can use fill characters like * as mentioned in the example of cheque printing. To accomplish all these tasks, there are different methods. So it becomes a little confusing that the same work is being done through 2-3 different methods. Some are inline manipulators like *endl*. We can use it with << and write inline as *cout << endl* ; The same work could be done with the *flush* method. We could also write *cout.flush* ; Thus it is confusing that there is an inline manipulator and a function for the same work.

Non-Parameterized Manipulators

Let's start with simple manipulators. We have been dealing with numbers like integers, floats etc for input and out put. We know that our number representations are associated with some base. In daily life, the numbers of base 10 are used in arithmetic. When we see 4000 written on a cheque, we understand that it is four thousands written in the decimal number system (base 10). But in the computer world, many systems are used for number representation that includes binary (base 2), octal (base 8), decimal (base 10) and hexadecimal (base 16) systems. A simple justification for the use of these different systems is that computers internally run on bits and bytes. A byte consists of eight bits. Now if we look at the values that can be in eight bits. 256 values (from 0 to 255) can be stored in eight bits. Now consider four bits and think what is the highest number that we can store in four bits. We know that the highest value in a particular number of bits can be determined by the formula $2^n - 1$ (where n is the number of bits). So the highest value that can be stored in four bits will be $2^4 - 1$ i.e. 15. Thus the highest value, we can store in four bits is 15 but the number of different values that can be stored will be 2^n i.e. 16 including zero. Thus we see that while taking half of a byte i.e. four bits, 16 (which is the base of hexadecimal system) different numbers can be stored in these four bits. It means that there is some relationship between the numbers that have a base of some power of two. So they can easily be manipulated as bit format. Thus four bits are hex. What about eight (octal)? If we have three bits, then it is $2^3 = 8$, which is the base of octal system. Thus, we can use three bits for octal arithmetic.

We can use manipulators to convert these numbers from one base to the other. The manipulators used for this purpose, can be used with *cin* and *cout*. These are non-parameterized manipulators. So if we say the things like *int i = 10* ; Here *i* has the decimal value 10. We write *cout << i* ; and 10 is being displayed on the screen. If we want to display it in octal form, we can use a manipulator here. If we write

```
cout << oct << i ;
```

it will display the octal value of i (10) which is 12. This manipulator converts the decimal number into an octal number before displaying it. So the octal representation of 10 which is 12, will be displayed on the screen. Similarly if we write

```
cout << hex << i ;
```


© Copyright Virtual University of Pakistan

Here hex stands for the hexadecimal. *hex* is the manipulator that goes into the out put stream before *i* and manipulates the stream by converting the decimal number into a hexadecimal number. As a result, the hexadecimal value of 10 is displayed on the screen. If we have a number in octal or hexadecimal system , it can be converted into a decimal number by putting the *dec* manipulator in the stream like

```
cout << dec << i ;
```

These (oct, hex and dec) are very simple inline manipulators without any argument. There is also a manipulator for white space. The white space has a special meaning. It is a delimiter that separates two numbers (or words). In *cin* and *cout*, the white space acts as a delimiter. If we want that it should not act as a delimiter, it can used as a *ws* manipulator. This manipulators skips white space. This manipulator takes no argument. This *ws* manipulator is sometime useful but not all the times. The following table shows the non-parameterized manipulators and their description.

Manipulator	**Domain**	**Effect**
dec	In / Out	Use decimal conversion base
hex	In / Out	Use hexadecimal conversion base
oct	In / Out	Use octal conversion base
endl	Output	Inserts a new line and flush the stream
ends	Output	Terminate a string with NULL
flush	Output	Flush the stream
ws	Input	Skip leading whitespace for the next string extraction only

The base becomes important while doing programming of scientific programs. We may want that there is the hexadecimal presentation of a number. We have discussed the justification of using hexadecimal or octal numbers, which is that they match with bits. Here is another justification for it. Nowadays, computers are just like a box with a button in front of them. A reset button is also included with the main power button. While seeing the pictures or in actual Miniframe and mainframe computers, you will notice that there is a row of switches in front of them. So there are many switches in front of these computers that we manipulate. These switches are normally setting directly the values of registers inside the computer. So you can set the value of register as 101011 etc by switching on and off the switches. We can do that to start a computer or signaling something to computer and so on. There are a lot of switches in front of those computers ranging between 8 to 16. You have to simply remember what is the value to start the computer. Similarly, it will require the reading the combinations of switches from the paper to turn on the computer. This combination tells you which switch should be on and which should be off. As a human being instead of remembering the whole pattern like 10110000111 etc, it could be easy to remember it as 7FF. Here we are dealing with HEX numbers. For the digit 7, we need four bits. In other words, there is need to set four switches. The pattern of 7 is 0111. So we set 7 with this combination. For F, all the four bits should be on as 1111 and so on. Thinking octal and hexadecimals straight away maps to the bits. It takes a little bit of practice to effectively map on the switches. On the other hand, decimal does not map to those bits. What will be its octal number in case of decimal number 52.? You have to calculate this. What is the binary representation of 52? Again you have to

© Copyright Virtual University of Pakistan

calculate. There are a lot of things which you have to calculate. On the hand, if we say 7ABC in case of HEX, we are mapping the number straight away on four bits. In octal system, we map the number on three bits. A is ten so it will be 1010 so you can quickly set the switches. It may not be relevant today. But when you are working with big computers, it will become quite relevant. There are many times when we have to manipulate binary data using mechanical device while thinking in hexadecimal and octal terms. In the language, you have the facility to set the base. You can use *setbase(), hex, oct, dec* and *setf*. There are many ways of doing the same thing. Programmers write these languages. Therefore they make this facility available in the language as built in.

Parameterized Manipulators

Suppose we want to print the number 10 within a particular width. Normally the numbers are written right justified. In case of no action on our part, *cout* displays a number left justified and in the space required by the number. If we want that all numbers should be displayed within the same particular width, then the space for the larger number has to be used. Let's say this number is of four digits. Now we want that there should be such a manipulator in the output that prints every number in a space of four digits. We have a manipulator *setw* (a short for set width), it takes as an argument the width in number of spaces. So to print our numbers in four spaces we write

```
cout << setw(4) << number ;
```

When printed, this number gets a space of four digits. And this will be printed in that space with right justification. By employing this mechanism, we can print values in a column (one value below the other) very neat and clean.

Now in the example of printing a cheque, we want that the empty space should be filled with some character. This is required to stop somebody to manipulate the printed figure. To fill the empty space, there is need of manipulator *setfill*. We can write this manipulator with *cout* as the following

```
cout << setfill (character) ;
```

where the *character* is a single character written in single quotes. Usually, in cheque printing, the character * is used to fill the empty spaces. We can use any character for example, 0 or x. The filling character has significance only if we have used *setw* manipulator. Suppose, we are going to print a cheque with amount in 10 spaces. If the amount is not of 10 digits, the empty space will be filled with *. Thus the usage of *setfill* is there where we use *setw* for printing a number in a specific width. So if we want to print an amount in 10 spaces and want to fill the empty spaces with *, it can be written as under.

```
cout << setfill(*) << setw(10) << amount ;
```

Thus the manipulators can also be of cascading nature. The stream insertion operator (<<) is overloaded and every overload of it returns a reference to the *cout* object itself. This means that while working from left to right, first the fill character will be set returning a reference to *cout* . Later, its width will be set to 10 character and return a

© Copyright Virtual University of Pakistan

reference to *cout again*. Finally, the amount will be displayed in the required format. Thus, we have manipulated the stream in two ways. Example of a pipe with two bends can help it understand further. Now whatever figure goes into it, its width and the fill character is set and things are displayed in the space of 10 characters. If we want to print an amount of Rs 4000 on the cheque, it will be printed on the cheque as ******4000. Thus, we have two manipulators, *setw* and *setfill* which are used with *cout*.

Let's further discuss the same example of cheque. In real world, if we look at a computer printed cheque , the amount is printed with a decimal point like 4000.00 even if there is no digit after decimal point. We never see any amount like 4000.123, as all the currencies have two- digit fractional part. Thus, we examine that the fractional part has been restricted to two decimal places. The decimal digits can be restricted to any number. We have a manipulator for this purpose. The manipulator used for this purpose is *setprecision*. This is a parameterized manipulator. It takes an integer number as an argument and restrict the precision to that number. If we write

```
cout << setprecision (2) << float number ;
```

The above statement will display the given float number with two decimal places. If we have the value of pi stored in a variable, say *pi*, of type float with a value of 3.1415926 and want to print this value with two decimal places. Here, manipulator *setprecision* can be used. It can be written as under.

```
cout << setprecision (2) << pi ;
```

This will print the value of *pi* with two decimal places.

Now think about it and write on the discussion board that whether the value of pi is rounded or truncated when we print it with setprecision manipulator. What will be the value of pi with five decimal places and with four decimal places? Will the last digit be rounded or the remaining numbers will be truncated?

At this point, we may come across some confusion. We have learned the inline manipulators that are parameter less. For these, we simply write *cout << hex << number;* which displays the number in hexadecimal form. There is also a parameterized manipulator that performs the same task. This manipulator is *setbase*. It takes the base of the system (base, to which we want to format the number) as an argument. Instead of using *oct*, *dec* and *hex* manipulators, we can use the *setbase* manipulator with the respective base as an argument. So instead of writing

```
cout << oct << number ;
```

we can write

```
cout << setbase (8) << number ;
```

The above two statements are equivalent in the way for having the same results. It is a matter of style used by one of these manipulators. We can use either one of these, producing a similar effect. The *cout << setbase(8)* means the next number will be printed in the base 8. Similarly *cout << setbase(16)* means the next number will be printed in hexadecimal (base 16) form. Here a point to note is that *setbase (0)* is the same as *setbase(10).*

© Copyright Virtual University of Pakistan

Following is the table, in which the parameterized manipulators with their effect are listed.

Manipulator	Domain	Effect
resetioflags(long f)	In / Out	Clear flags specified in f
setbase (int b)	In / Out	Set numeric conversion base to b (b may be 0, 8, 10 or 16)
setfill (int c)	Output	Set fill character to c
setiosflags(long f)	In / Out	St flags specified in f
setprecision (int p)	Output	Set floating point precision to p
setw (int w)	Output	Set field width to w

Format State Flags

We have discussed that there are flags with the stream objects. This set of flags is used to determine the state of the stream. The set includes *good, fail, eof* etc that tells the state of the stream. There is also another set of flags comprising the ones for input/output system (ios). We can use *setioflag,* and give it as an argument a *long* number. Different bit values are set in this number and the flags are set according to this. These flags are known as format state flags and are shown in the following table. These flags can be controlled by the *flags, setf* and *unsetf* member functions.

Format state flag	**Description**
ios::skipws	Skip whitespace character on an input stream.
ios::left	Left justify output in a field, padding characters appear to the right if necessary.
ios::right	Right justify output in a field, padding characters appear to the left if necessary.
ios::internal	Indicate that a number's sign should be left justified in a field and a number's magnitude should be right justified in that same field (i.e. padding characters appear between the sign and the number).
ios::dec	Specify that integers should be treated as decimal (base 10) values.
ios::oct	Specify that integers should be treated as octal (base 8) values.
ios::hex	Specify that integers should be treated as hexadecimal (base 16) values.
ios::showbase	Specify that the base of a number is to be output ahead of the number(a leading 0 for octals, a leading 0x or 0X for hexadecimals).
ios::showpoint	Specify that floating-point numbers should be output with a decimal point. This is normally used with ios::fixed.
ios::uppercase	Specify that uppercase letters (i.e X and A through F) should be used in the hexadecimal integers and the uppercase E in scientific notation.
ios::showpos	Specify that positive and negative numbers should be preceded by a + or - sign, respectively.

© Copyright Virtual University of Pakistan

ios::scientific	Specify output of a floating-point value in scientific notation.
ios::fixed	Specify output of a floating-point value in fixed point notation with a specific number of digits to the right of the decimal point.

Let's talk about more complicated things. We discussed a parameterized manipulator *setw* that sets the width to print in the output. There is an alternative for it i.e. the member function, called '*width()'*. This function also takes the same parameter and an integer, in which width the things are to display or read. This function applies to both input and output stream. For this, we write *cin.width (7).* This will create a format field of the width of 7 characters for an input. Now we write *cout.width (10)* ; this will set the width of output field to 10. With it, the next number to be printed will be printed in 10 spaces. Thus *setw*, inline manipulator has the alternative function *cin.width* and *cout.width* with single argument.

It equally applies to the *setprecision*. This is the parameterized, inline- manipulator that sets the places after the decimal point. There is a member function as well in these objects that is *precision*. The *setprecision* is an inline manipulator, used along with stream insertion (<<). If we want to do the same thing with a function call, *cout.precision(2)* is written. It has the same effect as that of *cout << setprecision (2)*. Thus we have different ways of doing things.
We have used *setfill* manipulator. Here is another member- function i.e. *cout.fill*. The behavior of this function is exactly the same. We simply write *cout.fill('*')* ; identical to *cout << setfill('*')*. The filling character is mostly used whenever we use financial transactions but not necessarily. We can also use zero to fill the space.
So *fill* and *setfill, width* and *setw, precision* and *setprecision* and almost for every inline manipulator, there are member functions that can be called with these streams. The member functions are defined in *iostream.h. However*, the manipulators are defined in *iomanip.h*. Normally we have been including *iostream.h* in our programs to utilize the member functions easily. But inclusion of a header file 'iomanip.h file is must for the use of manipulators.
We should keep in mind that when we can write inline manipulators in the following fashion.

```
cout << setw (7) << i ;
```

And in the next line we write

```
cout << j ;
```

Here the *setw* manipulator will apply to *i* only and not after that to *j*. This means that inline manipulators apply only to the very next piece of data i.e. output. It does not apply to subsequent output operations.

Formatting Manipulation

We can adjust the output to left side, right side or in the center. For this purpose, we have a member function of the object whose syntax is as under:

cout.setf(ios:: *flag,* ios:: adjust field)

The *setf* is a short for set flag. The flags are long integers, also the part of the objects. They are the bit positions, representing something. Here we can set these flags. The

© Copyright Virtual University of Pakistan

flags of *adjustfield* are set with values i.e. left, right, left | right and internal. The description of these is as follows.

Value of flag	Meaning	Description
left	Left-justify output	Justifies the output to left side
right	Right-justify output	Justifies the output to right side
left | right	Center output	Centralized the output
internal	Insert padding	Places padding between signs or base indicator and the first digit of a number. This applies only to number values and not to character array.

Following is the code of a program that shows the effects of these manipulators.

```
//This program demonstrate the justified output

#include <iomanip.h>
#include <iostream.h>

void main()
{
        int i = -1234;
        cout.setf(ios::left, ios::adjustfield);
        cout << "|" << setw(12) << i << "|" << endl;
        cout.setf(ios::right, ios::adjustfield);
        cout << "|" << setw(12) << i << "|" << endl;
        cout.setf(ios::internal, ios::adjustfield);
        cout << "|" << setw(12) << i << "|" << endl;
        cout.setf(ios::left | ios::right,
        ios::adjustfield);
        cout << "|" << setw(12) << i << "|" << endl;
        cin >> i ;
 }
```

Following is the output of the above program.

```
|-1234      |
|      -1234|
|-      1234|
|      -1234|
```

We have discussed two types of manipulators for base, the parameter less manipulator in which we look for *oct*, *dec* and *hex*. On the other hand, there is a parameterized manipulator *setbase* which takes an integer to set the base. It uses 0 or 10 for decimal, 8 for octal and 16 for hexadecimal notations.
Now we have a generic function *setf* that sets the flags. We can write something like

cout.setf(ios::hex)

© Copyright Virtual University of Pakistan

The hex is defined in *ios*. It has the same effect. It sets the output stream, in this case *cout*, to use hexadecimal display for integers. So it is the third way to accomplish the same task. The use of these ways is a matter of programming style.

Showing the base

Now there should be someway to know which base has the number output by the programmer. Suppose we have a number 7ABC, then it be nothing but hexadecimal. What will be the nature of 7FF. It is hexadecimal. However, the number 77 (seven seven) is a valid number in all of different basis. We have a built-in facility *showbase*. It is a flag. We can set the *showbase* for output stream that will manipulate the number before displaying it. If you have the *showbase* falg on (by default it is off), a number will be displayed with special notations. The *setf* function is used to set the flag for the base field. Its syntax is as under:

cout.setf(ios::*base*, ios::basefield);

Here base has three values i.e. oct, dec and hex for octal, decimal and hexadecimal systems respectively. If the basefield is set to *oct* (octal), it will display the number with a preceding zero. It shows that the number is in octal base. If the basefield is set to *hex* (hexadecimal), the number will be displayed with a preceding notation 0x. The number will be displayed as such if the basefield is set to *dec* (decimal). If there is a number, say 77, it will be difficult to say that it is in octal, decimal or hexadecimal base, a valid number for all the three systems. However, if we output it with the use of *showbase*, it will be easy to understand in which base the output number is being represented. The following example, demonstrates this by showing the number (77) along with the base notation.

```
/* This program demonstrate the use of show base.
It displays a number in hex, oct and decimal form.
*/

#include <iostream.h>

void main()
{
   int x = 77;
   cout.setf(ios::showbase);
   cout.setf(ios::oct,ios::basefield);   //base is 8
   cout << x << '\n';                    //displays number with octal notation
   cout.setf(ios::hex,ios::basefield);   //base is 16
   cout << x << '\n';                    //displays number with hexadecimal notation
   cout.setf(ios::dec,ios::basefield);
   cout << x << '\n';
}
```

Following is the output of the program.

```
 0115
0x4d
77
```


© Copyright Virtual University of Pakistan

Scientific Representation

When the numbers get bigger, it becomes difficult to write and read in digits format. For example, one million will be written as 1000000. Similarly hundred million will be 100000000 (one with eight zeros). How will we display the number which is of 20-digit long? For this, we use scientific notation. To do it, a manipulator *ios:: scientific* can be used. If the flag in *setf* to the *scientific* is set, it can be written as

```
cout.setf(ios::scientific, ios::floatfield) ;
```

Then the floating point numbers will be displayed in scientific notation. A number in scientific is like 1.946000e+009. So we can set the state of output stream to use scientific notation for outputting a number.
To do the scientific notation off and restore the default notation, we set the flag in *setf* function to *fixed* (which is a short for fixed point notation). This can be written as

```
cout.setf(ios::fixed, ios::floatfield) ;
```

Uppercase/Lowercase Control

Similarly, we have a manipulator *ios::uppercase*. While using this manipulator, the e in scientific notation is written in uppercase i.e. E. If we are using hexadecimal numbers, then the characters of it will be displayed in uppercase letters as A, B, C, D, E and F.

Exercise

- We have been using matrices i.e. a two dimensional array. As an exercise, try to print out a matrix of three rows and three columns, in such a way that it should be nicely formatted. The numbers should be aligned as we write it in a neat and clean way on the paper. You can use the symbol | at the start and end of each row as we don't have a so big square bracket to put around three rows. To be more elegant to print a matrix, we can use a proper graphic symbol to put square brackets around the matrix instead of using | symbol. In the ASCII table, there are many symbols that we can use to print in our programs. We have the integer values of these symbols in the table. Suppose you have a value 135 of a symbol. Now to print this symbol, press the 'alt' key and keeping the key pressed enter the integer value i.e. 135 from the num pad of the key board, release the 'alt' key. Now you will see that symbol on the screen. For the value 135, the symbol is ç. In programming, we can provide this symbol to be printed as a single character in single quotes. For this, put a single quote and then enter the symbol in the way stated above and then put the single quote. It will be written as 'ç'. Find out proper symbols from the ASCII table that can comprise to put a square bracket around the matrix.
- Write simple programs to demonstrate the use of different manipulators and examine their effects.

© Copyright Virtual University of Pakistan

Lecture No. 37

Reading Material

Deitel & Deitel - C++ How to Program ***Chapter 11***

11.3, 11.3.1, 11.4, 11.4.1

Summary

- Overloading Insertion and Extraction Operators
- Example 1
- Example 2
- Tips

Overloading Insertion and Extraction Operators

We are already aware that while overloading operators, functions are written and spirit or behavior of the operators (+, -, *, /) is maintained in their implementations. Similarly the operator's spirit is kept intact while overloading stream insertion and extraction operators.

We get an integer as input by writing the following lines:

```
int i;
cin >> i;
```

Have a look on the stream extraction operator's (>>) behavior here. The similar behavior is maintained when we overload this stream extraction operator (>>) or stream insertion operator (<<).

There are couple of important things to take care of, before starting implementation for overloading an operator:

The first thing to see is the type of the operator i.e. whether the operator is binary or unary. The binary operator takes two operands while unary operator takes one. The number of operands for an operator cannot be changed while overloading it.
Secondly, the programmer has to take care of, what an operator is returning back. For example, in case of addition (+), it returns back the result of addition. So the cascading statement like *a + b + c;* can be executed successfully. In this case, at first, *b + c* is executed. The result of this operation is returned by the *operator +*, to add it in the variable *a*. So in actual, the operation is carried out as:

© Copyright Virtual University of Pakistan

a + (b + c).

We want to overload stream extraction (>>) and insertion (<<) operators which are actually already overloaded. See the code lines below:

```
1.    int i = 123;
2.    double d = 456.12;
3.    float f = 789.1;
4.
5.    cout << i << " ";
6.    cout << d << " ";
7.    cout << f;
```

You can see the lines 5, 6 and 7. The same stream insertion operator (<<) has been used with different data types of *int*, *double* and *float*. Alternatively, these lines (5, 6 and 7) can be written within statement of one line:

```
cout << i << " "<< d << " "<< f;
```

Similarly, the stream extraction operator (>>) is used with different data types in the following manner:

```
cin >> i;
cin >> d;
cin >> f;
```

Here, stream extraction operator is used with different data types of *int*, *double* and *float*. The three lines given above can be written in one cascading line:

```
cin >> i >> d >> f;
```

The file *iostream.h* contains the operator overloading declarations for these stream insertion (<<) and extraction (>>) operators for native data types. The declarations inside this file look like the following:

```
istream& operator>>(char*);
istream& operator>>(unsigned char* p)
{
  return operator>>((char*)p);
}
istream& operator>>(signed char*p)
{
   return operator>>((char*)p);
   }
istream&  operator>>(char& c);
istream& operator>>(unsigned char& c)
    {
     return operator>>((char&)c);
    }
```


© Copyright Virtual University of Pakistan

```
istream& operator>>(signed char& c)
 {
  return operator>>((char&)c);
  }
istream& operator>>(int&);
istream& operator>>(long&);
#if defined(__GNUC__)
__extension__ istream& operator>>(long long&);
__extension__ istream& operator>>(unsigned long long&);
#endif
istream&  operator>>(short&);
istream& operator>>(unsigned int&);
istream& operator>>(unsigned long&);
istream&  operator>>(unsigned
short&); #if _G_HAVE_BOOL
istream& operator>>(bool&);
#endif
istream& operator>>(float&);
istream& operator>>(double&);
istream&  operator>>(long
double&); istream& operator>>(
__manip func)
 {
  (*func)(*this); return *this;
 }
istream& operator>>(__imanip func)
 {
  return (*func)(*this);
 }
istream& operator>>(streambuf*);
```


© Copyright Virtual University of Pakistan

function returns. Therefore, it does not make sense to return references of the objects, passed by value to the function.

As we are declaring this operator function as *friend* of our class *Vehicle*, the *private* members of *Vehicle* will be accessible to this operator function. For example, *tyre* is a *private* data member of type *int* inside *Vehicle* class and inside the operator function's implementation, we can access it by simply writing *v.tyre* as:

```
output << v.tyre;
```

The above statement actually is:

```
cout << v.tyre;
```

tyre is of native data type *int*. The *output* will work for native data types as it is actually *cout*, which is overloaded for all native data type. We are constructing a building using the basic building blocks. We can use the already used bricks to construct new walls. Similarly, while writing out programs, we implement our overloaded operators using the already available functionality of native data types.

Here is how we overload stream insertion operator (<<) for our *Date* class:

```
#include <iostream.h>
class Date
{
   friend ostream& operator << ( ostream & os, Date d );
          // this non-member function is a friend of class date
  . . .
  . . .
};


ostream & operator << ( ostream & os, Date d )
{
  os << d.day << "." << d.month << "." << d.year; // access private
       data
                                                  // as friend
  return os;
};
```

Likewise, we can overload stream extraction operator (>>). All the conditions for overloading this operator are similar to that of stream insertion operator (>>). It cannot be a member operator, always a non-member operator function, declared as *friend* of the class to be overloaded for. It returns an object of type *istream &*, accepts first parameter of type *istream &*. There is one additional restriction on extraction operator (>>) i.e. the second parameter is also passed by reference as that object is modified by this operator function. For our *Date* class, it is declared as:

```
istream & operator >> ( istream & input, Date & d );
```

Note that second parameter can also be passed by reference for insertion operator (<<) but that is not mandatory and may be used to gain performance. But in case of extraction operator (>>), it is mandatory to have second parameter of reference type.

© Copyright Virtual University of Pakistan

Example 1

Following is our* Date *class containing the overloaded insertion (<<) and extraction (>>) operators:

```
/* Date class containing overloaded insertion and extraction operators. */
# include <iostream.h>

class Date
{
    public:
    Date( )
    {
            cout << "\n Parameterless constructor called ...";
            month = day = year = 0;
     }

    ~Date ( )
    {
//          cout << "\n Destructor called ...";
     }

   // Methods, not directly related to the example have been taken out from the class

      friend ostream & operator << ( ostream & os, Date d );
      friend istream & operator >> ( istream & is, Date & d );

     private:
                int month, day, year;
};

ostream & operator << ( ostream & os, Date d )
{
      os << d.day << "." << d.month << "." << d.year; // access private data of
                                                      //Date being a friend
      return os;
};

istream & operator >> ( istream & is, Date& d )
{
  cout << "\n\n Enter day of the date: ";
  cin >> d.day;
  cout << " Enter month of the date: ";
  cin >> d.month;
  cout << " Enter year of the date: ";
  cin >> d.year;
```


© Copyright Virtual University of Pakistan

```
  return  is;
};

main(void)
{

   Date date1, date2;
   cout <<  "\n\n Enter two dates";
   cin  >> date1 >> date2;
   cout  <<  "\n Entered date1 is: "   <<   date1 << "\n Entered date2 is: " << date2;

}
```

The output of the program is:

```
Parameterless constructor called ...
Parameterless constructor called ...

Enter two dates: ...

Enter day of the date: 14
Enter month of the date: 12
Enter year  of the date: 1970

Enter day of the date: 05
Enter month of the date: 09
Enter year  of the date: 2000

Entered date1 is: 14.12.1970
Entered date2 is: 5.9.2000
```

Example 2

Following is an example of a *Matrix* class, where until now, we have not overloaded insertion (<<) and extraction operators (>>).

```
/* Matrix class, which is without overloading stream operators */
#include <iostream.h>
#include <stdlib.h>

class Matrix
{
     private :

          int numRows, numCols ;
          float elements [30] [30] ;

     public :
          Matrix( int rows , int cols ) ;
```


© Copyright Virtual University of Pakistan

```
        void getMatrix ( ) ;
        void displayMatrix ( ) ;

};

Matrix :: Matrix ( int rows = 0 , int cols = 0)
{
   numCols = cols ;
        numRows = rows ;

   for ( int i = 0 ; i < numRows ; i ++ )
   {
          for ( int j = 0 ; j < numCols ; j ++ )
        {
                elements [ i ] [ j ] = 0 ;
        }
   }
}

void Matrix :: getMatrix ( )
{
    for ( int i = 0 ; i < numRows ; i ++ )
    {
       for ( int j = 0 ; j < numCols ; j ++ )
       {
          cin >> elements [ i ] [ j ] ;
       }
    }
}


void Matrix :: displayMatrix ( )
{
    for ( int i = 0 ; i < numRows ; i ++ )
    {
       cout << "|  " ;
       for ( int j = 0 ; j < numCols ; j ++ )
       {
          cout << elements [ i ] [ j ] << "  " ;
       }
       cout << "|" << endl ;
    }
}

void main ( )
{

    Matrix matrix (2, 2) ;

    matrix.getMatrix ( ) ;
```


© Copyright Virtual University of Pakistan

```
    matrix.displayMatrix ( ) ;

    system ( "PAUSE" ) ;

}
```

The operator functions (<<, >>) are not overloaded for this program. A specific function *getMatrix()* has been called to get the values for the *matrix* object this entirely a different way than we used to do for primitive data types. For example, we used to get *int* i as; cin >> i. Similarly, we called a method *displayMatrix()* to display the values in the *matrix* object. We can see here, if we overload insertion (<<) and extraction (>>) operators then the user of our class, does not need to know the specific names of the functions to input and display our objects.

The changed program after overloading insertion, extraction operators and few additional statements to format the output properly:

```
/* Matrix class, with overloaded stream insertion and extraction operators. */
#include <iostream.h>
#include <stdlib.h>

class  Matrix
{
        float elements[30][30];
        int numRows, numCols;

public:
        Matrix ( int rows = 0 , int cols = 0 )
        {
                numRows = rows;
                numCols  =  cols;
        }

        friend ostream & operator << ( ostream & , Matrix & );
        friend istream &   operator    >> ( istream & , Matrix & );
};

istream &  operator  >> ( istream &  input , Matrix &   m )
{
   for ( int i = 0;  i < m.numRows;  i ++ )
   {
      for ( int j = 0;  j < m.numCols;  j ++ )
                {
                        input  >>  m.elements [ i ] [ j ] ;
                }
        }
        return  input;
}
```


© Copyright Virtual University of Pakistan

```
ostream &  operator  << ( ostream &   output , Matrix &   m )
{
        for ( int r = 0;  r  <  m.numRows;  r++ )
        {
                for ( int c = 0;  c < m.numCols;  c++ )
                {
                        output  <<  m.elements [ r ] [ c ]   <<   '\t' ;
                }
                output   <<    endl;
        }
        return  output ;
}

int  main ( )
{

        Matrix  matrix ( 3 ,3 );
        cout << "\nEnter a 3 * 3 matrix \n\n";
        cin  >>  matrix ;
        cout    << "\nEntered matrix is: \n";
        cout  <<  matrix;

        system ( "PAUSE" );
       return 0;
}
```

The output of the program is:

```
Enter a 3 * 3 matrix

45
65
34
23
72
135
90
78
45

Entered matrix is:
45      65      34
23      72      135
90      78      45
Press any key to continue . . .
```

You can see both the operators are declared *friend*s of the *Matrix* class so that they can directly access the *private* members of the *Matrix*.

The insertion operator (<<) is accepting both the parameters left and right by reference. We already know that for insertion operator (<<), it is not really required

© Copyright Virtual University of Pakistan

to pass the second parameter (the *Matrix* object in this case) by reference but we have used here to gain efficiency. The function is returning an object *ostream &*, i.e., it is returning a reference to a *ostream* object, that actually is the required in order to support cascaded operations using this operator.

The extraction operator (>>) is also accepting both the parameters by reference. But for this operator, it is mandatory to accept the *Matrix* object by reference because this function is modifying that object. Similar to the insertion operation, this function is also returning a reference to *istream* object in order to support cascaded operations.

Clearly after overloading the operators << and >>, it is more convenient for the programmer to use these already familiar operators to display and input the object data members. Readability of the program has also comparatively increased.

Tips

- ***Stream insertion (<<) and extraction operators (>>) are always implemented as non-member functions.***
- **operator << *returns a value of type* ostream & *and* operator >> *returns a value of type* istream & *to support cascaded operations.***
- ***The first parameter to* operator << *is an* ostream & *object.* cout *is an example of an* ostream *object. Similarly first parameter to* operator >> *is an* istream & *object.* cin *is an example of an* istream *object. These first parameters are always passed by reference. The compiler won't allow you to do otherwise.***
- ***For operator >>, the second parameter must also be passed by reference.***
- The second parameter to *operator* << is an object of the class that we are overloading the operator for. Similar is the case for *operator* >>.

© Copyright Virtual University of Pakistan

Lecture No. 38

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 11, 7, 3

11.6.4, page 181, 7.7, 3.10

Summary

34) User Defined Manipulator
35) Examples of user defined manipulator
36) Static keyword
37) Static Objects
38) Static data members of a class

Today, we will discuss the concepts like 'user-defined manipulators' and 'static keywords'. Despite being considered minor subjects, these become very important while carrying out complex programming. Let's start with 'User-defined manipulators'.

User Defined Manipulators

We have talked a lot about the manipulators that are provided with the streams in the C++. These are similar to 'setw' function, used to set the width of the output. These are employed as inline like *cout << endl << i;* Remember that these functions work for only the immediately next output. How can we write our own manipulator? To determine it, it is better to understand what parameter-less manipulators are? These are the manipulators without any parameter like *endl*. This is a parameter-less built-in manipulator that inserts the new line besides flushing the buffer. If we want to write our own manipulator, how can we do this? In case of operator overloading, it is pre-requisite to know that where the operator will be used, what will be on its left-hand and right-hand sides. On reviewing the manipulators, you will find a stream object, normally on the left-hand side. Here, we are talking about *ostream*, an output stream. So that object will be *cout*. The *cout* will take this manipulator to carry out some manipulation. These are written in cascading style as *cout << manipulator << "some data" << end*l. With this cascading style, you can get a hint about the operation of this manipulator and its requirements. The point is, the left-hand side is going to be *ostream* object that will call the manipulator. What will be passed to the manipulator and what will be the return type.

© Copyright Virtual University of Pakistan

Normally on the right-hand side of the manipulator, we have another stream insertion operator i.e. <<. Here we are considering a parameter-less manipulator, that is no argument or number will be passed to it. It may be something like inserting a tab between two numbers for formatting or a manipulator to end the line or to make a sound of bell and so on. The left hand side is *ostream* object. There are no other parameters. The right-hand side is normally a stream insertion operator. We use it as *cout << manipulator* which is itself an action. We overload the stream insertion operator in such a way that the cascading works. So we return an *ostream* object. More accurately, a reference to *ostream* objects is returned. Manipulator is also going to be used in the same way, so that it returns a reference to an object of type *ostream*. Therefore we want to return the *cout* object or whatever stream we are using. Secondly it also needs the object that is calling it. Here we are not talking about our own class. *ostream* class is built-in and not under our control. So it can not be modified. We can only extend it by defining external things. So it is not a member function or member operator, but only a standalone operator. Normally the declaration of this manipulator is as:

```
ostream& manipulator_name (ostream& os)
```

This is also not a friend function. We cannot define friends for the classes that are already written and not in our control. The argument *os* here is the same object which is calling this function. We have to explicitly declare it. After this, we have to define this. Definition is just as another function. You can always write whatever you want inside the function. But we have to look at the spirit of the manipulator. When we are talking about the spirit of the manipulator, it means that the manipulator should only do something regarding output and return. It is normally very simple. Its return type is *ostream* object. In case of tab character, we can write as return *os << '\t';* It can be bell or something else. We can write useful manipulators to leave single or double blank lines or formatting the strings etc. Remember that it has to return a reference of object of type *ostream*. It automatically gets that object as parameter passed in to the function.

Examples of user defined manipulator

Here is the sample program using the manipulators.

```
/*  A small program which uses the user defined manipulators.
*/

#include <iostream.h>
#include <stdlib.h>

// Gives System Beep
ostream & bell ( ostream & output ) // Manipulator
{
   return output << '\a' ;
}
```


© Copyright Virtual University of Pakistan

```
// Gives Tab
ostream & tab ( ostream & output )     // Manipulator
{
   return output << '\t' ;
}

// Takes the cursor to next line
ostream & endLine ( ostream & output )  // Manipulator
{
   return output << '\n' << flush ;
}

void main ( )
{
    cout << "Virtual " << tab << "University" << bell << endLine ; // Use of Mainpulator
    system ( "PAUSE" ) ;
}
```

Lets see another example of matrix using the user defined manipulators for displaying the matrix on the screen.

Here is the code:

```
/*
A small program showing the use of user defined manipulators.
The display function of matrix is using these manipulators to
format the display.
*/

#include <iostream.h>
#include <stdlib.h>
#include <iomanip.h>

// definition of class matrix
class Matrix
{
     private:
        int numRows;
        int numCols;
        float elements[3][3];

     public:
       // constructor
       Matrix(int rows = 0, int cols = 0)
       {
          numRows = rows ;
          numCols = cols;
       }

       // overloading the extraction and insertion operators
       friend ostream & operator << ( ostream & , Matrix & );
```


© Copyright Virtual University of Pakistan

```
        friend istream & operator  >> ( istream & , Matrix & );
        // defining the user defiined manipulators
        friend ostream & spaceFirst ( ostream & );
        friend ostream & spaceBetween ( ostream & );
        friend ostream & line ( ostream & );
        friend ostream & newLine ( ostream & );
        friend ostream & star ( ostream & );
        friend ostream & sound ( ostream & );
};
//defining the operator >>
istream & operator >> ( istream & input , Matrix & m )
{
   for ( int i = 0 ; i < m.numRows ; i ++ )
   {
      for ( int j = 0 ; j < m.numCols ; j ++ )
      {
       input >> m.elements [ i ] [ j ] ;
      }
   }
   return input;
}

//defining the operator <<
ostream & operator << ( ostream & output , Matrix & m )
{
   for ( int i = 0 ; i < 60 ; i ++ )
   {
      if ( i == 30 )
      {
          output << "Displaying The Matrix" ;
      }
      else
      {
          output << star ;
      }
   }
   output << newLine;
   for ( int r = 0 ; r < m.numRows ; r++ )
   {
      output << spaceFirst << line;
      for ( int c = 0 ; c < m.numCols ; c++ )
      {
         output << spaceBetween << m.elements [ r ] [ c ] << sound << spaceBetween ;
      }
      output << spaceBetween << line;
      output << newLine;
   }
   output << newLine;
   return output;
}
```


© Copyright Virtual University of Pakistan

```
//defining the user defined manipulator, inserting the space
ostream & spaceFirst ( ostream & output )
{
   output << setw(33);
   return output;
}

//defining the user defined manipulator, inserting the space
ostream & spaceBetween ( ostream & output )
{
   output << setw ( 4 );
   return output;
}
//defining the user defined manipulator, inserting the | sign
ostream & line ( ostream & output )
{
   output << "|" ;
   return output ;
}
//defining the user defined manipulator, inserting the new line
ostream & newLine ( ostream & output )
{
   output << endl;
   return output;
}
//defining the user defined manipulator, inserting the *
ostream & star ( ostream & output )
{
   output << "*" ;
   return output ;
}
//defining the user defined manipulator, making sound
ostream & sound ( ostream & output )
{
   output << "\a" ;
   return output ;
}

// the main function
int main ( )
{
     // declaring a matrix of 3*3, taking its input and displaying on the screen
     Matrix matrix( 3, 3);
     cin >> matrix;
     cout << matrix;
     system("PAUSE");
     return 0;
}
```

The output of the program:

© Copyright Virtual University of Pakistan

```
3
5
1
8
7
6
2
5
2
*******************************Displaying  The Matrix*******************************

                    |  3   5   1  |
                    |  8   7   6  |
                    |  2   5   2  |

Press any key to continue . . .
```

Static keyword

We have been using static keyword in our examples. What is the meaning of static? The word refers to something that is stationary, stopped and not moveable. What are the types of these variables, declared as static? How can we make use of them? Static as the word implies are variables which exist for a certain amount of time, much longer than that by ordinary automatic variables. Let's consider the example about the lifetime of data variables. One of the variable types is global variable. Global variables are those that are defined outside of main. They are written as standalone statements before main function as *int i;* the variable *i* is a global variable. It is not only accessible in main but also in all the functions. They can assign some value to *i* or obtain the value of *i*. Global variables come into existence whenever we execute the program and the memory is allocated for *i*. It exists all the time when the program is running. At the end of the program execution, the memory will be de-allocated and returned to the operating system. So it has a very long lifetime. We need a value which exists for the complete execution of the program and is available in all the functions. We use global variables. It is not a good idea to do that unless it is absolutely necessary. The major plus point of these variables is that these are accessible from everywhere in the program. The inconvenience is that theses variables are visible in those functions too which does not need them.

Suppose, we have a global variable *i* declared as *int i;* and in some function we are writing a *for* loop as *for(i = 0; i < n; i++);* Now which *i* is being used here. This is the variable *i*, declared as global. This global *i* may have some valuable value, like the number of cars or the number of students etc. Here, in the function when we run the loop, the value of *i* will be changed. The global variables have this bad habit of being around, even when we don't need them. What will happen if we declare another *i* variable inside the function? A local variable will be created inside the function at run time and the global *i* is not going to be accessible. But this can be very subtle and hard to track programming errors. These are not syntax errors but logical ones. So beware of using too many global variables. Now have a look on the other side of the picture. While writing functions, we pass values to them. So instead of passing the value of *i*

© Copyright Virtual University of Pakistan

again and again, we declare it as global. Now it is available in the function, leaving no need of passing it.

Let's now come to the next variety of variables. The variables, defined in the main function are local to the function main. It means that they are accessible in all parts of the main function. Their values can be assigned, used in computations and later displayed. When we enter some function other than main, these variables are not accessible there. They are hidden. The global and the local variables, declared in a function are visible. The arguments passed to a function, are also visible. We pass the parameters through stack. Parameters are written on the stack. Later, the function is called which reads from the stack and makes a temporary copy for its use. The variables, declared and used inside the function are called automatic variables. They automatically come into being when the function is called. When the function finishes, these variables are destroyed. So automatic variables are created constantly and destroyed all the time. Here, we are talking about variables ordinary as well as user defined objects. Their behavior is same. They are automatic when the function is called, memory is allocated normally on the stack at the same time and used. When the function exits, these variables are destroyed. What happens if we want that when the function exits, some value, computed inside the function, is remembered by the function and not destroyed. This should not be visible by the other parts of the program.

Let's consider the example of a refrigerator. When we open the door of a refrigerator, the light turns on and we can see the things inside it. However, on closing the door, the light turns off. Do we know that light is off because whenever we open the door the light is on. When we close the door what is inside. We do not know. May be things magically disappear. When we open the door, magically, the things are at their position. You can think of this like a function. When we enter in the function, these automatic variables are available there and visible. When we came out of the function, it is like closing the door of the refrigerator and the light is turned off. We cannot see anything. Function goes one step ahead of this and it actually destroys all the variables. Whereas, in the refrigerator, we know that things are there. Somehow we want the function to behave like that. Outside the refrigerator, these things are not available. We can not access them. Let's say there is a bottle of water inside the refrigerator. You open the door and place it some other place. Next time, when you will open the door, the bottle is seen at the same position where you have moved it. It would not have moved to some other position. If you think of automatic variables, suppose we say inside the body of the function *int i = 0;* Every time the function is called and you go into the function where *i* is created. It has always the value 0 to start with and later on we can change this value.

What we want is that whenever we go back into the function, once we call the function like we open the door of the refrigerator and move the bottle to some other place and close the door. So we made one function call. Next time, when we call the function, the bottle is found in its new place. In other words, if we have defined an integer variable, its value will be set at 10 in the function when we return from the function. Next time when we call the function, the value of that integer variable should be 10 instead of 0. We want somehow to maintain the state of a variable. We want to maintain its previous history.

© Copyright Virtual University of Pakistan

If we declare a global variable, the state would have been maintained. The global variable exists all the time. Whatever value is set to it, it is there and accessible from any function. The drawback is that variable exists even when we don't want it. Static keyword allows us a mechanism from getting away of the downside of the global variables and yet maintaining a state inside a function. When we visit, it is found out what are its values before that we go ahead with this value. For this, whenever we declare a variable inside the function, *static* keyword is employed before the variable declaration. So we write as:

```
static int i;
```

That declares *i* to be a static integer inside the function. Think about it. Should we declare static variables inside the main function? What will happen? 'main' itself is a function so it is not illegal. There is no objective of doing this in main because main is a function from where our programs start and this function executes only for once. So its state is like an ordinary variable, declared inside main. It is only relevant for the called functions. We write inside the function as *static int i;* while initializing it once. It will be created only once irrespective of the number of function calls. Now once it is created, we increment or decrement its value. The function should remember this value. The programmer may go out of the function and come back into it. We should get the value that should be same as that at the time of leaving the function. It is necessary for the static variables that when these are created, they should be initialized. This initialization will be only for once for the complete life cycle of the program. They will be initialized only once.

Here, we have to take care of the subtle difference. In case of ordinary variable declaration, we should initialize them before using. If you have to initialize an *int* with zero, it can be written as *int i;* and on the next line *i = 0;* But in case of static variables, we have to use a different type of initialization. We have to use it as *static int i = 0;* It means that creation of *i* and the allocation of memory for it takes place simultaneously. It is initialized and the value 0 is written. This is initialization process. If somewhere in the function, we have statement *i = 10;* it will not be treated as initialization. Rather, it is an assignment statement. Here we want that as soon as the variable is created, it should be initialized. This initialization will be only for once for the lifetime of the program and it takes place when first time we enter in to the function. However we can manipulate this variable as many times as we want. We can increment or decrement it. However, it will remember its last value. How does this magic work? So far, we have been talking about the stack and free store. There is another part of memory, reserved for the variables like static variables. On the stack, automatic variables are being created and destroyed all the time. The heap or free store has the unused memory and whenever we need memory, we can take it from there and after use return it. This is the third part which is static memory area where static variables are created and then they exist for the rest of the program. These variables are destroyed on the completion of the program. So they are different from automatic variables which are normally created on stack. They are different from dynamic variables that are obtained from free store.

To prove this whole point let's write a simple program to fully understand the concept and to see how this works. Write a small function while stating that *static int i = 0;* Here, we are declaring *i* as a static integer and initializing it with zero. Then write

© Copyright Virtual University of Pakistan

i++; print the value of *i* using *cout*. Now this function just increments the value of *i*. This *i* is a static integer variable inside the function. Now write a main function. Write a loop inside the main and call this function in the loop. Let's say the loop executes for ten times. You will notice that whenever you go inside the function, the value of *i* is printed. The value of *i* should be printed as 1.2.3…10. If you remove the word static from the declaration of *i*, you will notice that every time 1 is printed. Why 1? As *i* is now automatic variable, it is initialized with zero and we increment it and its value becomes 1. *cout* will print its value as 1. When we return from the function *i* is destroyed. Next time when function is called, *i* will be created again, initialized by zero, incremented by 1 and *cout* will print 1. By adding the static keyword, creation and initialization will happen once in the life time of our program. So *i* is created once and is initialized once with the value of zero. Therefore *i++* will be incrementing the existing value. At first, it will become 1. In this case, function will return from the loop in the main program**,** call this function again**.** Now its value is 1, incremented by 1 and now the value of *i* becomes 2 and printed by *cout*. Go back to main, call it again and so on, you will see it is incrementing the last value. You can prove that static works.

Here is the code of the program:

```
/* This is a simple program. This shows the use of static variables inside a function.
*/

#include <iostream.h>

void staticVarFun();
void nonstaticVarFun();

void main(void)
{
   cout << "\nCalling the function which is using static variable \n";
   for(int i = 0; i < 10; i++)
      staticVarFun();

   cout << " \nCalling the function which is using automatic variable \n";
   for(int i = 0; i < 10; i++)
      nonstaticVarFun();
}
// function definiition using static variables
void staticVarFun()
{
   static int i = 0;
   i++;
   cout << "The value of i is:" << i << endl;
}

// function definiition using automatic variables
void nonstaticVarFun()
{
   int i = 0;
```


© Copyright Virtual University of Pakistan

```
    i++;
    cout << "The value of i is:" << i << endl;
}
```

The output of the program:

```
Calling the function which is using static variables
The value of i is:1
The value of i is:2
The value of i is:3
The value of i is:4
The value of i is:5
The value of i is:6
The value of i is:7
The value of i is:8
The value of i is:9
The value of i is:10

Calling the function which is using automatic variables
The value of i is:1
The value of i is:1
The value of i is:1
The value of i is:1
The value of i is:1
The value of i is:1
The value of i is:1
The value of i is:1
The value of i is:1
The value of i is:1
```

Static Objects

Let us look at some more uses of this keyword. As mentioned earlier that the user defined data types are the classes and objects that we created. These are now variables as for as we are concerned. If these are variables, then we can declare them as static. Now we have to be careful when we think about it. When we declared static int, we said that it should be initialized there. We initialized it with zero. What is the initialization of objects? We have defined a class and instantiated an object of that class. So we can say something like vehicle A or truck B where vehicle and truck are the classes which we have defined. 'A' and 'B' are their objects, being created in some function or main. When are these objects initialized? You know that the initialization is done in constructors. So normally C++ provides a default constructor. Here we have to write our own constructors as initialization can happen only once, if declared static. Again we are talking about these static objects inside a function instead of the main. These objects should maintain their values while getting out of the function.

© Copyright Virtual University of Pakistan

Whenever we create a static object, it must be initialized. Most of the time, we want that when the object of our class is created, its data members should be initialized by some value. For this purpose, we have to provide a constructor so that whenever an object is created, its data members are initialized. Only then it will work. Otherwise we will have problems. We may want to do as truck A, but our constructor takes some arguments. Now how this object will be created. How many wheels this truck will have? How many seats will be there? We have a solution to overcome this problem. Define a constructor which takes arguments and provides the default value to it simultaneously. If you provide a constructor with default values, then the object which is created will automatically get these values. If you write *truck A(4, 6)*, there may be some constructor which will initialize it with 4 wheels and 6 seats. But the point to remember is if you ever go to use a static object, it is necessary to provide a constructor with default arguments so that the object which you have created is initialized properly. Other than that the whole behavior of a static object is exactly the same as we have a static variable of an ordinary data type or native data type. Static variable means maintaining the state of a variable. It exists and lives around even when we are outside the function. It is an alternative to using a global which exists even when we don't want it. Now we try to learn about the destructors of static objects. If you create an object inside a function as *truck A,* when the function finishes, the object *A* will be destroyed. Destructor for this static object will be called.

To prove this write a class, inside the constructor. Also write a *cout* statement which should print 'inside the constructor of 'and the name of the object which will be passed as an argument. In the destructor write a cout statement as *cout <<" Inside the destructor of " << name*, where name will tell us that which object is this. Now experiment with it. Declare a global variable of this class before main as *truck A('A')*. When the constructor for this object is called the line 'Inside the constructor of A' will be displayed. Now within the main function, declare another object as ordinary variable i.e. *truck B('B')*. Its constructor will also be called. You will see it. Write a small function and create another object within that function as *truck C('C')*. Define another function and declare a static object in it as *truck D('D')*. Call these two functions from main. Now compile and execute this program, as we have written *cout* statements inside the constructor and destructor. Now you will be able to determine which object is being created and which one being destroyed. Here you will also notice that first of all global object *A* will be created. There is going to be a line 'Inside the constructor of object A'. After that, object *B* will be created, followed by the display of constructor *cout* line. From main, we are calling function *F* which is creating object *C*. So object *C* will be created then. What next? The function *F* will finish and the control go back to main. If the function *F* finishes, its local data will be destroyed. So the object *C* will be destroyed. Here, you see it on the screen 'Inside the destructor C'. After this the function *G* will be called and we will have 'Inside the constructor for D'. This object D is a static object. Now when the function *G* finishes, you will not see the destructor line for object *D*. After this, the main function finishes and the destructors will be called for objects *A* (which is global), object *B* (which is inside the main) and object *D* (which is created as static inside the function G). In which order these will be called?. If you look at this very simple program, you will find that the last object to be created was the static object inside the function *G*. Should that deleted first i.e. the destructor of object *D* should be called? Well actually not true, the local variables of main function will be first destroyed. Static objects remain for longer period of time. Later, the static object *D* will be destroyed and the

© Copyright Virtual University of Pakistan

thing finally destroyed is the global object, which was created first of all. You will find that the destructor for object *A* is called. With this exercise, you will know the sequence in which things are created and destroyed. Another thing that you will notice is that when the function *G* finishes the static object is not destroyed.

The code of the program;

```
// An example of static objects, notice the sequence of their creation and destruction
#include <iostream>

// defining a sample class
class truck {
  private:
    char name; // Identifier
  public:
    // constructor displaying the output with the object name
    truck(char cc):name(cc) {
        cout << "inside the constructor of " << name << endl;
    }

    // distructor displaying the output with the object name
    ~truck() {
        cout << "Inside the destructor of " << name << endl;
    }
};
// defining a global object
truck A('A');

// a simple function creating an object
void f() {
  truck C('C');
}
// a simple function creating a static object
void g() {
  static truck D('D');
}

// main function
int main() {
  // an ordinary object
  truck B('B');
  // calling the functions
  f();
  g();
}
```

The output of the program:

```
inside the constructor of A
inside the constructor of B
inside the constructor of C
Inside the destructor of C
```


© Copyright Virtual University of Pakistan

```
inside the constructor of D
Inside the destructor of B
Inside the destructor of D
Inside the destructor of A
```

Lets recap these concepts. When you declare a static variable (native data type or object) inside a function, it is created and initialized only once during the lifetime of the program and therefore it will be destroyed or taken out of memory only once during the lifetime of the program. So it is a good way of maintaining state. It is an alternative to using a global data type which has some side effects. In the main, we can write static variables but it is a meaningless exercise because these are exactly like ordinary variables inside main.

Static data member of a class

Lets talk about the keyword static inside the class. Static variables are used to maintain state. We are talking about the state in which we left the function. While extending the concept, we will go inside an object. Here, we should find certain things left exactly the way they were initially. So now we are talking of static data members inside a class. What does it mean?

Literally speaking, the word 'Static' means the stationary condition of things. Stationary for object or class? Here it will be stationary for the class. That means that static data will be created once and initialized once for that class. Therefore it is not related to the objects of that class. There is only one copy of the static data member inside a class. The copy is not repeated for the objects. Whenever we create an object of a class, the complete data structure is copied for that object and there is one copy of functions which the objects may use. Static members are single for the whole class in the static memory area. It will not be repeated whenever we create an object of the class.

Now the question arises when it will be created? When it will be initialized? And when it will be destroyed? Now these are on class level and not on object level. To understand this, we have to talk about the lifetime of the static data member. The lifetime of the static data member of a class is the lifetime of the program. In other words, when you include a class in the program as a class definition, the memory is allocated for its static data members. We have some techniques to initialize it. We initialize it only once. Initialization is done at file scope which means almost at the global scope. We initialize it outside of the main. The memory is allocated for these static members. No other copy can be created for them. Therefore we can create and initialize them outside of main. There is no object so far. How can we initialize its static data members?

Suppose we have a class truck as:

```
class truck{
  public:
        int wheels;
        int seats;
```


© Copyright Virtual University of Pakistan

```
}
```

Now we refer the data members with the object as:

```
truck A;
A.wheels = 6;
A.seats = 4;
```

That's a way to refer to a data member. Here we are saying that we have some static data member of class and the object *A* has not been created yet. But we have the memory for the static members. Now we want to initialize that memory. How can we do that? We do this by using the scope resolution operator (::) and on its left hand side, we have class name and not the object name. On the right side, we write the name of the static data member. Suppose we have some static integer data member *i* in the class truck, so we can write it as:

```
truck::i = 10;
```

This initialization is taking place at file scope outside of the main .As it is happening only once in the program, it will not be executed again. It is being initialized once for the class. You can create as many object as you want. Objects can read and change that value.

Static data members of a class can be public or private. The objects of the class have access to them. They can manipulate it. But it is created and initialized only once. There is a single copy of these static data members regardless of how many objects of the class you create.

Let's take a look at the problems having static data members of a class. Suppose we have a class as 'savingsAccount'. We deposit money in that account. Some profit is also earmarked for it. Over a period of time, bank declares the rate of the profit. Profit rate is same for all PLS accounts. We have defined a class *savingsAccount* which have the information like person name, account number, current balance etc. We also have to keep the profit rate so that we can apply that on the account. Do we have different profit rate of every account? No, the bank has declared say 3% profit rate. That will be applied to all the PLS accounts. So we want that the profit rate should be defined at one place. It should be the part of the class but not defined for each object. So it is a good place to use a static variable as a data member of the class. We can initialize it at file scope as:

```
savingsAccount::profit_rate = 3.0;
```

We will write this before main function. As soon as, we compile the program and try to run it, the space is created in the static storage area. The above statement will initialize that static memory with 3.0. No savings account has been created yet. We will be creating saving accounts (object of class savingsAccount) in the main or some other function as account1, account2, etc. This profit rate will be available to every account. We can access it as:

```
account1.profit_rate;
```


© Copyright Virtual University of Pakistan

and can use it in computations. This is legal but a bad usage. Why it is a bad usage? Suppose we write as;

```
account1.profit_rate = 4.0;
```

What will happen? Does the profit rate for only *account1* has been changed? No. There is only one copy of profit_rate for all the objects of this class. That means if an object manipulates the static data member, which it can through the member functions or directly depending on it is private or public. It is actually modifying the value of that static data member for all objects of this class. So don't assume that it will change profit_rate for one object. It is a legal but a bad usage. Always use it with the class name and not with the object. So you should access it as *savingsAccount::profit_rate*. It means you are resolving it at class scope. Be careful while applying it.

Let's consider another example. We have a class as *student* and a data member for how many students in the class. Now every time, a student enrolls in the course, we want that number of students should be incremented. Whenever a student withdraws, fails or passes out from the course, the number of students should be decremented. We want this to be inside the *student* class. How does it work? We define a static data member as *static int how_many;* and initialize it to zero as:

```
student::how_many = 0;
```

In the constructor of the class we write as:

```
how_many++;
```

This way, whenever a student object is created, *how_many* will be incremented. Whenever a student leaves the course, its destructor should be called. We will write in the destructor as:

```
how_many--;
```

So it's a good way of keeping track of how many objects of a particular type exist at this time inside the program. To display that we can write a member function that will display 'how_many'. The merit of this technique is that we have done all this work inside the class. We did not use two classes or global variable or go through the source code to count the number of students. Using these, you can make your program more and more dynamic. The usage of static is very import. So you should understand it clearly. Static is maintaining the state. The state may be how many students are in the class.

Today we have covered the parameter-less manipulators which will return the *ostream* object and *ostream* object is passed as an argument to them. Then we discussed about the static data, both at the ordinary level and then the static data members inside the class. These are very useful. As you write bigger and more complex programs, you will find that these concepts are very useful. Again from a generic prospective, you will be working hopefully in your professional career with many different languages. You have to understand that every language might

© Copyright Virtual University of Pakistan

represent the static concept in a different way. But just knowing that concept empowers you and help you to understand more complex programming languages as well.

Lecture No. 39

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 5, 6, 7

Revise old topics

Summary

39) Pointers
40) References
41) Call by Value
42) Call by Reference
43) Dynamic Memory Allocation
44) Assignment and Initialization
45) Copy constructor
46) Example
47) Rules for Using Dynamic Memory Allocation
48) Usage of Copy Constructor
49) Summary
50) Exercise

In this lecture, we will review the concepts like pointers, references and memory allocation, discussed in the previous lectures. The review of these topics will help enhance our understanding of the programming. Let's discuss these topics one by one.

Pointers

Pointer is a special type of variable that contains a memory address. It is not a variable that contains a value, rather an address of the memory that is contained inside a pointer variable.

In C++ language, variables can be without type. Either we can have a void pointer or these can be typed. So we can have a pointer to an integer, a pointer to a character and a pointer to a float etc. Now we have a user-defined data type, which we call classes, so we can have pointers to classes.

© Copyright Virtual University of Pakistan

While talking about pointers, we actually refer to pointing to an area in memory. A pointer to an integer, points to a location opted by an integer in memory. If there is an array of integers in the memory, we can still use a pointer to point to the beginning of the array. This is a simple manipulation. To have the address of a memory location, we use & sign. The '&' sign is also used in references. So we have to be very cautious while making its use. To further overcome this ambiguity, we will now recapture the concept of reference.

References

A reference can be considered as a special type of pointer as it also contains memory address. There are some differences between pointers and references. Pointers may point to nothing while references always have to point to something. A reference is like an alias for an object or a variable. The references should be used when we are implementing the call by reference. This helps us make our syntax easier as we can implement the call by reference with out using the * operator.

Call by Value

Whenever we call a function and pass an argument, an object or variable to the function, then by the default rule of C and C++, it is a call by value. It means that the original data remains at its place and a temporary copy of it is made and passed to the function. Whatever the function does with this copy, the original value, in the calling function, remains intact. This is a call by value.

Call by Reference

If we want a function to change something in the original object variable or whatever, that variable or object by reference would be passed. To do this, we don't make temporary copy of that object or variable. Rather, the address of the variable is sent. When the function manipulates it, the original object will be manipulated, effecting change in its values. The use of call by reference is also important for the sake of efficiency. If we have a large object, sending of its copy will be something insufficient. It will occupy a large space on the stack. Here, we can use call by reference instead of call by value only for efficiency while we need not to change the original object. For this, we use a keyword *const* that means that a *const* (constant) reference is being passed. The function can use its values but cannot change it.
Now we come to the dynamic memory allocation.

Dynamic Memory Allocation

In C language, we have a method to allocate dynamic memory. In it, while executing the program, we allocate some memory from the free store (heap) according to our need, use it and after using, send it back to the free store. This, dynamic memory allocation, is a very common function in programming. While writing C++ language, it was decided that it should not be implemented by a function call. There should be native operators, supposed to be very efficient. These operators are *new* and *delete.* We allocate memory with the *new* operator from the free store. It returns a pointer. For example, if we say *p* is a pointer to an integer, the statement will be written as

```
int *p ;
```

Now we write

```
p = new int ;
```


© Copyright Virtual University of Pakistan

This statement performs the task in the way that it gets memory for an integer from the free store. There is no variable name for that memory location but the address of this location is stored in the pointer *p*. Now by using the syntax **p* (which means whatever p points to), we can manipulate that integer value. We can also write to and read from that location. This memory allocation is done during the execution of the program. Whenever we allocate memory with the *new* operator, it is our responsibility to de-allocate this memory after the termination of the program. To do this de-allocation, we have an operator *delete*. To de-allocate the memory, allocated with *p = new int ;* we will write

```
delete (p) ;
```

It will not delete the *p* rather, it will send the memory gotten and pointed by *p* back to the free store.

Same thing happens when we try to allocate more than a simple variable i.e. when we are trying to allocate arrays. When we use *new* operator to allocate a space for an array, we tell the size of the array in square brackets (i.e. []). We can write it like

```
p = new int [10] ;
```

This statement says *p* is pointing to an area of memory having the capability to store 10 integers. So there is an array of 10 integers whereas *p* is pointing to the beginning point of the array. Now we can access the elements of the array by manipulating the pointer *p*. To make the memory allocated for an array free, the syntax is a little bit different. Whenever, we allocate an array then to free it we write

```
delete [] p ;
```

This will free the array that is pointed by *p*. In this case, the space of 10 integers that was pointed by *p,* will be de-allocated despite the fact that we write empty brackets with the *delete* operator. This is due to the fact that C++ internally has the record that how much memory was allocated while allocating an array. After *delete,* the pointer points to nothing. So it's a free pointer and can be reused.

Now let's go on and look at a special type of objects. These are the objects with the data members as the pointers. In the previous lectures, we had discussed the example of a class Matrix. In that class, we said that it will be a two dimensional matrix while talking about its size etc. In this example, it will be a generic class besides being a matrix of 3 x 3. Now we want to see the Matrix class defined such a way that the programmer can take an object of it of any size at any time, say a matrix of 3 x 3, 5 x 5 or 10 x 10. It means that the size of the matrix should be variable. Now we have to bring every thing together in terms of how we declare and manipulate arrays inside a C++ program.

Whenever we declare an array, we have to mention its size. It is necessary, as otherwise no memory will be allocated, it's about the static memory allocation. In it, we declare like that it will be a two dimensional array of *m* rows and *n* columns. The compiler will allocate a memory for this array at the start of the program. Now we are talking about that at the compilation time, we don't know the size of the array. We want to allocate an array of the required size at run time. When the constructor of the class is called, the memory required by that object should be allocated at that time.

© Copyright Virtual University of Pakistan

For example, in case of Matrix class, we will like to tell the number of rows and columns of the object of the Matrix in the constructor, so that the constructor could allocate the memory for that object. A keyword *new* is used for this purpose. Here, we realize that in case of data member of this class Matrix, fixed rows and columns cannot be used in the class definition. We cannot define a two-dimensional array inside the class as it negates the concept of extensibility. Here, inside the class we define something like *int *m;* which means *m* is a pointer to an integer. It is the data member of the class. In the constructor of the class, we say that it will take two integer arguments, rows and columns. Whenever we declare an object, which is an instantiation of the class, the constructor of the class is called. Now in the constructor of this class we want to allocate memory from the free store equal to the memory required by *m* x *n* (m multiply by n) integers. So if we say a Matrix of 3 rows and 3 columns then we require memory for 9 (3 * 3) integers. If we say four rows and five columns then we require a memory for 20 (4 * 5) integers. Now it is very simple, inside the constructor we will write

```
m = new int [rows * columns] :
```

Thus we created a Matrix to which we tell the number of rows and columns, through variables, during the execution of the program. When we created an object of the class its constructor is called to which the variable values are passed and by multiplying these variables (number of rows and columns), we allocate a memory for these integers by the *new* operator and its address is assigned to *m*. Thus the object is initialized and the required memory is available to it. Now we can use it as a two dimensional array and can put values inside it. Now the class definition of the class Matrix can be written as under.

```
class Matrix
{
        private:
                int *m;
                int row, col;
        public:
        Matrix(int rows, int cols)
        {
                        m = new int[rows * cols];
        }
};
```

There is a requirement that if the constructor of a class allocates the memory, it is necessary to write a destructor of that class. We have to provide a destructor for that class, so that when that object ceases to exist, the memory allocated by the constructor, is returned to the free store. It is critically important. Otherwise, when the object is destroyed, there will be an unreferenced block of memory. It cannot be used by our program or by any other program. It's a memory leak that should be avoided. So whenever we have a class in which the constructor allocates dynamic memory, it is necessary to provide a destructor that frees the memory. Freeing the memory is an easy process. We have no need to remember that how many rows and columns were used to allocate the memory. We simply use the *delete* operator with empty brackets and the pointer that points to the allocated memory. We write this as follows

© Copyright Virtual University of Pakistan

```
delete [] m ;
```

This statement frees the allocated memory (whatever its size is) that is being pointed by *m*.

Assignment and Initialization

Let us discuss the assignment and initialization. We do initialization as

```
int i = 0 ;
```

This is declaring an integer and initializing it. The second way to do this is

```
int i ;
i = 0 ;
```

This has the same effect as the first statement but the behavior is different. At first, a space is allocated for *i* before assigning a value to it..
The same applies whenever we create an object. We can either create an object, initialize it at the creation time that means constructor is being called, or we can create an object and then assigns values to its data members later. This is usually done either by set functions or with the assignment statements. Here the thing to differentiate is that if we have two objects of a class, say Matrix, *m1* and *m2*. We have, in some way, created *m1*, its rows and columns have been allocated, and values have been put in these rows and columns. Now somewhere in the program, if we write

```
m2 = m1 ;
```

It is an assignment statement. If we have not defined the overloaded operator for assignment, the default assignment of C will be carried out. The default assignment is a member-to-member assignment. Now let's again look at the construct for the Matrix class. In it, we have only one data member i.e. a pointer to an integer. We have written *int *m ;* in the class definition. So in *m1*, there is a pointer to an integer but *m1* is a fully qualified developed object. It is, let's say, a 5 x 5 matrix and has values for its entities. Now when we write

```
m2 = m1 ;
```

m2 has also a pointer *m* to an integer as its own data member. If we have not written an overloaded assignment operator for this class, the value of *m* of the object *m1* will be assigned to *m* of the object *m2*. Now we have two objects, having pointer variables that hold the same address. So these are pointing to the same region in the memory. There arise many problems with this. Firstly, if we destroy *m1*, the destructor of *m1* is called and it frees the memory. So the memory being pointed by *m* of the object *m1* has gone back to the free store. Now what happens to *m2*? There is a pointer in *m2*, pointing to the same memory, which has been sent to the free store by the destructor of *m1*. The memory is no longer allocated. It has been sent to the free store. So we have a serious problem. We don't want two objects to point to the same region in the memory. Therefore, we write an assignment operator of our own. This assignment operator is such that whenever we do some object assignment, it allocates separate

© Copyright Virtual University of Pakistan

memory for one object and copies the values of the second object into that space. Thus the second object becomes an independent object. So we have to be careful. Whenever we have a class with a dynamic memory allocation, there is need for writing an assignment operator for it.

Copy Constructor

Now we will see what a copy constructor is? We have discussed the dangers that a programmer may face during the process of assigning the objects. The same danger comes to the scene, when we pass an object like the Matrix class to a function that does some manipulations with it. Suppose, we have a function that takes an object of a class Matrix as an argument. The default mechanism of calling a function in C or C++ is call by value. Now what is the value for this object, being passed to the function? The values of the data members of the object will be placed on the stack. The function will get a temporary object, as it is a call by value. The original object remains intact. The values of data members of that temporary object will be the same as the values in the original object. Now if it is a simple class, there will be no problem. However, if there is a class with a pointer as its data member and that pointer has allocated some memory, then the value of that pointer is copied. This value is passed to the function and not the memory. Now in the function, we have a temporary object that has a pointer as data member, pointing to the same memory location as the original object. If we are just reading or displaying that object, there is no problem with it. If we change the values of the object, the values in the temporary object get changed. However, when we manipulate the pointer, it changes the values in the memory of the original object. This change in the original values is the mechanism of the call by reference and here we have done call by value. We know that a temporary object is passed to the function. So how we get around this problem? The way to resolve this problem is to create a complete copy of the object. We want that in this copy of the object the pointer value must not point to the same memory location. Rather, it must point to the memory location of its own. So we want to have a copy constructor. That means a constructor that will create a new object with a full copy of the other object. This is also known as deep copy as opposed to shallow copy. The shallow copy makes a member-to-member copy of the object without taking care whether the pointer is a pointer or an ordinary variable. The copy of the ordinary variables is perfectly valid and legal. But if we make a member copy of a pointer, there may be the problem in which a new pointer variable is created with the same value, without creating a new area of memory. Therefore, we have to write something special that is called copy constructor.

The basic line in the syntax of the copy constructor is that we are trying to create a new object by passing an object of the same class as the argument. So we should think of a constructor. For example if we have the class Matrix, its prototype will be as under.

```
Matrix (Matrix &) ;
```

The '&' sign shows that a reference of the object of type Matrix will be passed. Now whenever we write a copy constructor, there is need to be very cautious. We have to allocate a new memory for that copy. When we go into this copy constructor, at first, it is determined that how much memory the original object has allocated? Since we pass the object, so all the queries might have answers. For example, in case of Matrix class, we can find the number of rows and columns. We create a temporary object

© Copyright Virtual University of Pakistan

inside the constructor and allocate it a memory. Then the values of memory of the original object are copied into this memory. Now the pointer of this new object will point to this new location. As the constructor returns nothing and just creates a new object, so there is no return statement in the constructor. Thus, this copy constructor is completed. The syntax of this constructor is given below

```
Matrix::Matrix ( const Matrix  &other )
{
        size = other.size ;                    // size is a function
to determine the
    memory allocated by object
        m = new int [size] ;
        copyvalues ( m, other ) ;
}
```

In this case, it creates a new object that actually creates memory. It does not make the shallow copy so there is no copy of the pointer value. In fact, the pointer has a new value. But the values pointed to by this pointer (i.e. the values in the new allocated memory) are the copy of the values in the memory of the original object, and then this fully constructed object is returned.

Now we have the facility of copy constructor. With it, we can define new objects based on the existing objects. This copy constructor is necessary for the objects in which the memory is allocated dynamically. We can use this copy constructor without causing any problems. Suppose we have written as

```
Matrix a (3,3) ;
```

We have defined a constructor that takes two arguments rows and columns, as a matrix 'initializer'. We allocate memory for these rows and columns and create an object. Then somewhere, later in the program, We write

```
Matrix b (a) ;
```

This statement makes a complete copy of already created object *a* and a new object *b* is created. This new object has its own pointer and own memory allocation. This memory location is different from the memory allocated inside the matrix *a*. Now if *a* dies then *b* is still a valid object. So a copy constructor is critically useful. It is used when we want to create a duplicate copy of an object. It is always used whenever we want to do a call by value into a function to which an object is being passed, and the object is of a class in which we have allocated the memory dynamically.

Example

Let's look at an example to further understand these concepts. We write a class *String*. It has a data member *c* that is a pointer to a character. We write a member function (copy function) of the class, which can copy values in the character array of the class. There is a constructor that will allocate a space for the character arrays i.e. string. The starting address of this array will be stored in data member of the class i.e. in a pointer. We have not written any copy constructor. Now we want to make two objects of this *String* class, say, *s1* and *s2*. We create object *s1* and assign a string to it. We write it as

© Copyright Virtual University of Pakistan

String s1 (“test1”) ;

Now after this we write

String s2 = s1 ;

Thus we create a new object *s2*. The values of *s2* are initialized with the values of *s1*. As we have written no copy constructor, C will provide the default copy constructor. Now if we display the string of *s2*, it will be the same as of *s1*. Now use the copy function to assign new values to the string inside the object *s1*. So we write

s1.copy(“A new string”) ;

Thus we write a new string in *s1*. Now again if we display the string of *s2* by writing

s2.print ;

We will see that it displays the same value, assigned to *s1* in the previous statement. The reason is that the default copy constructor has done member-to-member copy. It has copied the value of the character pointer to the pointer of *s2* and thus both pointers are pointing to the same memory location.
Now there is need of providing a copy constructor for a class like this. We also have to provide a destructor as we are doing memory allocation inside the constructor of the class.

Following is the code of the example in which we provide a copy constructor. We create an object based on an existing object. The copy constructor creates an object with full copy of the existing object with its values in a new memory location.

```
/*This program has a copy constructor and demonstrate the use of it.
We create a new object by passing it an existing object, this calls
the copy constructor and thus creates a complete copy of the passing
object, and has its values in new location of memory.
*/

#include <iostream.h>
#include <stdlib.h>

// class definition
class String
{
        char *c;
    public:
         // copy function
         void copy (char *s)
        {
             c = s ;
         }
         // getting the length of the string
```


© Copyright Virtual University of Pakistan

```
        int length ()const
        {
            return strlen(c);
         }
         //constructors
        String ();
        String (const char *s)
        {
              c = new char [ 30 ];
               strcpy (c, s);
         }
        // copy constructor
        String( const String &other );

        //display the string
        void print()
        {
            cout << c << endl ;
         }
         //destructor
         ~String()
         {
             delete []c ;
          }
};
// definition of copy constructor
String::String( const String &other )
{
        int length;
        length = other.length();
        c = new char[length + 1];
        strcpy( c, other.c );

}

main ()
{
        String s1("test1");
        cout << "The string of s1 is     "  ;
        s1.print();
        String s2(s1);
        cout << "The string of s2 is     " ;
        s2.print();
        s1.copy("A new string"); // assign new value to string s1
        cout << "The string of s1 is     " ;
        s1.print();
        cout << "The string of s2 is     " ;
        s2.print();             //s2 has its own previous value
}
```


© Copyright Virtual University of Pakistan

The following is the output of the program which shows the use of copy constructor.

```
The string of s1 is    test1
The string of s2 is    test1
The string of s1 is     A new string
The string of s2 is    test1
```

The other affected part is the assignment operator itself. We know that there are dangers in the assignment operators of a class in which memory is being allocated. We cannot write an assignment operator for such a class blindly. When we write an assignment operator for such a class, that operator must first look at the fact whether there is self-assignment being done.

Suppose we have an integer *i*. We have written as

```
int i ;
i = 10 ;
```

And down to this we write

```
i = i ;
```

There is no problem with it. It is a legal statement. It is complicated if we do such assignment through pointers. In such a case, pointer is pointing to itself and even it has no problem. But when we do this with objects that have allocated dynamic memory, the method of assignment is changed. Let's take the example of Matrix. We have an object of Matrix, say *m1*, which has three rows and three columns. Another object, say *m2*, has five rows and five columns. Somewhere in the program we write

```
m1 = m2 ;
```

Here *m2* is a big object while m1 is a small one. We want to assign the big object to the smaller one. The assignment operator for this type of class, first de-allocates the memory reserved by the left-hand side object. It frees this by the *delete* operator. Then it will determine the memory required by the object on right hand side. It will get that memory from the free store by using *new* operator. When it gets that memory, it will copy the values and thus the statement *m1 = m2*; becomes effective. So assignment has a requirement.

Now if we say

```
m1 = m1 ;
```

We have defined assignment operator. This operator will delete the memory allocated by *m1* (i.e. object on L.H.S.). Now it wants to determine the memory allocated by the object on the right hand side, which in this case, is the same i.e. *m1*. Its memory has been deleted. So here we get a problem. To avoid such problem, whenever we write an assignment operator, for objects of the class that has done memory allocation. After this, we do other things.

We have discussed the example in which we create an object of Matrix. We create it using a copy constructor by giving it another object. The syntax of it we have written as

```
Matrix m2 (m1) ;
```

This is the syntax of creating an object based on an existing object. We can write it in the following fashion.

```
Matrix m2 = m1 ;
```

While this statement, we should be very clear that it is not an assignment only. It is also a construction. So whenever we are using initialization, the assignment operator

© Copyright Virtual University of Pakistan

seems as equal to operator. But actually assignment operator is not called. Think about it logically that why assignment operator is not called? The assignment operator is called for the existing objects. There should be some object on the left-hand side, which will call the assignment operator. When we have written the declaration line

```
Matrix m2 = m1 ;
```

The m2 object has not constructed yet. This object of Matrix does not exist at the time of writing this statement. So it cannot be calling the assignment function or assignment operator. This is an example of the use of a copy constructor. Thus, there are two different ways to write it. Remember that whenever we create an object and initialize it in the declaration line, it calls the copy constructor.

Let's talk about another danger faced by the programmers when they do not provide copy constructor. The ordinary constructor is there which allocates memory for the objects of this class. Suppose we do a call by value to a function. Here, we know that a temporary copy of the object is made and provided to the function. The function does manipulations with this copy. When the function returns that temporary copy is destroyed. As no copy constructor is there, a shallow copy, with values of pointers, is made. The destructor should be there as we do memory allocation in the class. Now suppose that there is a destructor for that class. Now when this temporary object destroys its destructor executes and de-allocates the memory. Now as it was a shallow copy so its pointers were pointing to the same memory as of the original object. In this way, actually, the memory of the original object is de-allocated. So the pointer of the original object now points to nothing. Thus, in the process of function call, we destroyed the original object as it is an invalid object now. Its pointer is pointing to an unknown memory location. This is a subtle but very critical. This can be avoided by providing a copy constructor, which actually constructs a fully formed object with its own memory. That temporary object will go to the function . When it is destroyed, its destructor will de-allocate this memory. However, the original object will remain the same.

Rules for Using Dynamic Memory Allocation

Whenever we have a class in which we do dynamic memory allocation, there are some rules that should be followed.

First, we must define a constructor for it. Otherwise, we will not be able to carry out dynamic memory allocation. This constructor should be such that it gets memory from the free store, initializes the object properly, sets the value of the pointer and returns a fully constructed object.

Secondly, we must write an assignment operator for that class. This assignment operator should first check the self-assignment and then make a deep copy.. So that a properly constructed object should be achieved..

Thirdly, as we are doing dynamic memory allocation in the constructor, it is necessary to provide a destructor. This destructor should free the allocated memory.
These three rules are must to follow.

© Copyright Virtual University of Pakistan

Usage of Copy Constructor

Let us see where the copy constructors are being used?
First, it is used explicitly at some places, where we write

```
Matrix m2 (m1) ;
```

This is an explicit call to a copy constructor. Here *m1* is being passed by reference. If we want that there should be no change in *m1*, then it is necessary to use the key word *const* with it to prevent any change in *m1*. The presence of this key word means that the constructor will do nothing with the object, being passed to it. The use of copy constructor in this explicit way is clear.

The second way to use the copy constructor is by writing the declaration line as

```
Matrix m2 = m1 ;
```

Here the use of copy constructor is not clear. It is not clear by the statement that copy constructor is being used. It seems an assignment operator is being used. Be careful that it is not an assignment operator. It is a copy constructor.

The third use of the copy constructor is calling a function and passing it an object by value. If we have provided a copy constructor, it will be called automatically and a complete temporary copy (with memory allocation) of the object is given to the function. If we do not provide copy constructor, the call by value functions will create problems. In the function, if we change any value of the object, it will change the value in the original object.
In function calling, when we do the call by value, the copy constructor is called. On the other hand, in call by reference, copy constructor is not called and the address of the original object is passed.

Summary

A pointer is a variable that holds memory address. The & operator is used to get the address of a variable or an object. The & sign is also used as a short hand for a reference. Whenever we have a & sign in the declaration, it implies a reference. Whenever we have & sign on right hand side of an assignment operator, it is taken as address of an object. We can do dynamic memory allocation by using pointers.
In C++ language, we have two very efficient operators provided which are *new* and *delete*. We use the new operator for obtaining memory from the free store. It returns a pointer to the allocated memory. We store the value of this pointer in a pointer variable.
In a class, which allocates memory dynamically, there is a data member i.e. a pointer. When we create an object of the class at run time, it will allocate memory according to our requirement. So there is no waste of memory and the situations in which we want to store large data in small memory or vice versa are prevented. So we do dynamic memory allocation inside these classes.
Whenever we have dynamic memory allocation inside a class, we have to provide few things. We must provide a constructor that does the memory allocation for us producing a well-formed object.

© Copyright Virtual University of Pakistan

We must provide a copy constructor that is able to create fully formed copies of the objects. That means is should not only make the copies of the values of the pointers but it should give the pointers new values by allocating new memory for the object. And should copy the values of the original object into this new memory location.
We must provide an assignment operator. This operator should be able to check the self-assignment and can assign one object to the other in a proper fashion using the concept of deep copy and not a shallow copy. So we allocate memory space then copy element by element in this allocated memory.
And finally we must do de-allocation which means whenever we destroy an object and it goes out of scope, we should free the memory allocated by that object. To do the memory de-allocation we must provide the destructor of the class in which we free (delete) the memory by using deletes operator.

Exercise

You should write small programs to examine the working of these rules. You can check this if we allocate memory and do not delete it in the destructor. Then the next time, when we execute the program it will allocate a new memory. We can find that which memory is assigned by displaying the value of the pointer (not the value it points too). It will be a number with 0x-notation i.e. it will be in hexadecimal. We don't care about the exact value but we will find that if we have provided a proper destructor. Then on the same computer, in the same session, we execute the program, a specific address of memory will be assigned to the program. With the proper destructor, we stop the program and then again start it. Nine out of ten times, we get the same memory. That means we will see the same address. Nine times out of ten is because the operating system can use this memory somewhere else between the times of two executions of the program. If we do not provide a destructor i.e. we do not deallocate the memory, it is necessary that each time we will get a new memory. The previous memory is being wasted. You can prove it by yourselves by writing small programs.

© Copyright Virtual University of Pakistan

CS201-Introduction to Programming
Final Term Solved MCQs
Prepared by: JUNAID MALIK

AL-JUNAID TECH INSTITUTE

www.vulmshelp.com







Language Courses Training Available

I'm providing paid courses in different languages within 3 Months,Certificate will be awarded after completion.

- HTML
- CSS
- JAVASCRIPT
- BOOTSTRAPS
- JQUERY
- PHP MYSQL
- NODES.JS
- REACT JS

LMS Handling Services

LMS Activities Paid Task

Assignments 95% Results

Quizes 95% Results

GDB 95% Results

For CS619 Project Feel Free To Contact With Me

Ph# 0304-1659294

Email: junaidfazal08@gmail.com

AL-JUNAID INSTITUTE GROUP

ALL answers are verified if found any mistake then Correct ACCORDINGLY

Question no.1

Which of the following function call is "call by Reference" for the following function prototype?

a. Func(&num)

b. Func(name)

c. A&b

d. None of the given option

Question no.2

When break statement is encountered in switch statement,it_____

a. **Exits from switch statement**

b. enter from switches statement

c. both a&b

d. none of the given option

0304-1659294
www.vulmshelp.com

Question no.3

The operator ++ and – used to increment or decrement the value of a variable by_____

a.3

b.2

c. 1

d.4

Question no.4

AL-JUNAID INSTITUTE GROUP

Which of the following operator is used to access the value of variable pointed by a pointer?

a. *operator
b.->operator
c. &&operator
d. & operator

Question no.5

In case of single dereferencing the value of the____ is the address of the _____

a. Pointer ,variable
b. pointer, constant
c. variable, pointer
d. constant, pointer

Question no.5

The remainder(%) operator is a _____ operator.

a. logical
b. Arithmetic
c. Relational
d. Conditional

Question no.6

What will be the output of the following code?

Intx=10;
Cout<<"x="<<x;

a. x=10
b. 10
c. 10=x

Question no.7

The perpose of using cout<<is to_____

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

a. Read the data from keyboard
b. Read the data from file
c. Write into a file
d. Display information on the screen

Question no.8

When an array element is passed to function, it is passed by _____

a.reference
b. data type
c. value
d. data

Question no.9

.C is a/an____language.

Ans.function oriented

Question no.10

____ of a function is also known as signature of a function.

a.
b. Declaration
c.
d.

Question no.11

While programming,it is good to provide an easy to understand and easy to use interface; this programming skill is called_____

a. Scalability
b. Reliability
c. sustainability
d. usability

Question no.12

How many dimensions does n-dimensional array has?

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

a. 2n dimension
b. (n+1) dimension
c. (n-1) dimension
d. n dimensions

Question no.13

Which of the following function call is"call by reference" for the following function protocol?

Void func(int*)

a. func(int&num)
b. func(&num)
c. func(*num)
d. func(num)

Question no.14

The loop which is most suitable to be used when the number of iteration is known is called_____

a. for
b. while
c. do-while
d. all looping processes require that the iterations be known

Question no.15

In C/C++, the string constant is enclosed in_____

a. curly braces()
b. parentheses()
c. single quotes' '
d. double quotes" "

Question no.16

In order to get the right most digit of a number, we divide this number by 10 and take_____

a. Its remainder

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

b. Its quotient
c. Its divisor
d. The number

Question no.17

What is the correct syntax to declare an array of size 10 of int data type?

a. int[10] name;
b. name[10] int;
c. int name[10];
d. int name[];

Question no.18

How many bytes of memory are occupied by array'star'?

Char str[]="programming";

a. 10
b. 11
c. 12
d. 13

Question no.19

When the if statement consist more than one statement then enclosing these statement in curly braces is,

a. Not required
b. Good programming
c. Relevant
d. Must

Question no.20

Suppose that an integer type pointer contains a memory address 0x22f230. What will be the new memory address if we increment this pointer by one?

a. 0x22f231

AL-JUNAID TECH INSTITUTE
0304-1639294
www.vulmshelp.com

b. 0x22f234
c. 0x22f226
d. 0x22f238

Question no.21

Which of the following if missing would result in infinite recursion in case of recursive function?

a. Recursive call
b. Base case
c. Function parameters
d. Local variables

Question no.22

Whenever we use a library function or a predefined object or macro, we need to use a_____

a. Source file
b. Object file
c. header file
d. exe file

Question no.23

Switch statement deals with ____type of data

a. Integer
b. Float
c. Character
d. Both Integer and Character

Question no.24

All a template function must have at least_____ generic dats tyoe?

a. Zero
b. One
c. Two
d. Three

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

AL-JUNAID INSTITUTE GROUP

Question no.25

Which one of the following is the declaration of overloaded pre-increment operator implemented as number of function?

a. Class-name operator+()
b. Class-name operator+(int)
c. Class-name operator++()
d. Class-name++operator++(int)

Question no.26

Class is a user defined_____.

a. Data type
b. Memory reference
c. Value
d. None of the given option

Question no.27

How many bytes will the pointer intPtr of type int move in the following statement? intPtr+=3;

a. 3 bytes
b. 6 bytes
c. 12 bytes
d. 24 bytes

Question no.28

Which of the following is the correct C++ syntax to allocate space dynamically for an array 10 int?

a. new int(10);
b. new int[10];
c. int new(10);
d. int new[10];

Question no.29

AL-JUNAID TECH INSTITUTE
0304-1659294
www.ajmshelp.com

A pointer is a special variable that contain

a. Data values
b. Memory address
c. Both data and value
d. None of the given option

Question no.30

Reference Value Type Data the code is written to___

a. Implement
b. Design
c. Analysis
d. None of the given option

Question no.31

Operator overloading can be performed through____

a. Classes
b. Operator
c. Function
d. Reference

Question no.32

When a value is referred by a normal variable then it is known as,

a. Direct reference
b. Indirect reference
c. Partial reference
d. Proper reference

Question no.33

Which of the following function is used to increase the size of already allocated memory chunk?

a. malloc
b. calloc
c. realloc

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

d. free

Question no.34

Which of the following is NOT a preprocessor directive?

a. #error
b. #define
c. #line
d. #ndefine

Question no.35

The Stream object **cin** and **cout** are included in which header file?

a. iostream.h
b. fstream.h
c. istream.h
d. ostream.h

Question no.36

Overloaded delete operator function takes the same parameter as an arguments returned by a new operator function.

a. True
b. False

Question no.37

When an array of object is created by dynamically then there is no way to provide parameterized constructors for array of object.

a. True
b. False

Question no,38

C is widely known as development language of____ operating system.

a. Linux
b. Unix

AL-JUNAID TECH INSTITUTE
www.vulmshelp.com

c. Windows
d. Mac os

Question no.39

Computer can understand only machine language code.

a. True
b. False

Question no.40

We cannot define a function as a friend of a Template class.

a. True
b. False

Question no.41

What will be the value of 'a' and 'b' after executing the following statement?

A=3;
b= a++;

a. 3,4
b. 4,4
c. 3,3
d. 4,3

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

Question no.42

_____is used to trace the logic of the program and correct the logical errors.

a. Compiler
b. Editor
c. Linker
d. Debugger

Question no.43

New and delete are_____ whereas malloc and free are____.

a. Function, operator
b. Classes , operator
c. Operator , function
d. Operator, classes

Question no.45

Like member function,_____ can also access the private data members of a class.

a. Non-member function
b. Friend function
c. Any function outside class
d. None of the given options

Question no.46

Which situation would require the use of non-member overloaded operator?

a. The overloaded operator is an Assignment operator
b. The left most operand is an object of a class.
c. The left operand is built-in data type.
d. The operator returns a reference.

Questions no.47

The stream insertion and stream extraction operator are already overloaded for_____.

a. User-defined data type
b. Built in data type
c. Both “a” and “b”
d. None of the given option

Question no.48

If we define an identifier with the statement **#define PI 3.1415926** then the execution of the program the value of PI_____.

AL-JUNAID TECH INSTITUTE
0304-1659291
www.vulmshelp.com

a. Can not be replaced
b. None of the given option
c. Remain constant
d. Can be changed by some option

Question no.49

Assignment operator is _____ associative.

a. Right
b. Left
c. Binary
d. Unary

Question no.50

Whenever dynamic memory allocation is made in C/C==, it is freed____.

a. Explicitly
b. Implicitly
c. Both explicitly and implicitly
d. None of the given option

Question no.51

The appropriate data type to store the number of rows and columns of the matrix is _____.

a. Float
b. Int
c. Char
d. None of the given option

Question no.52

The function free() return back the allocated memory got through calloc and malloc to_____.

a. Stack
b. Heap

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

c. Stack and heap
d. None of the given option

Question no.53

Width() is member function of _____.

a. Cin object
b. Cout object
c. Both cin and cout object
d. None of the given object

Question no.54

Templates are not type safe.

a. True
b. false

Question no.57

A Matrix can be composed of int, float or doubles as their elements. Best way is to handle this,

a. write a separate class to handle each
b. use string to store all types
c. use templates
d. none of the given option

Question no.58

In if structure the block of statement is executed only,

a. when the condition is false
b. when it contain arithmetic operator
c. when it contain logical operator
d. when the condition is true

Question no.59

Header file **fstream.h** include the definition of the stream classes_____.

AL JUNAID TECH INSTITUTE
0304-1659294
www.ajtihelp.com

a. Ifsream, fstream, ofstream
b. Ifstram, ofstream, cout
c. Fstream, cin, cout
d. None of the given option

Question no.60

To access the data members of structure_____is used.

a. Dot operator(.)
b. * operator
c. &operator
d. None of the given

Question no.61

Eof(), bad(), good(), clear() all are manipulators.

a. True
b. False

Question no.62

Which kind of function can access private member variables of a class?

a. Friend function of the class
b. Private member function of the class
c. Public member function of the class
d. Friend, private and public function

Question no.63

The return type of operator function must always be void.

a. True
b. False

Question no.64

Friend function of a class is____.

a. Member function

AL-JUNAID TECH INSTITUTE 03046659294 www.aihelp.com

b. Non-member function
c. Private function
d. Public function

Question no.65

Function implementation of friend function must be defined outside the class.

a. True
b. False

Question no.66

The normal source of **cin** object is,

a. File
b. Disk
c. Keyboard
d. Ram

Question no.67

Which of the following is correct way to initialize a variable x of int type with value 10?

a. Int x; x = 10
b. Int x=10;
c. Int x, x = 10
d. X =10

Question no.68

With the function, the compiler automatically detects the passed data and generate a new copy of function using passed data.

a. True
b. False

Question no.69

What will be the correct syntax to declear two-dimensional array of float data type?

AL-JUNAID TECH INSTITUTE 0304-1659294 www.aljunaidtech.com

a. Float{2}{2};
b. Float arr[2][2];
c. Float arr[2,2]
d. Float[2][2]

Question no.70

Heap is a constantly changing in size.

a. True
b. False

Question no.71

While calling function, the arguments are assigned to the parameters from _____.

a. Left to right
b. Right to left
c. No space order is followed
d. None of the given option

Question no.72

Classes defined inside other classes are called____.

a. Looped
b. Nested
c. Overloaded
d. None of the given option

Question no.73

If we define an identifier with the statement #**defined PI 3.1415926** then during the execution of the program the value of PI____.

a. Cannot be replaced
b. None of the given option
c. Remain constant
d. Can be changed by some operation

Question no.74

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vuhelp.com

Which value is returned by the destructor of a class?

a. A pointer to the class.
b. An object of the class
c. A status code determining whether the class was destructed
d. Destructor do not return a value.

Question no.75

Every class contains_____.

a. Constructor
b. Destructor
c. Both a and b
d. None

Question no.76

A template function must have

a. One or more than one arguments
b. Only one arguments
c. Zero arguments
d. None

Question no.77

Structured query language is used for _____.

a. Database management
b. Network
c. Writing operating system
d. None

Question no.78

When a call to user defined function finishes the variable defined inside the function still in existence.

a. True
b. False

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

Question no.79

The precedence of an operator can be changed through operator overloading.

a. True
b. False

Question no.80

"delete" operator is used to return memory to free store, which is allocate by the "**new**" operator.

a. True
b. False

Question no.81

Pointer is a variable which store.

a. Data
b. Memory address
c. Data type
d. values

Question no.82

All preprocessor directives are started with the symbol____.

a. *
b. +
c. @
d. #

Question no. 83

The number 544.53 must be stored in_____ data type.

a. Int
b. Short
c. Float
d. Char

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

AL-JUNAID INSTITUTE GROUP

Question no.84

A template function can have different types of arguments.

a. True
b. False

Question no.85

Templates class cannot have static variable.

a. True
b. False

Question no.86

Which of the following is used with bit manipulation?

a. Signed integer
b. Un signed integer
c. Signed double
d. Un-signed double

Question no.87

Structure is a collection of____ under a single name.

a. Only function
b. Only variable
c. Both function and variable
d. None

Question no,88

We can also do conditional compilation with preprocessor directives.

a. True
b. False

Question no.89

If a symbolic constant has been defined, it will be an error to define it again.

a. True

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

b. False

Question no.90

The code is written is to___ the program.

a. Implement
b. Design
c. Analysis
d. None

Question no.91

Template are not type safe.

a. True
b. false

Question no.92

*doc is _____ by type.

a. sequential file
b. Random Access File
c. Data File
d. Record File

Question no.93

Which of the following is NOT a preprocessor directive?

a. #error
b. #define
c. #line
d. #ndefine

Question no.94

The return type of operator function must always be void.

a. True
b. false

Question no.95

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

What does (*this) represent?

a. The current function of the class.
b. The current pointer of the class.
c. The current object of the class.
d. A value of a data number.

Question no.96

The statement **cin.get()** is used to

a. Read a string from keyboard
b. Read a char from keyboard
c. Read a string from file
d. Read a char from file

Question no.97

C++ is a sensitive language

a. True
b. False

Question no.98

Which one of the statement given below is equivalent to **sum+=j*j**?

a. sum=sum+j*j
b. sum= sum+j
c. sum= sum*j
d. sum*j=j

Question no.99

.All element of an array must be of____

a.different data type
b.:same datatype
c.float data type
d. char data type

Question no.100

AL-JUNAID TECH INSTITUTE
0304-1659294
www.aliishelp.com

AL-JUNAID INSTITUTE GROUP

In C/C++, null character is represented as_____

a. /n
b. %c
c. #d
d. *c

Question no.101

The dynamic memory allocation uses memory from the____

a. Heap
b. Stack
c. Values
d. none

Question no.102

When a value is reffered by a normal variable then it is known as___

a. Direct Referance
b. Indirect reference
c. Both a and b
d. none

Question no.103

Assignment operator is used for_____

a. assigning values to variables
b. assigning value to a constant
c. assigning constant and variables
d. none of the given option

Question no.104

Which of the following is not an example of int datatype?

a. -4.0
b. +4.2
c. -3.0
d. -2.0

AL JUNAID TECH INSTITUTE
AJ
ISO 9001-1659294
www.vulmshelp.com

AL-JUNAID INSTITUTE GROUP

Question no.105

For which array,the size of the array should be one more than the number of elements in an array?

a. Char
b. Variable
c. Float
d. Intiger

Question no.106

Which header file should be included to use functions like malloc() and calloc()?

Ans:stdio.h

Question no.107

Using a segment of source code again by adding new functionalities with slight or no modification is termed as____

Ans:code reusability

Question no.108

The program in which we allocate static memory run essentially on_____

Ans:Stack

Question no,109

TWAIN stand for___

Ans:Technology Without An Interesting Name

Question no.110

Which of the following will be the correct function call for function prototype given below?

Ans:func(&num)

AL-JUNAID TECH INSTITUTE
0304-1659294
www.aljunaidtechinstitute.com

AL-JUNAID INSTITUTE GROUP

Question no.111

The operator used for casting in C is standard____

Ans:cast

Question no.112

The stream insertion and extraction operators___

Ans:must be overloaded as non-member function

Question no.113

The operator function for stream insertion(>>)and stream extraction(<<) must be_____

Ans:Non-Member function of Class

Question no.114

In statement Matrix m2=m1;

Ans:Assignment operator is being used

Question no.115

A template function must have at least_____generic datatype

Ans:One

Question no.116

Templates provide way of abstracting_____information.

Ans:type

Question no.117

ANSI stand for_____

Ans:American National Standards Institute

Question no.118

By using object as class members,_____is achieved.

Ans:Accessibility

0304-1659294
www.vulmshelp.com

Question no.119

In operator overloading,a non-member unary operator function takes_____ arguments.

Ans:1

Question no.120

A class whose object is contained as const object, must have____

Ans:Parameterized Constructor

Question no.121

It is possible to define a class within other class.

a. True
b. false

Question no.122

Where we can include a header file in the program?

a. Any where
b. In start
c. At the end
d. None of the given option

Question no.123

The return type of the operator function >> operator.

a. Class for which we overloaded this operator
b. Reference of ostream class
c. Reference of istream class
d. Void

Question no.124

With user defined data type variables(object) self assignment can produce_____.

a. Syntax error
b. Logical error

AL JUNAID TECH INSTITUTE www.vulmshelp.com

c. Link error
d. None of the given option

Question no.125

We can also do conditional compilation with preprocessor directives.

a. True
b. False

Question no.126

What will be the output of the following statement?

Cout<<setbase(16)<<52;

a. 74
b. 52
c. 34
d. 64

Question no.127

Dec, her, oct are all_____.

a. Member function
b. Object of input/output stream
c. Parameterized manipulator
d. Non- parametrized manipulator

Question no.127

The input output stream; cin cout are___.

a. Operator
b. Function
c. Object
d. Structures

Question no.128

Friend function are ____ of a class

AL JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

a. Member functions
b. Public member function
c. Private member function
d. Non-member function

Question no.129

Template class can not have static variables.

a. True
b. False

Question no.130

In a group of a nested loops, which loop is executed the most member of times?

a. The outermost loop
b. The innermost loop
c. All loops are executed
d. Cannot be determined

Question no.131

Which looping process checks the test condition at the end of the loop?

a. For
b. While
c. Do while
d. No looping process checks the test condition at the end.

Question no.131

We can delete an array of object without specifying[] bracket if a class is not dynamic memory allocation internally.

a. True
b. False

Question no.132

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

A stream is an ordered sequence of bytes.

a. True
b. False

Question no.133

The normal source of **cin** object is,

a. File
b. Disk
c. Keyboard
d. Ram

Question no.134

We can also create an array of user define data type.

a. True
b. False

Question no.135

Which of the following is valid class declaration?

a. Class A{int x;};
b. Class B{]
c. Public class A{ }
d. Object A{ int x;};

Question no.136

If Num is an integer variable then Num++ means,

a. Add 1 two times with Num
b. Add 1 with Num
c. Add 2 with Num
d. Subtract 2 from Num

Question no.137

Automatic variable are created on-_____.

a. Heap

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

b. Free storage
c. Static storage
d. Stack

Question no.138

We can also define a user-defines manipulator.

a. True
b. False

Question no.139

Class is a user defined ____.

a. Data type
b. Memory reference
c. Value
d. None of the given option

Question no.140

The static members of a class are initialized_____.

a. At file scope
b. Within class definition
c. Within member function
d. Within main function

0304-1659294
www.ulmshelp.com

Question no.141

The data members of the class are initialized_____

a. At runtime
b. Within main group
c. Outside the function
d. At compile time

Question no.142

In flow chart, flow of control is represented by____

a. Rectangle
b. Circle

c. Diamond
d. Arrow

Question no.143

A constructor has the same name as that of_____

a. Variable
b. Class
c. Structure
d. Function

Question no.144

Whenever **New** operator is called to create an object, the _____

a. Constructor
b. Destructor
c. Pointer
d. Array

Question no.145

The destructor for all the objects in the program in automatically called when____

a. Memory is dynamically allocated to object
b. Object are used inside function
c. Array of object is decleared
d. Program is terminated

Question no.146

What will be the correct syntax of declearation of the following statement?

Ptr is a pointer to cons int

a. Const int*ptr
b. Int* const ptr,
c. Int const* ptr

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

d. Const* int ptr

Question no.148

Sequence of event(s) when deallocating memory using delete operator____

a. Only block of memory is deallocated for object
b. Only destructor is called for object
c. Memory is deallocated first before calling destructor
d. Destructor is called first before deallocating memory

Question no.149

Consider the following segment'
Class M{
Public;
M&operator+ (constM&);

a. Member function
b. Non- member function
c. Friend function
d. Virtual function

Question no.150

if there is a symbol(& sign) used with the variable name following by data type then it refers to ____

a. Logic expression
b. Value of variable
c. Reference variable
d. Address of variable

Question no.151

New and delete are also used with ____ and primitive data typed as well.

a. Loops
b. Sructures , pointer

0304-1659294 www.ulmshelp.com

c. Classes , structures
d. Constant

Question no.152

Users must not know about____

a. Methods functionality with in class
b. Object of class
c. Functions name within class
d. Class implementation of functionality and interfaces

Question no.153

What should be the return type of the constructed?

a. Void
b. Int
c. Same as object type
d. Construction do not return any thing

Question no.154

Which is NOT a protection level provided by classes in C++?

a. Protected
b. Hidden
c. Private
d. Public

Question no.155

A macro when takes arguments is called_____

a. Function
b. Procedure
c. Parameterized macro
d. Simple macro

Question no.156

The order of destruction of an object is_____

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

a. Same as the construction order
b. Reverse as the construction order
c. Independent of the construction order
d. Same as the order of object decleared

Question no.157

We can use cin.peak() function to__-

a. See the next character that we are going to get
b. Get to the end of the object buffer
c. Get to the end of input buffer
d. See the first character of input buffer

Question no.158

The normal source of cin object is____

a. File
b. Disk
c. Keyboard
d. Ram

Question no.159

The initializer list is used to initialize the contained objects of a class at____

a. Run time
b. Debugging time
c. Construction time
d. Compile time

Question no.160

_____data members of a contained object can be accessed from inside of the containing class.

a. Private
b. Public
c. Protected

0304-1659294
www.vulmshelp.com

d. All

Question no.161

A class whose object is contained as const object, must have_____

a. Getter/setter function
b. Default constructor
c. Parameterized constructor
d. Operator overloading function

Question no.162

The appropriate data type to store the number of rows and coloums of the matrix_____

a. Float
b. Strings
c. Int
d. Char

Question no.163

The code is written to_____ the program

a. Implement
b. Design
c. Analysis
d. Test

0304-1659294
www.vulmshelp.com

Question no.164

When a call to user-defined function finishes, the variable defined inside the function will_____

a. Still in existence
b. Not exist further
c. Finish but a copy will remain in memory
d. None of the given

Question no.165

The static data member of a class can be accessed by____

a. Only class
b. Only object
c. Both class and object
d. Function out side the class

Question no.166

The static data members of a class are initialized____

a. At fill scope
b. Within class definition
c. Within member function
d. Within main function

Question no.167

A structure brings together a group of_____

a. Same data type
b. Different data type
c. Constant
d. Both a,b

Question no.168

The C language was developed in late 60's and 70's in____

a. Richards laboratories
b. Microsoft laboratories
c. Bell laboratories
d. IBM laboratries

Question no.169

To assigned a value to a character type variable____ are used around the value.

a. Parenthesis()
b. Single quotes ' '
c. Double quotes " "
d. Curly braces{ }

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

AL-JUNAID INSTITUTE GROUP

Question no.170

The order of destruction of an object is ____

a. Same as the construction order
b. Reverse as the construction order
c. Independent of the construction order
d. Same as the order of object declare

Question no.171

In flow chart, flow of control is represented by _____.
(Please select correct option)

a. Circle
b. Diamond
c. Arrow
d. All

Question no.172

The operator ++ and – are used to increment or decrement the value of a variable by _____.

a. 3
b. 2
c. 1
d. 4

Question no.173

To assigned a value to a character type variable,_____ are used around the value.

a. Parenthesis ( )
b. Single quotes ‘ ’
c. Double quotes “ ”
d. Curly braces { }

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

AL-JUNAID INSTITUTE GROUP

Question no.174

Which of the following syntax will be used to initialize an array of size 5 of int data type to value 0?

a. Arr[5] = {0} ;
b. int arr[5] = 0 ;
c. int arr[5] = (0) ;
d. int arr[] = 0 ;

Question no.175

Let ptr 1 and ptr 2 are pointer variables that points to integer data types then which one of the following arithmetic is allowed?

a. Ptr1 + ptr2
b. Ptr1 – ptr2
c. Ptr1 * ptr2
d. Ptr1 / ptr2

Question no.176

When an argument is passed by reference, _____

a. The function cannot access the argument's value
b. A variable is created in the function to hold the argument's value
c. A temporary variable is created in the calling program to hold the argument's value
d. The function accesses the argument's original value in the calling program

Question no.177

In year _____ C was approved as a standard language worldwide.

a. 1988
b. 1967

AL-JUNAID TECH INSTITUTE
0304-1639294
www.vulmshelp.com

c. 1989

d. 1986

Question no.178

The C language was developed in late 60's and early 70's, in _____

a. Richards Laboratories

b. Microsoft Laboratory

c. Bell Laboratory

d. IBM Laboratory

Question no.179

If a friend function outside the class declares itself friend of the class, this may violate the concept of _____

a. Encapsulation and data accessing

b. Encapsulation and data hiding

c. Friend member functions of the class

d. Interface of the class

Question no.180

A structure brings together a group of _____

a. Same data type

b. Different data type

c. Constants

d. Both a, b

Question no.181

You can use a _____ statement to transfer control elsewhere out of the nested structure.

a. Continue

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

b. Goto
c. Pause
d. Break

Question no.182

Which of the given operator is not a valid compound assignment operator?

a. >>=
b. =>>
c. *=
d. &=

Question no.183

The purpose of using cout<< is to ____

a. Display information on the screen
b. Read the data from keyboard
c. Read the data from file
d. Write into a file

0304-1659294
www.vulmshelp.com

Question no.184

Assignment operator is used for_____

a. Calculation
b. Reading
c. Assigning values to variables
d. Equality checking

Question no.185

Missing semicolon ‘ ;’ at the end of C++ statement causes a ____ error.

a. Logical
b. Syntax
c. Runtime
d. Compile time

Question no.186

Which of the following is not an example of int data type?

a. 0
b. -32
c. 65531
d. -4.0

Question no.189

For which array , the size of the array should be one more than the number of elements in array?

a. Int
b. Double
c. Float
d. Char

Question no.190

If an integer takes 4 bytes in the memory then how many bytes will be in the memory for following code?

#include<iostream.h>

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

```
#include< conio.h>

Int main()
```

a. 12 bytes
b. 24 bytes
c. 48 bytes
d. 32 bytes

Question no.191

Using a segment of source code again by adding new functionalities with slight or no modification is termed as _____

a. Code efficiency
b. Code reusability
c. Code notification
d. Code compatibility

Question no.192

A matrix can be composed of ints, floats or doubles as their elements. Best way is to handle this____

a. Wrire a separate class to handle each
b. Use templates
c. Use strings to store all types
d. None of the given option

Question no.193

When a call user defined function finishes, the variable defined inside the function is still in existence.

AL JUNAID TECH INSTITUTE
03041659294
www.aimshelp.com

a. True
b. false

Question no. 194

Within the statement **obj1 = obj2 ; *obj1*** will call the assignment operator function and ***obj2*** will be passed as an argument to function.

a. True
b. False

Question no.195

What is the sequence of events(s) when deallocating memory using delete operator?

a. Only block of memory is deallocated for object
b. Only destructor is called for objects
c. Memory is deallocated first before calling destructor
d. Destructor is called first before deallocating memory

Question no.196

The second parameter of operator function for << and >> are objects of the class for which we are overloading these operator.

a. True
b. False

Question no.197

0304-1659294
www.vulmshelp.com

For which values of the integer_ value will be the following code becomes an infinite loop?

```
Int number=1;
While(true) {
Cout<< number;
If(number==3) break;
Number+= integer_ value: }
```

a. Any number other than 1 or 2
b. Only 0
c. Only 2
d. Only 1

Question no.198

Unary operator implemented as member function takes ___ arguments whereas non-member function takes____ arguments.

a. New int (10)
b. New int [10]
c. Int new (10)
d. Int new [10]

Question no.199

What will be the correct syntax to assign an array named ***arr*** of 5 elements to a pointer ***ptr***?

a. *ptr = arr;

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

b. Ptr = arr;
c. *ptr = arr[5];
d. Ptr = arr[5];

Question no.200

We can also do conditional compilation with preprocessor directive?

a. True
b. false

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

FINALTERM EXAMINATION 2011
CS201- Introduction to Programming

Question No: 1 (Marks: 1) - Please choose one
All A template function must have at least ---------- generic data type

► Zero
► One (Page 499)
► Two
► Three

Question No: 2 (Marks: 1) – Please choose one
Which of the following statement is best regarding declaration of friend function?

► Friend function must be declared after public keyword
. ► Friend function must be declared after private keyword.
► Friend function must be declared at the top within class definition.
► It can be declared anywhere in class as these are not affected by the public and private keywords. (Page 346)

Question No: 3 (Marks: 1) - Please choose one
Which one of the following is the declaration of overloaded pre-increment operator implemented as member function?

► Class-name operator +() ;
► Class-name operator +(int) ;
► **Class-name operator ++() ; (Page 389)**
► Class-name operator ++(int) ;

Question No: 4 (Marks: 1) - Please choose one
Class is a user defined_____.

► **data type (Page 317)**
► memory referee
► value
► none of the given options.

Question No: 5 (Marks: 1) - Please choose one
How many bytes will the pointer intPtr of type int move in the following statement? intPtr += 3 ;

► 3 bytes
► 6 bytes
► **12 bytes**
► 24 bytes

Question No: 6 (Marks: 1) - Please choose one
Which of the following is the correct C++ syntax to allocate space dynamically for an array of 10 int?

► new int(10) ;
► **new int[10] ;**
► int new(10) ;
► int new[10];

Question No: 7 (Marks: 1) - Please choose one
A Pointer is a special variable that contain

► Data values
► **Memory Address (Page 267)**
► Both data and values
► None of the given option

Question No: 8 (Marks: 1) - Please choose one
Reference Value Type Data The code is written to _____ the program.

► **implement**
► design
► analysis
► none of the given options

FINALTERM EXAMINATION
CS201 - Introduction to programming
Final Term Spring 2010

Operator overloading can be performed through_____.
► Classes
► **Function (Page 371)**
► Operators
► Reference

Question No: 2 (Marks: 1) - Please choose one
When a value is referred by a normal variable then it is known as,

► **Direct Reference (page 143)**
►Indirect Reference.

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

► Partial Reference
► Proper Reference

When a value is referred by a normal variable is known as direct reference

Question No: 3 (Marks: 1) - Please choose one

Which of the following function is used to increase the size of already allocated memory chunk?

► malloc
► calloc
► **realloc (Page 285)**
► free

Question No: 4 (Marks: 1) - Please choose one

Which of the following is NOT a preprocessor directive?

► #error
► #define
► #line
► **#ndefine (Page 272)**

Question No: 5 (Marks: 1) – Please choose one

The stream objects **cin** and **cout** are included in which header file?

► **iostream.h (Page 15)**
► fstream.h
► istream.h
► ostream.h

Question No: 6 (Marks: 1) – Please choose one

Overloaded delete operator function takes the same parameter as an argument returned by new operator function.

► **True (Page 414)**
► False

The same pointer that is returned by the new operator, is passed as an argument to the delete operator. These rules apply to both, if operators (new and delete) are overloaded as member or non-member operators (as global operators).

Question No: 7 (Marks: 1) – Please choose one

When an array of object is created dynamically then there is no way to provide parameterized constructors for array of objects.

► **True (Page 414)**
► False

Question No: 8 (Marks: 1) – Please choose one

C is widely known as development language of _____ operating system.

- ► Linux
- ► Windows
- ► **Unix (Page 12)**
- ► Mac OS

In the start C became widely known as the development language of the UNIX operating system, and the UNIX operating system was written by using this C language. The C language is so powerful that the compiler of C and other various operating systems are written in C.

Question No: 9 (Marks: 1) - Please choose one

Computer can understand only machine language code.

- ► **True**
- ► False

Question No: 10 (Marks: 1) - Please choose one

We can not define a function as a friend of a Template class.

- ► True
- ► **False (Page 151)**

Class templates can have friends. A class or class template, function, or function template can be a friend to a template class. Friends can also be specializations of a class template or function template, but not partial specializations.

Question No: 11 (Marks: 1) – Please choose one

What will be the value of 'a' and 'b' after executing the following statements?

```
A = 3;
b = a++;
```

- ► 3, 4
- ► 4, 4
- ► 3, 3
- ► **4, 3**

Question No: 12 (Marks: 1) – Please choose one

Consider the following code segment. What will be the output of following code?

```
Int addValue (int *a){
int b = (*a) + 2;
return b ;
}
main () {
int x =6 ;
cout <<  x << "," ;
cout << addValue(&x) << "," ;
cout <<  x ;
}
```

► **6,8,6**
► 6,6,8
► 6,8,8
► 6,6,6

Question No: 13 (Marks: 1) – Please choose one
_____ is used to trace the logic of the program and correct the logical errors.
► Compiler
► Editor
► Linker
► **Debugger (Page 13)**

Question No: 14 (Marks: 1) - Please choose one
new and delete are _____ whereas malloc and free are _____.

► Functions, operators
► Classes, operators
► **Operators, functions (Page 342-285)**
► Operators, classes

Hence, we can call new and delete operators, P# 342
we have allocated a memory space for our use by malloc function. P# 285

Question No: 15 (Marks: 1) - Please choose one
Like member functions, _____ can also access the private data members of a class.

► Non-member functions
► **Friend functions (Page 346)**
► Any function outside class
► None of the given options

Question No: 16 (Marks: 1) – Please choose one
Which situation would require the use of a non-member overloaded operator?

► The overloaded operator is an Assignment operator.
► **The left most operand is an object of a class.(Page 375)**
► The left operand is built-in data type.
► The operator returns a reference.

When an operator function is implemented as a non-member function, the left-most operand may be an object of the operator's class, an object of a different class, or a built-in type

Question No: 17 (Marks: 1) - Please choose one
The stream insertion and stream extraction operators are already overloaded for _____.
► User-defined data types
► **Built-in data types http://www.sstutor.com/cpp/stream1.htm**
► User-defined and built-in data types
► None of the given options

Question No: 18 (Marks: 1) - Please choose one
If we define an identifier with the statement **#define PI 3.1415926** then during the execution of the program the value of PI _____.

- ► can not be replaced
- ► None of the given options
- ► **Remain constant. (Page 271)**
- ► can be changed by some operation

Question No: 19 (Marks: 1) - Please choose one
Assignment operator is ----------associative.

- ► **right**
- ► left
- ► binary
- ► unary

Question No: 20 (Marks: 1) - Please choose one
When ever dynamic memory allocation is made in C/C++, it is freed_____.

- ► **Explicitly (Page 527)**
- ► Implicitly
- ► Both explicitly and implicitly
- ► None of the given options

Question No: 21 (Marks: 1) – Please choose one
The appropriate data type to store the number of rows and colums of the matrix is_____.

- ► float
- ► **int (Not sure)**
- ► char
- ► none of the given options.

Question No: 22 (Marks: 1) – Please choose one
Which of the following function do NOT initialize the chunk of memory to all zero?

- ► calloc() function
- ► Both malloc() and calloc()
- ► None of the above
- ► **malloc() function** (Page 283)

The **malloc** function differs from **calloc** in the way that the space allocated by **malloc** is not initialized and contains any values initially.

Question No: 23 (Marks: 1) – Please choose one
The function free() returns back the allocated memory got thorough calloc and

malloc to _____ .

- ► stack
- ► **heap**
- ► stack and heap
- ► None of the given options

Question No: 24 (Marks: 1) – Please choose one
width() is member function of _____

- ► cin object
- ► cout object
- ► **Both cin and cout object (Page 440)**
- ► None of the given option

Question No: 25 (Marks: 1) - Please choose one
Templates are not type safe.

- ► true
- ► **false (Page 518)**

Templates are type-safe. This is because the types that templates act upon are known at compile time, so the compiler can perform type checking before errors occur.

Question No: 26 (Marks: 1) - Please choose one
A Matrix can be composed of ints, floats or doubles as their elements. Best way is to handle this ,
_____

- ► Write a separate class to handle each
- ► **Use templates (Page 527)**
- ► Use strings to store all types
- ► None of the given options

FINALTERM EXAMINATION
Spring 2010
CS201- Introduction to Programming

Question No: 1 (Marks: 1) - Please choose one
In if structure the block of statements is executed only,

- ► When the condition is false
- ► When it contain arithmetic operators
- ► When it contain logical operators
- ► **When the condition is true (Page 38)**

Question No: 2 (Marks: 1) - Please choose one
Header file: **fstream.h** includes the definition of the stream classes _____.

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

► ifstream, fstream, cout
► **ifstream, fstream, ofstream (Page 199)**
► fstream, cin, cout
► None of the above

Question No: 3 (Marks: 1) - Please choose one
To access the data members of structure _____ is used.

► **dot operator (.) (Page 231)**
► * operator
► □ operator
► None of given.

Question No: 4 (Marks: 1) - Please choose one
eof( ), bad( ), good( ), clear( ) all are manipulators.

► **True (Page 433)**
► False

Question No: 5 (Marks: 1) - Please choose one
Which kind of functions can access private member variables of a class?

► **Friend functions of the class (Page 374)**
► Private member functions of the class
► Public member functions of the class
► Friend, private and public functions

Question No: 6 (Marks: 1) - Please choose one
The return type of operator function must always be void.

► True
► **False (Page 79)**

Question No: 7 (Marks: 1) - Please choose one
Friend function of a class is _____ .

► Member function
► **Non-member function (Page 348)**
► Private function
► Public function

Question No: 8 (Marks: 1) - Please choose one
Function implementation of friend function must be defined outside the class.

► True
► **False (Page 346)**

Question No: 9 (Marks: 1) - Please choose one

The normal source of **cin** object is,

- ► File
- ► Disk
- ► **Keyboard (Page 424)**
- ► RAM

Question No: 10 (Marks: 1) - Please choose one
Which of the following is correct way to initialize a variable x of int type with value 10?

- ► int x ; x = 10;
- ► **int x = 10;**
- ► int x, x = 10;
- ► x = 10;

Question No: 11 (Marks: 1) - Please choose one
Consider the following code segment. What will be the output of the following program?

```
int func(int) ;
int num = 10 ;

int main(){
int num ;
num = 5 ;
cout << num ;
cout << func(num) ;
}
int func(int x){
return num ;
}
```

- ► 5, 5
- ► 10, 5
- ► **5, 10**
- ► 10, 10

Question No: 12 (Marks: 1) - Please choose one
With template function, the compiler automatically detects the passed data and generates a new copy of function using passed data.

- ► **True Page 510**
- ► False

Question No: 13 (Marks: 1) - Please choose one
What will be the correct syntax to declare two-dimensional array of float data type?

- ► float arr{2}{2} ;
- ► **float arr[2][2] ;**

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

► float arr[2,2] ;
► float[2][2] arr ;

Question No: 14 (Marks: 1) - Please choose one

The first parameter of operator function for << operator,

► Must be passed by value
► **Must be passed by reference (Page 446)**
► Can be passed by value or reference
► Must be object of class

Question No: 15 (Marks: 1) - Please choose one
Heap is constantly changing in size.

► **True (Page 280)**
► False

Question No: 16 (Marks: 1) - Please choose one
While calling function, the arguments are assigned to the parameters from _____.

► **left to right. (Page 295)**
► right to left
► no specific order is followed
► none of the given options.

Question No: 17 (Marks: 1) - Please choose one
Classes defined inside other classes are called _____ classes

► looped
► **nested (Page 492)**
► overloaded
► none of the given options.

Question No: 18 (Marks: 1) - Please choose one
If we define an identifier with the statement **#define PI 3.1415926** then during the execution of the program the value of PI _____.

► can not be replaced
► None of the given options
► **Remain constant. (Page 271)**
► can be changed by some operation

Question No: 19 (Marks: 1) - Please choose one
Which value is returned by the destructor of a class?

► A pointer to the class.

► An object of the class.
► A status code determining whether the class was destructed correctly
► **Destructors do not return a value. (page 323)**

Question No: 20 (Marks: 1) - Please choose one
Every class contains _____.

► Constructor
► Destructor
► **Both a constructor and a destructor (Lecture 27)**
► None of the given options

Question No: 21 (Marks: 1) - Please choose one
A template function must have

► **One or more than one arguments**
► Only one argument
► Zero arguments
► None of the given options
http://en.wikipedia.org/wiki/Template_%28programming%29

Question No: 22 (Marks: 1) - Please choose one
Structured Query Language is used for _____

► **Databases Management (page 563)**
► Networks
► Writing Operating System
► none of the given options

Question No: 23 (Marks: 1) - Please choose one
When a call to a user-defined function finishes, the variable defined inside the function is still in existence.

► True
► **False (Page 460)**

Question No: 24 (Marks: 1) - Please choose one
The precedence of an operator can be changed through operator overloading.

► True
► **False (Page 372)**

Question No: 25 (Marks: 1) - Please choose one
A Matrix can be composed of ints, floats or doubles as their elements. Best way is to handle this ,
_____

► Write a separate class to handle each
► **Use templates (Page 527)**
► Use strings to store all types
► None of the given options

Question No: 26 (Marks: 1) - Please choose one
"delete" operator is used to return memory to free store, which is allocated by the **"new"** operator.

► **True (page 332)**
► False

FINALTERM EXAMINATION
Spring 2010
CS201- Introduction to Programming

Question No: 1 (Marks: 1) - Please choose one
Pointer is a variable which store,

► Data
► **Memory Address (Page 267)**
► Data Type
► Values

Question No: 2 (Marks: 1) - Please choose one
All preprocessor directives are started with the symbol_____.

► *
► +
► @
► **# (Page 269)**

Question No: 3 Marks: 1) - Please choose one

Within the statement **obj1=obj2;** ***obj1*** will call the assignment operator function and ***obj2*** will be passed as an argument to function.

► True
► False

Question No: 4 (Marks: 1) - Please choose one
What is the sequence of event(s) when deallocating memory using delete operator?

► Only block of memory is deallocated for objects
► Only destructor is called for objects
► Memory is deallocated first before calling destructor
► **Destructor is called first before deallocating memory (Page 414)**

Question No: 5 (Marks: 1) - Please choose one
The second parameter of operator functions for << and >> are objects of the class for which we are

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

overloading these operators.

► **True (page 453)**
► False

Question No: 6 (Marks: 1) - Please choose one
To include code from the library in the program, such as iostream, a directive would be called up using this command.

► #include “iostream.h”
► include <iostream.h>
► include <iostream.h>
► **#include <iostream.h> (sure)**

Question No: 7 (Marks: 1) - Please choose one
The number 544.53 must be stored in _____ data type.

► int
► short
► **float (Sure)**
► char

Question No: 8 (Marks: 1) - Please choose one
A template function can have different type of arguments.

► **True (Page 503)**
► False

Question No: 9 (Marks: 1) - Please choose one
For which values of the integer _value will the following code becomes an infinite loop?

```
int number=1;
while (true) {
        cout << number;
        if (number == 3) break;
        number +=  integer_value; }
```

► any number other than 1 or 2
► **only 0**
► only 1
► only 2

Question No: 10 (Marks: 1) - Please choose one
Template class cannot have static variables.

► True
► False

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

Question No: 11 (Marks: 1) - Please choose one
Which of the following is used with bit manipulation?

- ► Signed integer
- ► **Un-signed integer (Page 245)**
- ► Signed double
- ► Un-signed double

Question No: 12 (Marks: 1) - Please choose one
Structure is a collection of _____ under a single name.

- ► Only Functions
- ► **Only Variables (Page 229)**
- ► Both Functions and Variables
- ► None of the given options

Question No: 13 (Marks: 1) - Please choose one
Which of the following is the correct C++ syntax to allocate space dynamically for an array of 10 int?

- ► new int(10) ;
- ► **new int[10] ;**
- ► int new(10) ;
- ► int new[10];

Question No: 14 (Marks: 1) - Please choose one
Unary operator implemented as member function takes ____ arguments whereas non-member function takes _____ arguments.

- ► One, zero
- ► **Zero, one**
- ► One, two
- ► Two, one

Question No: 15 (Marks: 1) - Please choose one
The first parameter of overloaded stream insertion operator is _____ where second parameter is _____

- ► input stream, object of class
- ► object of class, output stream
- ► output stream, object of class
- ► object of class, input stream

Question No: 16 (Marks: 1) - Please choose one
We can also do conditional compilation with preprocessor directives.

- ► **True (Page 272)**
- ► False

Question No: 17 (Marks: 1) - Please choose one
If a symbolic constant has been defined, it will be an error to define it again.

► **True (Page 273)**
► False

Question No: 18 (Marks: 1) - Please choose one
While calling function, the arguments are assigned to the parameters from _____.

► **left to right. (Page 295)**
► right to left
► no specific order is followed
► none of the given options.

Question No: 19 (Marks: 1) - Please choose one
Classes defined inside other classes are called _____ classes

► looped
► **nested (Page 492)**
► overloaded
► none of the given options.

Question No: 20 (Marks: 1) - Please choose one
If we define an identifier with the statement **#define PI 3.1415926** then during the execution of the program the value of PI _____.

► cannot be replaced
► None of the given options
► **Remain constant. (Page 271)**
► can be changed by some operation

Question No: 21 (Marks: 1) - Please choose one
Assignment operator is ----------associative.

► **right**
► left
► binary
► unary
http://msdn.microsoft.com/en-us/library/ms173145.aspx

Question No: 22 (Marks: 1) - Please choose one
If **text** is a pointer of class **String** then what is meant by the following statement?
text = new String [5];

► Creates an array of 5 string objects statically
► **creates an array of 5 string objects dynamically**

Mc100401285 moaaz.pk@gmail.com
Bc100400662 bc100400662asad@gmail.com

► Creates an array of pointers to string
► Creates a string Object

Question No: 23 (Marks: 1) - Please choose one
The return type of the operator function for << operator is _____.

► **class for which we overload operator**
► reference of ostream class (ostream&)
► reference of istream class (istream&)
► void

Question No: 24 (Marks: 1) - Please choose one
The code is written to _____ the program.

► **implement (Page 520)**
► design
► analysis
► none of the given options.

Question No: 25 (Marks: 1) - Please choose one
Memory allocated at run time is a system resource and it is the responsibility of _____ to de-allocate the memory.
► System
► **Programmer (Page 331)**
► User of program
► None of given options

Question No: 26 (Marks: 1) - Please choose one
Templates are not type safe.

► True
► **False (Page 518)**

FINALTERM EXAMINATION
Spring 2010
CS201- Introduction to Programming

Question No: 1 (Marks: 1) - Please choose one
*.doc is _____ by type.
.
► Sequential File
► Random Access File
► Data File
► Record File

Question No: 2 (Marks: 1) - Please choose one
Which of the following is NOT a preprocessor directive?

► #error

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

► #define
► #line
► **#ndefine (Page 2727)**

Question No: 3 (Marks: 1) - Please choose one
The return type of operator function must always be void.

► True
► **False (Page 79)**

Question No: 4 (Marks: 1) - Please choose one
What does **(*this)** represents?

► The current function of the class
► The current pointer of the class
► **The current object of the class (Page 390)**
► A value of the data member

Question No: 5 (Marks: 1) - Please choose one
The statement **cin.get ();** is used to,

► Read a string from keyboard
► **Read a character from keyboard (Page 426)**
► Read a string from file
► Read a character from file

Question No: 6 (Marks: 1) - Please choose one
When we do dynamic memory allocation in the constructor of a class, then it is necessary to provide a destructor.

► **True (Page 479)**
► False

Question No: 7 (Marks: 1) - Please choose one
Overloaded new operator function takes parameter of type *size_t* and returns

► void (nothing)
► **void pointer (Page 415)**
► object pointer
► int pointer

Question No: 8 (Marks: 1) - Please choose one
The second parameter of operator functions for << and >> are objects of the class for which we are overloading these operators.

► **True (Page 453)**
► False

Question No: 9 (Marks: 1) - Please choose one

C++ is a case-sensitive language

► **True (Page 68)**
► False

Question No: 10 (Marks: 1) - Please choose one
To include code from the library in the program, such as iostream, a directive would be called up using this command.

► #include “iostream.h”
► include <iostream.h>
► include <iostream.h>
► **#include <iostream.h> (Sure)**

Question No: 11 (Marks: 1) - Please choose one
A template function must have only generic data types.

► True
► **False (Page 501)**

Question No: 12 (Marks: 1) - Please choose one
Template class can not have static variables.

► True
► **False**

Question No: 13 (Marks: 1) - Please choose one
What will be the correct syntax to assign an array named ***arr*** of 5 elements to a pointer ***ptr***?

► *ptr = arr ;
► ptr = arr ;
► *ptr = arr[5] ;
► ptr = arr[5] ;

Question No: 14 (Marks: 1) - Please choose one
What will be the correct syntax to access the value of fourth element of an array using pointer ptr?

► ptr[3]
► (ptr+3)
► *(ptr+3)
► **Both 1and 3**

Question No: 15 (Marks: 1) - Please choose one
If most significant bit of un-signed number is 1 then it represents a positive number.

► True
► **False (Page 251)**

Question No: 16 (Marks: 1) - Please choose one

If there is a symbol (& sign) used with the variable name followed by data type then it refers to _____ and if & is being used with variable name then it refers to _____.

- ► Address of variable, reference variable
- ► Reference variable, value of variable
- ► **Reference variable, address of variable (Page 367)**
- ► Address of variable, value of variable

Question No: 17 (Marks: 1) - Please choose one
We can also do conditional compilation with preprocessor directives.

- ► **True (Page 272)**
- ► False

Question No: 18 (Marks: 1) - Please choose one
The default value of a parameter can be provided inside the _____

- ► function prototype
- ► function definition
- ► **both function prototype or function definition (Page 294)**
- ► none of the given options.

Question No: 19 (Marks: 1) - Please choose one
Classes defined inside other classes are called _____ classes

- ► looped
- ► **nested (Page 492)**
- ► overloaded
- ► none of the given options.

Question No: 20 (Marks: 1) - Please choose one
What purpose do classes serve?

- ► Data encapsulation
- ► Providing a convenient way of modeling real-world objects
- ► Simplifying code reuse
- ► **All of the given options**

Question No: 21 (Marks: 1) - Please choose one
Every class contains _____.

- ► Constructor
- ► Destructor
- ► **Both a constructor and a destructor (Lecture 27)**
- ► None of the given options

Question No: 22 (Marks: 1) - Please choose one
new operator is used to allocate memory from the free store during

► Compile Time
► **Run Time**
► Link Time
► None of the given options

Question No: 23 (Marks: 1) - Please choose one
When an object of a class is defined inside another class then,

► Destructor of enclosing class will be called first
► Destructor of inner object will be called first
► Constructor and Destructor will be called simultaneously
► None of the given options

Question No: 24 (Marks: 1) - Please choose one
It is possible to define a class within another class.

► **True (Page 492)**
► False

Question No: 25 (Marks: 1) - Please choose one
New and Delete are also used with _____ and data types as well.

► **Class, Objects (Page 342)**
► Structures, Pointers
► Both Class and structures
► None of above

Question No: 26 (Marks: 1) - Please choose one
With New keyword, data types and class members are initialized with meaningful values instead of garbage.

► **True (Page 334)**
► False

FINALTERM EXAMINATION
Spring 2010
CS201- Introduction to Programming

Question No: 1 (Marks: 1) - Please choose one
If it is required to copy an array to another array then,

► **Both arrays must be of the same size and data type (Page 105)**
► Both arrays may be of different size
► Both arrays may be of different data type
► Both arrays may be of different size and type

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

Question No: 2 (Marks: 1) - Please choose one
Dealing with structures and functions passing by reference is the most economical method

► **True**
► False
http://mathbits.com/MathBits/CompSci/Structures/Functions.htm

Question No: 3 (Marks: 1) - Please choose one
eof( ), bad( ), good( ), clear( ) all are manipulators.

► **True (Page 433)**
► False

Question No: 4 (Marks: 1) - Please choose one
Overloaded new operator function takes parameter of type *size_t* and returns

► void (nothing)
► **void pointer (Page 415)**
► object pointer
► int pointer

Question No: 5 (Marks: 1) - Please choose one
When new operator is overloaded at global level then corresponding built-in new operator will not be visible to whole of the program.

► **True (Page 413)**
► False

Question No: 6 (Marks: 1) - Please choose one
If there is more than one statement in the block of a for loop, which of the following must be placed at the beginning and the ending of the loop block?

► parentheses ( )
► **braces { } (Page 37)**
► brackets [ ]
► arrows < >

Question No: 7 (Marks: 1) - Please choose one
The return type of a function that do not return any value must be _____

► float
► int
► **void (Page 79)**
► double

Question No: 8 (Marks: 1) - Please choose one
UNIX has been developed in _____ language.

► JAVA
► B
► **C (Page 12)**
► FORTRAN

Question No: 9 (Marks: 1) - Please choose one
Like member functions, _____ can also access the private data members of a class.

► Non-member functions
► **Friend functions (Page 346)**
► Any function outside class
► None of the given options

Question No: 10 (Marks: 1) - Please choose one
Which of the following statement is best regarding declaration of friend function?

► Friend function must be declared after public keyword.
► Friend function must be declared after private keyword.
► Friend function must be declared at the top within class definition.
► **It can be declared anywhere in class as these are not affected by the public and private keywords. (Page 346)**

Question No: 11 (Marks: 1) - Please choose one
The operator function overloaded for an Assignment operator (=) must be

► Non-member function of class
► **Member function of class (Page 372)**
► Friend function of class
► None of the given options

Question No: 12 (Marks: 1) - Please choose one
The **endl** and **flush** are _____

► Functions
► Operators
► **Manipulators (Page 434)**
► Objects

Question No: 13 (Marks: 1) - Please choose one
If a symbolic constant has been defined, it will be an error to define it again.

► **True (Page 273) 15**
► False

Question No: 14 (Marks: 1) - Please choose one
The operator used for casting, in C, is standard _____ operator.

► none of the given options.
► **cast (Page 281)**
► cost
► const

Question No: 15 (Marks: 1) - Please choose one
Constructors can not be overloaded like ordinary functions.

► True
► **False (Page 323)**

Question No: 16 (Marks: 1) - Please choose one

Which of the following function call is correct for the function prototype?
defaultParameters (int a, int b = 7, char z = '*');

► **defaultParameters (5);**
► defaultParameters (5, '8');
► defaultParameters (6, '#');
► defaultParameters (0, 0, '*', 0);

Question No: 17 (Marks: 1) - Please choose one
When an operator function is defined as member function for a binary Plus (+) operator then the number of argument it take is/are.

► Zero
► One
► **Two (Page 371)**
► N arguments

Question No: 18 (Marks: 1) - Please choose one
We can not define a variable of user-defined data type in the class.

► True
► **False (Page 320)**

Question No: 19 (Marks: 1) - Please choose one
When an object of a class is defined inside an other class then,

► **Constructor of enclosing class will be called first (Page 464)**
► Constructor of inner object will be called first
► Constructor and Destructor will be called simultaneously
► None of the given options

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

Question No: 20 (Marks: 1) - Please choose one
The appropriate data type to store the number of rows and colums of the matrix is_____.

- ► float
- ► **int (Not sure)**
- ► char
- ► none of the given options.

Question No: 21 (Marks: 1) - Please choose one
Class is a user defined_____.

- ► **data type (page 317)**
- ► memory referee
- ► value
- ► none of the given options.

Question No: 22 (Marks: 1) - Please choose one
A pointer variable can be,

- ► Decremented
- ► Incremented
- ► Multiplied
- ► **Both Iecremented and Decremented (Page 366)**

Question No: 23 (Marks: 1) - Please choose one

NULL value has been defined in _____ and _____ header files.

- ► strings.h and iostream.h
- ► ctype.h and conio.c
- ► conio.c and conio.h
- ► **stdlib.h and stddef.h (Page 281)**

Question No: 24 (Marks: 1) - Please choose one

A Matrix can be composed of ints, floats or doubles as their elements. Best way is to handle this , _____

- ► Write a separate class to handle each
- ► **Use templates (Page 527)**
- ► Use strings to store all types
- ► None of the given options

Question No: 25 (Marks: 1) - Please choose one
"setprecision" manipulator will set

► **The number of digits after the decimal point (Page 440)**
► The number of digits before the decimal point
► The number of digits in a number
► None of the given options

Question No: 26 (Marks: 1) - Please choose one
Which of the following option will be true, if we overload "-=" operator?

► only - operator needs to be overloaded
► Minus (-) and = operators need to be overloaded
► **the -= operator need to be overloaded explicitly**
► the - and = operators need to be overloaded implicitly

Fall 2009
CS201- Introduction to Programming

Question No: 1 (Marks: 1) - Please choose one
In C/C++ if we define an array of size eight (8) i.e. int Arr [8]; then the last element of this array will be stored at,

► Arr[0]
► Arr[8]
► **Arr[7]**
► Arr[-1]

Question No: 2 (Marks: 1) - Please choose one
When an array is passed to a function then default way of passing this array is,

► By data
► **By reference (Page 154)**
► By value
► By data type

Question No: 3 (Marks: 1) - Please choose one
Array is a data structure which store

► Memory addresses
► Variables
► **Data Type (Page 100)**
► Data

Question No: 4 (Marks: 1) - Please choose one
We can also create an array of user define data type.

► **True (Page 408)**
► False

Question No: 5 (Marks: 1) - Please choose one
When we define an array of objects then,

► Destructor will call once for whole array
► **Destructor will call for each object of the array**
► Destructor will never call
► Depends on the size of array

Question No: 6 (Marks: 1) - Please choose one
What is the sequence of event(s) when allocating memory using new operator?

► Only block of memory is allocated for objects
► Only constructor is called for objects
► **Memory is allocated first before calling constructor (Page 414)**
► Constructor is called first before allocating memory

Question No: 7 (Marks: 1) - Please choose one
We can delete an array of objects without specifying [] brackets if a class is not doing dynamic memory allocation internally.

► True
► **False**

Question No: 8 (Marks: 1) - Please choose one
The second parameter of operator functions for << and >> are objects of the class
For which we are overloading these operators.

► **True (Page 453)**
► False

Question No: 9 (Marks: 1) - Please choose one
Which of the following is correct way to initialize a variable x of int type with value 10?

► int x ; x = 10 ;
► **int x = 10 ; (sure)**
► int x, x = 10;
► x = 10 ;

Question No: 10 (Marks: 1) - Please choose one
Default mechanism of function calling in case of array is _____ and in case of variable is ___.

► Call by value, call by reference
► Call by referene, call by reference
► **Call by reference, call by value (Page 119)**
► Call by value, call by value

Question No: 11 (Marks: 1) - Please choose one
What does STL stand for?
► Source template library

► **Standard template library (Page 519)**
► Stream template library
► Standard temporary library

Question No: 12 (Marks: 1) - Please choose one
Skill(s) that is/are needed by programmer's _____.

► Paying attention to detail
► Think about the reusability
► Think about user interface
► **All of the given options (Page 4 & 5)**

Question No: 13 (Marks: 1) - Please choose one
For which array, the size of the array should be one more than the number of elements in an array?

► int
► double
► float
► **char (Page 113)**

Question No: 14 (Marks: 1) - Please choose one
new and delete are _____ whereas malloc and free are _____.

► Functions, operators
► Classes, operators
► **Operators, functions (Page 342-285)**
► Operators, classes

Question No: 15 (Marks: 1) - Please choose one
Friend functions are _____ of a class.

► Member functions
► Public member functions
► Private member functions
► **Non-member functions (Page 346)**

Question No: 16 (Marks: 1) - Please choose one
The prototype of friend functions must be written ____ the class and its definition must be written ____

► inside, inside the class
► **inside, outside the class (Page 346)**
► **outside, inside the class**
► outside, outside the class

Question No: 17 (Marks: 1) - Please choose one
If overloaded plus operator is implemented as non-member function then which of the following statement will be true for the statement given below?
obj3 = obj1 + obj2 ;
► obj2 will be passed as an argument to + operator whereas obj2 will

drive the + operator
► obj1 will drive the + operator whereas obj2 will be passed as an argument to + operator
► **Both objects (obj1, obj2) will be passed as arguments to the + operator (not sure)**
► Any of the objects (obj1, obj2) can drive the + operator

Question No: 18 (Marks: 1) - Please choose one
Which one of the following is the declaration of overloaded pre-increment operator implemented as member function?

► Class-name operator +() ;
► Class-name operator +(int) ;
► **Class-name operator ++() ;** **(page 389)**
► **Class-name operator ++(int) ;**

Question No: 19 (Marks: 1) - Please choose one
For cin, the source is normally a _____ and destination can be _____.

► File, native data type
► Disk, user-define type
► **Keyboard, variable (Page 424)**
For *cin*, the source is normally keyboard and the destination can be an ordinary variable

► File, user-define type

Question No: 20 (Marks: 1) - Please choose one
We can also do conditional compilation with preprocessor directives.

► **True (Page 272)**
► False

Question No: 21 (Marks: 1) - Please choose one
The programs, in which we allocate static memory, run essentially on _____

► Heap
► System Cache
► None of the given options
► **Stack (Page 280)**
The programs, in which we allocate static memory, run essentially on stack.

Question No: 22 (Marks: 1) - Please choose one
The default value of a parameter can be provided inside the _____

► function prototype
► **function definition (Page 249)**
► both function prototype or function definition
► none of the given options.

The default value of a parameter is provided inside the function prototype or function definition.

Question No: 23 (Marks: 1) - Please choose one

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

While calling function, the arguments are assigned to the parameters from _____.

► **left to right. (page 295)**
► right to left
► **no specific order is followed**
► none of the given options.

Question No: 24 (Marks: 1) - Please choose one
When an operator function is defined as member function for a binary Plus (+) operator then the number of argument it take is/are.

► Zero
► One
► **Two (Page 371)**
► **N arguments**

Question No: 25 (Marks: 1) - Please choose one
With user-defined data type variables (Objects), self assignment can produce _____.

► Syntax error
► **Logical error**
► Link error
► **Non of the given options (not sure)**

Question No: 26 (Marks: 1) - Please choose one
Assignment operator is used to initialize a newly declared object from existing object.

► **True (Page 479)**
► False

Question No: 27 (Marks: 1) - Please choose one
When an object of a class is defined inside an other class then,

► **Constructor of enclosing class will be called first (Page 496)**
► Constructor of inner object will be called first
► Constructor and Destructor will be called simultaneously
► None of the given options

Question No: 28 (Marks: 1) - Please choose one
In the member initializer list, the data members are initialized,

► From left to right
► From right to left
► **In the order in which they are defined within class**
► None of the given options

Question No: 29 (Marks: 1) - Please choose one
new operator allocates memory from free store and return _____.

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

► **A pointer**
► A reference
► An integer
► A float

Question No: 30 (Marks: 1) - Please choose one
"new" and "delete" keywords are _____ in C++ language.

► Built-in- Function
► Operators
► **Memory Allocation Function (Page 332)**
► None of the given options

Spring 2010
CS201- Introduction to Programming

Question No: 1 (Marks: 1) - Please choose one
&& is ---------- operator.

► An arithmetic
► **Logical (Page 40)**
► Relational
► Unary

Question No: 2 (Marks: 1) - Please choose one
Which of the following function(s) is/are included in **ctype.h** header file?

► isdigit(int c)
► isxdigit(int c)
► tolower(int c)
► **All of the above (Page 188)**

Question No: 3 (Marks: 1) - Please choose one
Disks are _____ devices having access time of _____ milliseconds.

► Electro-physical, 6
► Electro-Mechanical, 4
► Electro-physical, 5
► **Electro-Mechanical, 7 (Page 427)**
http://www.tns.com/taocs_dp.asp

Question No: 4 (Marks: 1) - Please choose one
All preprocessor directives are started with the symbol_____.

► *

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

► ++
► @
► # (Page 269)

Question No: 5 (Marks: 1) - Please choose one
When we use manipulators in our program then which header file should be included?

► iostream.h
► stdlib.h
► stdio.h
► **iomanip.h (Page 433)**

Question No: 6 (Marks: 1) - Please choose one
setprecision is a parameter less manipulator.

► True
► **False (Page 439)**

Question No: 7 (Marks: 1) - Please choose one
Which of the following is NOT a preprocessor directive?

► #error
► #define
► #line
► **#ndefine (Page 272)**

Question No: 8 (Marks: 1) - Please choose one
We can do arithmetic with references like pointers.

► True
► **False (Page 369)**

Question No: 9 (Marks: 1) - Please choose one
What does **(*this)** represents?

► The current function of the class
► The current pointer of the class
► **The current object of the class (Page 390)**
► A value of the data member

Question No: 10 (Marks: 1) - Please choose one
Friend function of a class is _____ .

► Member function
► **Non-member function (Page 348)**
► Private function
► Public function

Question No: 11 (Marks: 1) - Please choose one
A function can declare itself a friend of a class.

► True
► **False (Page 346)**

Question No: 12 (Marks: 1) - Please choose one
Overloaded delete operator function takes the same parameter as an argument returned by new operator function.

► **True (Page 414)**
► False

Question No: 13 (Marks: 1) - Please choose one
The second parameter of operator functions for << and >> are objects of the class for which we are overloading these operators.

► **True (Page 453)**
► False
The second parameter to *operator* << is an object of the class that we are overloading the operator for. Similar is the case for *operator* >>

Question No: 14 (Marks: 1) - Please choose one
A template function must have at least ---------- generic data type

► Zero
► **One (Page 499)**
► Two
► Three

Question No: 15 (Marks: 1) - Please choose one
A template function must have only generic data types.

► True
► **False (Page 501)**

Question No: 16 (Marks: 1) - Please choose one
We can not make a member function of a class as template function.

► True
► **False (Page 506)**

Question No: 17 (Marks: 1) - Please choose one
Which character is inserted at the end of string to indicate the end of string?

► new line
► tab
► **null (Page 113)**
► carriage return

Question No: 18 (Marks: 1) - Please choose one
is used for _____.

► calculation
► reading
► **assigning value to variables (Page 372)**
► None of the given options.

Question No: 19 (Marks: 1) - Please choose one vuzs
The object _____may be used both for file input and file output

► **fstream, (Page 199)**
► ifstream,
► ofstream,
► none of the given options.

Question No: 20 (Marks: 1) - Please choose one
Which of the following function calling mechanism is true for the function prototype given below?

float func(float &);
► Call by value
► Call by reference using pointer
► **Call by reference using reference variable**
► None of the given options

Question No: 21 (Marks: 1) - Please choose one
The programs, in which we allocate static memory, run essentially on _____

► Heap
► System Cache
► None of the given options
► **Stack (Page 280)**

Question No: 22 (Marks: 1) - Please choose one
Overloaded delete operator function takes parameter of void pointer and returns
_____.

► **void**
► void pointer
► pointer to an object
► pointer of type int

Question No: 23 (Marks: 1) - Please choose one
What should be the return type of the constructor?

► void pointer
► int
► same as object type
► **constructors do not return any thing (Page 313)**

Question No: 24 (Marks: 1) - Please choose one
It is a way of reusing the code when we contain objects of our already written classes into a new class,

► **True**

► False

Question No: 25 (Marks: 1) - Please choose one vuzs
Templates are not type safe.

► true
► **false (Page 518)**

Question No: 26 (Marks: 1) - Please choose one
The functions used for dynamic memory allocation return pointer of type _____

► int
► float
► **void**
► double

FINALTERM EXAMINATION
Fall 2008
CS201- Introduction to Programming

Question No: 1 (Marks: 1) - Please choose one
There are mainly ---------- types of software

► Two (Page 9)
► Three
► Four
► Five

Question No: 2 (Marks: 1) - Please choose one
seekg() and write() are functionally _____ .

► Different
► Identical
► Two names of same function
► None of the above

Question No: 3 (Marks: 1) - Please choose one

When a pointer is incremented, it actually jumps the number of memory addresses

► **According to data type (Page 160)**
► 1 byte exactly
► 1 bit exactly
► A pointer variable can not be incremented

Question No: 4 (Marks: 1) - Please choose one
setw is a parameterized manipulator.

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

► **True (Page 437)**
► False

Question No: 5 (Marks: 1) - Please choose one
eof( ), bad( ), good( ), clear( ) all are manipulators.

► **True (Page 433)**
► False

Question No: 6 (Marks: 1) - Please choose one
In functions that return reference, use _____variables.

► Local
► Global
► **Global or static (Page 369)**
► None of the given option

Question No: 7 (Marks: 1) - Please choose one
The declarator of Plus (+) member operator function is

► **Class-Name operator + (Class-Name rhs) (Page 371)**
► operator Class-Name + ( )
► operator Class-Name + (rhs)
► Class-Name operator + ( )

Question No: 8 (Marks: 1) - Please choose one
The compiler does not provide a copy constructor if we do not provide it.

► True
► **False (Page 476)**

Question No: 9 (Marks: 1) - Please choose one
What is the functionality of the following syntax to delete an array of 5 objects named ***arr*** allocated using new operator?
delete arr ;

► Deletes all the objects of array
► **Deletes one object of array (Page 411)**
► Do not delete any object
► Results into syntax error
This statement will call the destructor only for the object pointed by the ***arr*** and deallocate the space allocated to this object

Question No: 10 (Marks: 1) - Please choose one
What is the sequence of event(s) when allocating memory using new operator?

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

► Only block of memory is allocated for objects
► Only constructor is called for objects
► **Memory is allocated first before calling constructor (Page 414)**
► Constructor is called first before allocating memory

Question No: 11 (Marks: 1) - Please choose one
What is the sequence of event(s) when deallocating memory using delete operator?

► Only block of memory is deallocated for objects
► Only destructor is called for objects
► Memory is deallocated first before calling destructor
► **Destructor is called first before deallocating memory (Page 414)**

Question No: 12 (Marks: 1) - Please choose one
new and **delete** operators cannot be overloaded as member functions.

► True
► **False (Page 414)**

Question No: 13 (Marks: 1) - Please choose one
The operator function of << and >> operators are always the member function of a class.

► True
► **False (Page 446)**

Question No: 14 (Marks: 1) - Please choose one
A template function must have at least ---------- generic data type

► Zero
► **One (Page 499)**
► Two
► Three

Question No: 15 (Marks: 1) - Please choose one
If we do not mention any *return_value_type* with a function, it will return an _____ value.

► **int (Page 79)**
► void
► double
► float

Question No: 16 (Marks: 1) - Please choose one
Suppose a program contains an array declared as **int arr[100];** what will be the size of array?

► 0
► 99
► **100 (Page 103)**
► 101

Question No: 17 (Marks: 1) - Please choose one
The name of an array represents address of first location of array element.

► **True**
► False
The name of the array is a constant pointer which contains the memory is the address of first element of the array

Question No: 18 (Marks: 1) - Please choose one
Reusing the variables in program helps to save the memory

► **True**
► False

Question No: 19 (Marks: 1) - Please choose one
Which of the following option is true about new operator to dynamically allocate memory to an object?

► The new operator determines the size of an object
► Allocates memory to object and returns pointer of valid type
► Creates an object and calls the constructor to initialize the object
► **All of the given options**

Question No: 20 (Marks: 1) - Please choose one
New and delete are _____ whereas malloc and free are _____.

► Functions, operators
► Classes, operators
► **Operators, functions (Page 342-285)**
► Operators, classes

Question No: 21 (Marks: 1) - Please choose one
Like member functions, _____ can also access the private data members of a class.

► Non-member functions
► **Friend functions (Page 346)**
► Any function outside class
► None of the given options

Question No: 22 (Marks: 1) - Please choose one
Which of the following statement is best regarding declaration of friend function?

► Friend function must be declared after public keyword.
► Friend function must be declared after private keyword.
► Friend function must be declared at the top within class definition.
► **It can be declared anywhere in class as these are not affected by the public and private keywords. (Page 346)**

Question No: 23 (Marks: 1) - Please choose one
The operator function overloaded for an Assignment operator (=) must be

- ► Non-member function of class
- ► **Member function of class (Page 372)**
- ► Friend function of class
- ► None of the given options

Question No: 24 (Marks: 1) - Please choose one
For non-member operator function, object on left side of the operator may be

- ► Object of operator class
- ► Object of different class
- ► Built-in data type
- ► **All of the given options (Page 375)**

Question No: 25 (Marks: 1) - Please choose one
The operator function will be implemented as _____, if obj1 drive the - operator whereas obj2 is passed as arguments to - operator in the statement given below.
obj3 = obj1 - obj2;

- ► **Member function (Page 373)**
- ► Non-member function
- ► Friend function
- ► None of the given options

Question No: 26 (Marks: 1) - Please choose one
Which one of the following is the declaration of overloaded pre-increment operator implemented as member function?

- ► Class-name operator +() ;
- ► Class-name operator +(int) ;
- ► **Class-name operator ++() ; (Page 389)**
- ► Class-name operator ++(int) ;

Question No: 27 (Marks: 1) - Please choose one
The static data members of a class are initialized _____

- ► **at file scope (Page 466)**
- ► within class definition
- ► within member function
- ► within main function

Question No: 28 (Marks: 1) - Please choose one
Class is a user defined_____.

- ► **data type (Page 317)**

- memory referee
- value
- none of the given options.

Question No: 29 (Marks: 1) - Please choose one
We can also define a user-defines manipulators.

- **True**
- False

Parameterized manipulators require one or more arguments. setfill (near the bottom of the iomanip.h header file) is an example of a parameterized manipulator. You can create your own parameterized manipulators and your own simple manipulators.

Question No: 30 (Marks: 1) - Please choose one
Automatic variable are created on _____.

- Heap
- Free store
- static storage
- **stack (Page 461)**

FINALTERM EXAMINATION
fall 2008
CS201- Introduction to Programming (Session - 2)

Question No: 1 (Marks: 1) - Please choose one
If Num is an integer variable then Num++ means,

- Add 1 two times with Num
- **Add 1 with Num**
- Add 2 with Num
- Subtract 2 from Num

Question No: 2 (Marks: 1) - Please choose one
If the return type of a function is void then it means that it will,

- Return any type of data
- Return some specific type of data
- **Return no data**
- Return just character data

Question No: 3 (Marks: 1) - Please choose one
Which of the following is a valid class declaration?

- **class A { int x; };**
- class B { }
- public class A { }
- object A { int x; };

Question No: 4 (Marks: 1) - Please choose one

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

When we use manipulators in our program then which header file should be included?

- iostream.h
- stdlib.h
- stdio.h
- **iomanip.h (Page 433)**

Question No: 5 (Marks: 1) - Please choose one
We can also create an array of user define data type.

- **True (Page 408)**
- False

Question No: 6 (Marks: 1) - Please choose one
The normal source of **cin** object is,

- File
- Disk
- **Keyboard (Page 424)**
- RAM

Question No: 7 (Marks: 1) - Please choose one
A stream is an ordered sequence of bytes.

- **True (Page 423)**
- False

Question No: 8 (Marks: 1) - Please choose one
What is the sequence of event(s) when allocating memory using new operator?

- Only block of memory is allocated for objects
- Only constructor is called for objects
- **Memory is allocated first before calling constructor (Page 414)**
- Constructor is called first before allocating memory

Question No: 9 (Marks: 1) - Please choose one
We can delete an array of objects without specifying [] brackets if a class is not doing dynamic memory allocation internally.

- **True (Page 412)**
- False

Question No: 10 (Marks: 1) - Please choose one
The second parameter of operator functions for << and >> are objects of the class for which we are overloading these operators.

- **True (Page 453)**
- False

Question No: 11 (Marks: 1) - Please choose one
Which looping process checks the test condition at the end of the loop?

- for
- while
- **do while**
- no looping process checks the test condition at the end

http://mathbits.com/MathBits/CompSci/looping/MCLooping.htm

Question No: 12 (Marks: 1) - Please choose one
In a group of nested loops, which loop is executed the most number of times?

- the outermost loop
- **the innermost loop**
- all loops are executed the same number of times
- cannot be determined without knowing the size of the loops

http://mathbits.com/mathbits/java/Looping/MCLooping.htm

Question No: 13 (Marks: 1) - Please choose one
Template class can not have static variables.

- True
- **False (Page 513)**

Question No: 14 (Marks: 1) - Please choose one
Consider the following statements to initialize a two-dimensional array.

- int arr[2][3] = {4, 8, 9, 2, 1, 6} ;
- int arr[3][2] = {4, 8, 9, 2, 1, 6} ;
- int arr[][2] = {{4,8},{9, 2},{1, 6}} ;

Which of the following option(s) are correct to initialize a two-dimensional array with 3 rows and 2 columns?

- (ii) only
- (iii) only
- **(ii) and (iii)**
- and (iii)

Question No: 15 (Marks: 1) - Please choose one
There is a pointer variable named ***ptr*** of type int then address of which type of variable the ***ptr*** will store in it?

- variable of type char
- variable of type short
- **variable of type int**
- variable of type double

Question No: 16 (Marks: 1) - Please choose one
Let suppose
Union intorDouble{
Int ival;
Double charvar;
};
main(){

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

intorDouble VAZ;
int size ;
size = sizeof(VAZ);
}
What will be the value of variable "size", if int occupies 4 bytes and double occupies 8 bytes?

- 2
- 4
- 8
- 12

Question No: 17 (Marks: 1) - Please choose one
new and delete are _____ whereas malloc and free are _____.

- Functions, operators
- Classes, operators
- **Operators, functions (Page 342-285)**
- Operators, classes

Question No: 18 (Marks: 1) - Please choose one
The member functions of a class occupy _____ region in memory for ____ object(s) of class.

- separate, each
- **common, all (Page 334)**
- different, each
- different, all

Question No: 19 (Marks: 1) - Please choose one
Friend functions are _____ of a class.

- Member functions
- Public member functions
- Private member functions
- **Non-member functions (Page 346)**

Question No: 20 (Marks: 1) - Please choose one
Which of the following is true while overloading operators?

- Precedence of an operator can be changed
- The arity (number of operands) can be changed
- No new operators can be created
- Associativity of an operator can be changed

Question No: 21 (Marks: 1) - Please choose one
Which of the following option will be true to overload the -= operator?

- **only - operator needs to be overloaded (not sure)**
- Minus (-) and = operators need to be overloaded
- the -= operator need to be overloaded explicitly
- the - and = operators need to be overloaded implicitly

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

Question No: 22 (Marks: 1) - Please choose one
The input/output streams; cin and cout are ____

- Operators
- **Functions**
- Objects
- Structures

Question No: 23 (Marks: 1) - Please choose one
dec, hex, oct are all _____

- Member functions
- Objects of input/output streams
- Parameterized manipulators
- **Non-parameterized manipulators (Page 435)**

Question No: 24 (Marks: 1) - Please choose one
What will be the output of the following statement?
cout << setbase(16) << 52 ;

- 74
- 52
- **34**
- 64

Question No: 25 (Marks: 1) - Please choose one
The first parameter of overloaded stream insertion operator is _____ where second parameter is _____

- input stream, object of class
- object of class, output stream
- **output stream, object of class**
- object of class, input stream

Question No: 26 (Marks: 1) - Please choose one
We can also do conditional compilation with preprocessor directives.

- **True (Page 272)**
- False

Question No: 27 (Marks: 1) - Please choose one
With user-defined data type variables (Objects), self assignment can produce _____.

- Syntax error
- **Logical error**
- Link error
- Non of the given options

Question No: 28 (Marks: 1) - Please choose one
The return type of the operator function for >> operator is _____.

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

- **class for which we overload this operator**
- reference of ostream class (ostream&)
- reference of istream class (istream&)
- void

Question No: 29 (Marks: 1) - Please choose one
When an object of a class is defined inside an other class then,

- **Constructor of enclosing class will be called first (Page 496)**
- Constructor of inner object will be called first
- Constructor and Destructor will be called simultaneously
- None of the given options

Question No: 30 (Marks: 1) - Please choose one
Where we can include a header file in the program?

- **any where (Page 270)**
- in start
- at the end
- none of the given options.

Moaaz.pk@gmail.com

bc100400662asad@gmail.com